

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2024

Tập luận văn đội tuyển quốc gia Olympic Tin học

China Computer Federation, 2024

Nguồn gốc: Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2024

IOI China candidate team papers / National training team 2024 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2024. Tập được dàn trang bằng Adobe InDesign; phần bìa và mục lục có lớp văn bản đọc được, nhưng nhiều trang thân bài trích xuất thành mojibake và sinh cảnh báo phong chữ. Bản LaTeX này chuyển ngữ phần nhận diện tập sách và toàn bộ mục lục, đồng thời dịch bảy bài đầu tiên: bài của Zhou Kangyang về bài toán tính tổng hàm nhân tính, bài của Guo Yuchong về parity của matroid tuyến tính, bài của Huang Luotian về hợp nhất–tách cấu trúc dữ liệu, bài của Shi Kaicheng về nhân tử Lagrange và đối ngẫu trong OI, bài của Shen Jiyu về cập nhật đoạn và truy vấn đoạn, bài của Cao Li về một số phương pháp giải bài toán quy hoạch động trong OI, và bài của Feng Zhengwei về các cách xử lý hợp nhất thông tin, dựa trên OCR và đối chiếu ảnh trang PDF.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2024*. Tập PDF gồm 424 trang và được tạo bằng Adobe InDesign. Phần văn bản trích xuất được không ghi riêng tên huấn luyện viên.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả
1	Một số tiến triển về bài toán tính tổng hàm nhân tính	Zhou Kangyang
11	Bàn về bài toán parity của matroid tuyến tính	Guo Yuchong
24	Bàn về hợp nhất và tách cấu trúc dữ liệu	Huang Luotian
37	Bàn về nhân tử Lagrange và đối ngẫu trong OI	Shi Kaicheng
54	Thuật toán liên quan tới bài toán cập nhật đoạn và truy vấn đoạn cùng ứng dụng	Shen Jiyu
69	Bàn về một số phương pháp giải bài toán quy hoạch động trong OI	Cao Li
85	Bàn về một số cách xử lý hợp nhất thông tin	Feng Zhengwei
100	Một số phương pháp cho bài toán đếm thứ tự topo trên cây	Chen Xinyang
121	Bàn về tối ưu hằng số trên phần cứng hiện đại	Song Jiaxing
171	Bàn về ứng dụng của một lớp phương pháp bao hàm–loại trừ và nghịch đảo	Dong Yiqi
182	Một lớp kỹ thuật dùng hàm sinh để mô tả thời điểm dừng của quá trình ngẫu nhiên	Ye Kai
196	Bàn về bài toán đồ thị đường trong thi tin học	Zhai Jincheng
206	Bàn về một lớp bài toán đoạn trên cây	Zhu Yifan
217	Báo cáo ra đề <i>Thế giới chìm trong cổ tích ngủ say</i>	Ye Liqi
230	Bài toán matching trên đồ thị có trọng số đỉnh	Ke Yisi
253	Bàn lại về đếm đường đi trên lưới	Wu Chang
277	Ứng dụng liên phân số và thuật toán Euclid trong OI	Ke Yijing
299	Bàn về độ phức tạp và ứng dụng của nó trong giải bài toán	Yang Minxing
315	Bàn về thuật toán duy trì một loại cặp số nguyên tố cùng nhau và ước chung lớn nhất	Du Guancheng
321	Bàn về lý thuyết tính toán và các bài toán khó trong OI	Yan Chenxiao
336	Bàn về ứng dụng thuật toán lattice trong phân tích đa thức hệ số nguyên	Xie Zihan
358	Bàn về bài toán tô màu danh sách của đồ thị	Zhong Zihou
367	Từ <i>Kén</i> : phân tích phương pháp quan sát trong thi tin học qua ví dụ	Li Jiayou
376	Bàn về q -analogue	Li Mingleyang
400	Bàn về chu trình tỉ lệ nhỏ nhất và ứng dụng	Wei Jiaze

Bài 1. Một số tiến triển về bài toán tính tổng hàm nhân tính

Zhou Kangyang, Trường Văn Uyên thuộc Tập đoàn Giáo dục Trung học Học Quân, Hàng Châu

Tóm tắt

Tính tổng hàm nhân tính là một lớp bài toán quan trọng trong số học. Với bài toán này, các nghiên cứu trước đây chủ yếu hướng tới giảm số lượng thừa số log trong độ phức tạp $\tilde{O}(n^{2/3})$.¹ Sau khi suy nghĩ, tác giả nhận thấy có thể đạt độ phức tạp thời gian $O(n^{1/2} \text{poly log } n)$.²

Bài viết này giới thiệu thuật toán đó và các tối ưu của nó.

¹Một nghiên cứu liên quan: <https://negiizhao.blog.uoj.ac/blog/7165>.

²Bản đầu được viết ngày 2023-07-11 và công bố tại <https://www.cnblogs.com/zkyJuruo/p/17544928.html>.

1. Kiến thức chuẩn bị

1.1. Định nghĩa và quy ước

Định nghĩa 1.1. Hàm số học là hàm có miền xác định là tập số nguyên dương và miền giá trị là tập số phức.

Với một hàm số học f , ta quy ước

$$S_f(n) = \sum_{i=1}^n f(i).$$

Định nghĩa 1.2. Hàm nhân tính là một hàm số học $f(n)$ xác định trên các số nguyên dương, thỏa

$$f(1) = 1, \quad \gcd(a, b) = 1 \Rightarrow f(ab) = f(a)f(b).$$

Với một hàm nhân tính f , chỉ cần biết mọi giá trị $f(p^k)$ với $p \in \mathbb{P}, k \in \mathbb{Z}^+$, ta có thể xác định toàn bộ hàm f . Một số hàm nhân tính thường gặp có giá trị trên p^k như sau:

- hàm ϵ (hàm đơn vị), thỏa $\epsilon(p^k) = 0$;
- hàm μ , thỏa $\mu(p^k) = -[k = 1]$;
- hàm φ , thỏa $\varphi(p^k) = (p - 1)p^{k-1}$;
- hàm I , thỏa $I(p^k) = 1$;
- hàm id_t , thỏa $\text{id}_t(p^k) = p^{kt}$.

Định nghĩa 1.3. Với hai hàm số học f, g , định nghĩa tích chập Dirichlet của chúng là

$$(f * g)(x) = \sum_{d|x} f(d)g\left(\frac{x}{d}\right).$$

Nếu xem tích chập Dirichlet là phép nhân của các hàm số học, và phép cộng trực tiếp của hàm là phép cộng, thì phép cộng và phép nhân trên các hàm số học thỏa giao hoán, kết hợp và phân phối.

Định lý 1.1. Tích chập Dirichlet h của hai hàm nhân tính f, g cũng là một hàm nhân tính.

Chứng minh. Chỉ cần chứng minh với mọi $\gcd(x, y) = 1$ ta có $h(xy) = h(x)h(y)$:

$$\begin{aligned} h(xy) &= \sum_{d|xy} f(d)g\left(\frac{xy}{d}\right) \\ &= \sum_{u|x, v|y} f(uv)g\left(\frac{xy}{uv}\right) \\ &= \sum_{u|x} f(u)g\left(\frac{x}{u}\right) \sum_{v|y} f(v)g\left(\frac{y}{v}\right) \\ &= h(x)h(y). \end{aligned}$$

1.2. Powerful Number

Định nghĩa 1.4. Với một số nguyên dương m , nếu với mọi thừa số nguyên tố p của m đều có $p^2 | m$, thì gọi m là một *Powerful Number*.

Trong phần sau, ta viết tắt *Powerful Number* là PN. Trong công thức, PN biểu thị tập tất cả các PN.

Định lý 1.2. Mọi PN đều có thể biểu diễn dưới dạng a^2b^3 , trong đó a, b là các số nguyên dương.

Chứng minh. Giả sử

$$x = \prod_i p_i^{\alpha_i}$$

là một PN. Chỉ cần biểu diễn từng $p_i^{\alpha_i}$ dưới dạng $a_i^2b_i^3$, rồi lấy $a = \prod_i a_i, b = \prod_i b_i$.

Nếu α_i lẻ, lấy

$$a_i = p_i^{(\alpha_i-3)/2}, \quad b_i = p_i;$$

nếu α_i chẵn, lấy

$$a_i = p_i^{\alpha_i/2}, \quad b_i = 1.$$

Định lý 1.3. Số lượng PN không vượt quá n là $O(\sqrt{n})$.

Chứng minh. Biểu diễn mọi PN không vượt quá n dưới dạng a^2b^3 . Khi liệt kê a , số PN không vượt quá

$$\sum_{a=1}^{\sqrt{n}} \left(\frac{n}{a^2} \right)^{1/3}.$$

Tổng này có thể ước lượng bởi

$$\int_1^{\sqrt{n}} \left(\frac{n}{x^2} \right)^{1/3} dx = n^{1/3} \int_1^{\sqrt{n}} x^{-2/3} dx = n^{1/3} (3n^{1/6} - 3),$$

tức ở cấp $O(\sqrt{n})$.

Sàng tuyến tính mọi số nguyên tố không vượt quá $\sqrt{n}$, rồi DFS trên các số nguyên tố sẽ tìm được mọi PN không vượt quá n . Độ phức tạp bằng số PN, tức $O(\sqrt{n})$.

Ví dụ 1.1 (phần $n = 1$ của UOJ Long Round 7, Mã kiểm tra). Cho c . Hàm nhân tính $q(x)$ thỏa

$$q(p^k) = p^{c\lfloor k/2 \rfloor}.$$

Tính $\sum_{i=1}^m q(i)$, đáp án lấy modulo 2^{32} , với $m \leq 1.2 \times 10^{11}$.

Thiết kế hàm nhân tính $f(x)$ thỏa $f(p) = q(p)$, và dựng hàm nhân tính g thỏa $g * f = q$. Với số nguyên tố p ,

$$g(p)f(1) + g(1)f(p) = q(p),$$

suy ra $g(p) = 0$. Vì vậy g chỉ có giá trị khác không tại các PN.

Ta có

$$\begin{aligned} \sum_{i=1}^m q(i) &= \sum_{i=1}^m \sum_{jk=i} g(j)f(k) \\ &= \sum_{\substack{j \in PN \\ j \leq m}} g(j) \sum_{k=1}^{\lfloor m/j \rfloor} f(k) \\ &= \sum_{\substack{j \in PN \\ j \leq m}} g(j) \left\lfloor \frac{m}{j} \right\rfloor. \end{aligned}$$

Tiền xử lý $g(p^k)$ cho từng p^k . Từ

$$q(p^k) = \sum_{i=0}^k f(p^i)g(p^{k-i})$$

có thể giải $g(p^k)$ theo thứ tự tăng dần của k , đồng thời duy trì giá trị hàm g trong lúc DFS tìm PN. Như vậy bài toán được giải trong $O(\sqrt{m})$.

1.3. Sàng Du Jiao

Với một số hàm nhân tính đặc biệt như μ và φ , ta có thể xây dựng tích chập Dirichlet để tính tổng tiền tố trong thời gian $O(n^{2/3})$. Trong cộng đồng thi tin học Trung Quốc, thuật toán kiểu này được gọi là “sàng Du Jiao”.

Với hàm μ , ta có $\mu * I = \epsilon$.

Xét bài toán tổng quát hơn. Nếu có các hàm số học A, B, C thỏa

$$A(1) = B(1) = C(1) = 1, \quad A * B = C,$$

và ta tính nhanh được các giá trị điểm của S_B, S_C , thì làm sao tính $S_A(n)$?

Chú ý rằng

$$\begin{aligned} \sum_{i=1}^n C(i) &= \sum_{i=1}^n \sum_{jk=i} A(j)B(k) \\ &= \sum_{k=1}^n B(k) \sum_{j=1}^{\lfloor n/k \rfloor} A(j) \\ &= S_A(n) + \sum_{k=2}^n B(k) S_A\left(\left\lfloor \frac{n}{k} \right\rfloor\right). \end{aligned}$$

Do đó

$$S_A(n) = S_C(n) - \sum_{k=2}^n B(k) S_A\left(\left\lfloor \frac{n}{k} \right\rfloor\right),$$

có thể tính $S_A(n)$ bằng đệ quy. Lại có

$$\left\lfloor \frac{\lfloor n/a \rfloor}{b} \right\rfloor = \left\lfloor \frac{n}{ab} \right\rfloor,$$

nên trong quá trình đệ quy, các giá trị thật sự cần tính chỉ là

$$S_A\left(\left\lfloor \frac{n}{k} \right\rfloor\right) \quad (k \in \mathbb{Z}^+).$$

Định lý 1.4. Với số nguyên dương n , tập

$$\left\{ \left\lfloor \frac{n}{i} \right\rfloor \mid i \in \mathbb{Z}^+ \right\}$$

có kích thước $O(\sqrt{n})$.

Chứng minh. Với $k \leq \sqrt{n}$, chỉ có $\sqrt{n}$ giá trị $\lfloor n/k \rfloor$, hiển nhiên là $O(\sqrt{n})$ loại. Với $k > \sqrt{n}$, ta có $\lfloor n/k \rfloor \leq \sqrt{n}$, nên cũng chỉ có $O(\sqrt{n})$ loại.

Vì vậy ta chỉ cần tính $O(\sqrt{n})$ giá trị điểm của S_A .

Khi tính một $S_A(m)$, gom các đoạn liên tiếp có cùng giá trị $\lfloor m/k \rfloor$ lại với nhau (tức chia khối theo phép chia nguyên), ta có thể đệ quy tới các bài toán con trong $O(\sqrt{m})$.

Với $m \leq n^{2/3}$, ta xử lý trước $n^{2/3}$ giá trị điểm đầu của A . Với $m > n^{2/3}$, dùng công thức đệ quy ở trên. Phần này có độ phức tạp

$$O\left(\sum_{i=1}^{n^{1/3}} \sqrt{\frac{n}{i}}\right) = O\left(\int_1^{n^{1/3}} \sqrt{\frac{n}{x}} dx\right) = O(n^{2/3}).$$

Tổng độ phức tạp là $O(n^{2/3})$ cộng với phần xử lý trước $O(n^{2/3})$ giá trị đầu của A . Với μ và φ , phần xử lý trước này đều làm được trong $O(n^{2/3})$.

Định nghĩa 1.5. Định nghĩa sàng khối của hàm số học f đối với n là các giá trị điểm của $S_f(n)$ trên tập

$$\left\{ \left\lfloor \frac{n}{i} \right\rfloor \mid i \in \mathbb{Z}^+ \right\}.$$

Trong quá trình trên, ta dùng sàng khối của B và C đối với n để tìm sàng khối của A đối với n . Hiển nhiên, nếu biết sàng khối của A và B đối với n , ta cũng có thể tìm sàng khối của C đối với n . Ta gọi C là **tích chập sàng khối** của A và B . Các phép toán trên sàng khối vẫn thỏa giao hoán, kết hợp và phân phối.

2. Tích chập sàng khối

Với các hàm số học A, B, C thỏa

$$A(1) = B(1) = C(1) = 1, \quad A * B = C,$$

sàng Du Jiao ở trên đã dùng sàng khối của B, C đối với n để giải sàng khối của A đối với n .

Ở đây trước hết xét bài toán đơn giản hơn: đã biết sàng khối của A, B đối với n , hãy tính sàng khối của $C = A * B$ đối với n . Bài toán này thực ra có thể giải trong $O(n^{1/2} \text{poly log } n)$. Trong phần dưới, mặc định sàng khối là sàng khối đối với n .

Ta cần tính các tổng tiền tố trên mọi $\lfloor n/t \rfloor$ của

$$C(z) = \sum_{xy=z} A(x)B(y).$$

Trước hết xét trường hợp $x > \sqrt{n}$; trường hợp $y > \sqrt{n}$ là đối xứng. Nếu $xy \leq \lfloor n/t \rfloor$ thì $x \leq n/(ty)$. Do đó khi liệt kê t, y , các x có đóng góp là một đoạn tiền tố. Phần này có độ phức tạp $O(\sqrt{n} \log n)$.

Như vậy chỉ còn cần xử lý trường hợp $x \leq \sqrt{n}$ và $y \leq \sqrt{n}$. Điều kiện $xy \leq n/t$ tương đương

$$\ln x + \ln y \leq \ln \left(\frac{n}{t} \right).$$

Ta làm một ước lượng như sau. Chọn một độ dài khối nguyên S với $3 \leq S \leq n$, và cho rằng

$$\ln x + \ln y \leq \ln \left(\frac{n}{t} \right)$$

khi và chỉ khi

$$\lfloor S \ln x \rfloor + \lfloor S \ln y \rfloor \leq S \ln \left(\frac{n}{t} \right).$$

Ước lượng này có thể làm bằng nhân đa thức. Thiết kế hai đa thức $F(z), G(z)$ sao cho $[z^k]F(z)$ là tổng các $A(x)$ thỏa $\lfloor S \ln x \rfloor = k$, và $[z^k]G(z)$ là tổng các $B(x)$ thỏa $\lfloor S \ln x \rfloor = k$. Tính $H(z) = F(z)G(z)$, khi đó hệ số $[z^k]H(z)$ là tổng $A(x)B(y)$ với $\lfloor S \ln x \rfloor + \lfloor S \ln y \rfloor = k$. Chỉ cần làm một phép chập độ dài $S \ln n$.

Nhưng làm như vậy hiển nhiên không đúng hoàn toàn. Nó chỉ sai khi

$$\ln x + \ln y \leq \ln \left(\frac{n}{t} \right) \quad \text{và} \quad \lfloor S \ln x \rfloor + \lfloor S \ln y \rfloor > S \ln \left(\frac{n}{t} \right).$$

Khi đó

$$S \ln x + S \ln y \in \left(S \ln \left(\frac{n}{t} \right) - 2, S \ln \left(\frac{n}{t} \right) \right],$$

nên

$$\ln x + \ln y + \ln t \in \left(\ln n - \frac{2}{S}, \ln n \right],$$

tức

$$xyt \in (ne^{-2/S}, n].$$

Trong đó $e^{-2/S}$ ở cấp $1 - O(1/S)$. Vì vậy xyt nằm trong một đoạn có độ dài $O(n/S)$.

Giả sử đoạn này là $[n - L, n]$. Dùng sàng đoạn để lấy phân tích thừa số nguyên tố của mọi số trong đoạn. Cụ thể hơn, trước hết sàng mọi số nguyên tố nhỏ không vượt quá $\sqrt{n}$. Với mỗi số nguyên tố, ta tìm được các bội của nó trong $[n - L, n]$, từ đó sàng ra mọi thừa số nguyên tố $\leq \sqrt{n}$ của các số trong đoạn. Một số có nhiều nhất một thừa số nguyên tố $> \sqrt{n}$, nên chỉ cần xét phần còn lại sau khi chia hết các số nguyên tố nhỏ.

Sau khi có phân tích thừa số nguyên tố, DFS trên các thừa số nguyên tố sẽ nhanh chóng tìm được mọi bộ (x, y, t) có thể. Với một bộ (x, y, t) , kiểm tra trong $O(1)$ xem nó có bị ước lượng sai hay không. Với một $v \in [n - L, n]$, thời gian đóng góp là $d_3(v)$, trong đó

$$d_3(v) = \sum_{xyz=v} 1.$$

Theo kết quả trong số học giải tích [1],

$$\sum_{i \leq n} d_3(i) = nP_3(\ln n) + O(n^{43/96+\epsilon}),$$

nên

$$\sum_{i=n-L}^n d_3(i) = LP_3(\ln n) + O(n^{43/96+\epsilon}),$$

trong đó P_3 là một đa thức bậc hai. Bản thân đầu ra của thuật toán đã có $O(\sqrt{n})$, lớn hơn $O(n^{43/96+\epsilon})$, nên có thể bỏ qua phần độ phức tạp này. Phần còn lại có độ phức tạp

$$O(L \log^2 n) = O\left(\frac{n}{S} \log^2 n\right).$$

Tổng độ phức tạp là

$$O\left(S \log^2 n + \frac{n}{S} \log^2 n\right).$$

Lấy $S = \sqrt{n}$ thu được độ phức tạp thời gian

$$O(\sqrt{n} \log^2 n).$$

3. Tính tổng hàm nhân tính

Với một hàm nhân tính f thỏa một số tính chất đặc biệt, chẳng hạn

$$f(p^k) = p^k(p^k - 1),$$

làm sao tính sàng khối của f ?

Trước hết, giá trị $f(p^k)$ với $k \geq 2$ không quá quan trọng. Nếu có một hàm nhân tính khác g thỏa $f(p) = g(p)$, và tổng tiền tố của g dễ tính, thì chỉ cần thiết kế hàm nhân tính h thỏa $h * g = f$. Theo kết luận ở trên, h chỉ có giá trị tại PN, và các giá trị đó có thể tính trong $O(\sqrt{n})$. Vì vậy dễ dàng tính sàng khối của h trong $O(\sqrt{n})$, rồi dùng tích chập sàng khối của h và g để tính sàng khối của f .

Ta trước hết thiết kế một hàm số học q chỉ khác không tại các số nguyên tố và thỏa $q(p) = f(p)$.

Với một hàm số học q có $q(1) = 0$, định nghĩa

$$\exp(q) = \sum_{i=0}^{\infty} \frac{q^i}{i!},$$

trong đó phép chập là tích chập sàng khối. Xét $d = \exp(q)$. Với

$$x = \prod_{i=1}^k p_i^{\alpha_i},$$

hàm $d(x)$ chỉ có đóng góp từ hạng

$$q^{\sum_{i=1}^k \alpha_i},$$

và giá trị bằng

$$\frac{\prod_{i=1}^k q(p_i)^{\alpha_i}}{(\sum_{i=1}^k \alpha_i)!} \binom{\sum_{i=1}^k \alpha_i}{\alpha_1, \alpha_2, \dots, \alpha_k} = \frac{\prod_{i=1}^k q(p_i)^{\alpha_i}}{\prod_{i=1}^k \alpha_i!}.$$

Do đó d là một hàm nhân tính và

$$d(p^k) = \frac{f(p)^k}{k!},$$

đặc biệt $d(p) = f(p)$. Như vậy, nếu tính được sàng khối của q , ta có thể dùng $\log n$ lần tích chập sàng khối (khi $i > \log n$, q^i không còn giá trị trong n hạng đầu) để tính sàng khối của d , từ đó suy ra sàng khối của f .

Làm sao tính sàng khối của q ? Xét một ý tưởng tương tự sàng Min_25. Tách hàm q thành tổng có trọng số của vài q_i , rồi dùng sàng khối của các q_i để tính sàng khối của q . Ví dụ, nếu

$$q(p) = p(p-1),$$

ta có thể tách thành

$$q_1(p) = p^2, \quad q_2(p) = p, \quad q = q_1 - q_2.$$

Khi $q(p)$ là đa thức bậc hằng theo p , ta đều có thể tách như vậy.

Làm sao tính sàng khối của q_i ? Ta làm ngược lại. Lấy $q_1(p) = p^2$ làm ví dụ. Với hàm nhân tính d thỏa

$$d(p^k) = q_1(p)^k,$$

ta có $d(x) = x^2$, nên sàng khối rất dễ tính. Với hàm nhân tính d thỏa

$$d(p^k) = \frac{q_1(p)^k}{k!},$$

do nó có cùng giá trị tại các điểm p với hàm trước, ta có thể dùng PN để chuyển đổi lẫn nhau trong độ phức tạp của một lần tích chập sàng khối. Vì vậy sàng khối của nó cũng tính được.

Hàm d thỏa $d(p^k) = q_1(p)^k/k!$ chính là $\exp(q_1)$. Điều này gợi ý dùng một ý tưởng tương tự $\ln$ để suy ngược q_1 từ d .

Với một hàm số học d có $d(1) = 1$, định nghĩa

$$\ln(d) = \sum_{i=1}^{\infty} \frac{(-1)^{i-1} (d - \epsilon)^i}{i}.$$

Ta có $q_1 = \ln(d)$.

Chứng minh. Chỉ cần lần lượt chứng minh $\exp(\ln(d)) = d$ và hàm q thỏa $\exp(q) = d$ là duy nhất.

Trước hết chứng minh $\exp(\ln(d)) = d$:

$$\exp(\ln(d)) = \sum_{j=0}^{\infty} \frac{\left(\sum_{i=1}^{\infty} \frac{(-1)^{i-1} (d - \epsilon)^i}{i} \right)^j}{j!}.$$

Dùng tính phân phối và kết hợp của sàng khối để khai triển, ta sẽ nhận được biểu thức dạng $\sum_i a_i d^i$. Có thể dùng đa thức $A(z)$ để mô tả nó, trong đó $[z^k]A(z) = a_k$. Định nghĩa của $\exp$ và $\ln$ trong tích chập sàng khối tương ứng với đa thức:

$$\exp(z) = \sum_{i=0}^{\infty} \frac{z^i}{i!}, \quad \ln(1+z) = \sum_{i=1}^{\infty} \frac{(-1)^{i-1} z^i}{i}.$$

Vì $A(z) = \exp(\ln z) = z$, nên $a_i = [i = 1]$, do đó $\exp(\ln(d)) = d$.

Nếu $\exp(q) = d$, tức

$$\epsilon + q + \sum_{i=2}^{\infty} \frac{q^i}{i!} = d,$$

ta có thể xác định lần lượt giá trị $q(i)$ theo thứ tự tăng dần. Đặt

$$p = \sum_{i=2}^{\infty} \frac{q^i}{i!},$$

khi đó $p(i)$ chỉ liên quan tới $q(j)$ với $j < i$, nên có thể dùng

$$q(i) = d(i) - [i = 1] - p(i)$$

để giải $q(i)$. Do đó q được xác định duy nhất.

Trong công thức $\ln$ ở trên cũng chỉ cần liệt kê $i \leq \log n$, vì vậy cũng chỉ cần $\log n$ lần tích chập sàng khối.

Do đó thuật toán này có độ phức tạp thời gian

$$O(\sqrt{n} \log^3 n).$$

4. Tối ưu

Dù ta đã có một thuật toán $O(n^{1/2} \text{poly log } n)$, nó vẫn quá chậm để dùng thực tế trong OI. Có tối ưu được không?

Dưới đây là một số hướng tối ưu đơn giản, có thể hạ độ phức tạp thời gian của thuật toán xuống

$$O\left(\frac{\sqrt{n} \log^2 n}{\sqrt{\log \log n}}\right).$$

4.1. Tối ưu exp

Khi tính $\exp(f)$, với f chỉ có giá trị tại các số nguyên tố, ta phải làm $\log n$ lần tích chập sàng khối; điều này khá lãng phí. Xét cách tối ưu.

Trước hết tách f thành phần có chỉ số $\leq \sqrt{n}$, gọi là f_0 , và phần có chỉ số $> \sqrt{n}$, gọi là f_1 . Khi đó

$$\exp(f) = \exp(f_0 + f_1) = \exp(f_0) \exp(f_1).$$

Vì $\exp(f_1) = \epsilon + f_1$ dễ tính, phần khó là $\exp(f_0)$.

Khi tính $\exp(f_0)$, $\sqrt{n}$ hạng đầu của kết quả cuối có thể tính trực tiếp; khó khăn nằm ở phần $> \sqrt{n}$.

Xét cách tính trực tiếp bằng phương pháp lấy phần nguyên rồi sửa sai ở trên. Chọn số nguyên S , và cộng $f_{0,i}$ vào vị trí $v_{[S \ln i]}$. Sau đó trực tiếp lấy $\exp$ đa thức theo v .

Khi nào cách này sai? Giả sử

$$m = \prod_{i=1}^k p_i^{\alpha_i}.$$

Nếu vị trí của m bị ước lượng sai, đặt $t = \lfloor n/m \rfloor$, thì

$$\sum_{i=1}^k \alpha_i \lfloor S \ln p_i \rfloor \in \left(S \ln \left(\frac{n}{t} \right), S \ln \left(\frac{n}{t} \right) + \sum_{i=1}^k \alpha_i \right).$$

Suy ra

$$\sum_{i=1}^k \alpha_i \ln p_i + \ln t - \ln n \in \left[-\frac{\sum_{i=1}^k \alpha_i}{S}, 0 \right],$$

tức

$$\ln(mt) \in \left[\ln n - \frac{\sum_i \alpha_i}{S}, \ln n \right].$$

Đây là một đoạn có độ dài

$$O\left(\frac{n \log n}{S}\right),$$

vì vậy chỉ các mt trong đoạn này mới bị ước lượng sai.

Dùng sàng đoạn để sàng mọi thừa số nguyên tố $\leq \sqrt{n}$ trong đoạn đó. Sau khi sàng ra thừa số nguyên tố, DFS trên các thừa số nguyên tố sẽ nhanh chóng tìm các cặp (m, t) ; trong lúc DFS cũng có thể đồng thời tìm giá trị hàm tại m . Độ phức tạp DFS cùng cấp với số cặp.

Số cặp không vượt quá

$$O\left(\sum_{m=1}^{\sqrt{n}} \frac{n \log n}{Sm}\right) = O\left(\frac{n \log^2 n}{S}\right).$$

Phần đa thức phía trước có độ phức tạp $O(S \log^2 n)$. Lấy $S = \sqrt{n}$ thu được độ phức tạp $O(\sqrt{n} \log^2 n)$.

Ta còn có thể chọn một ngưỡng L và xử lý riêng các thừa số nguyên tố $\leq L$. Khi tính $\exp(f_0)$, không xét các thừa số $\leq L$, tức đặt chúng về 0 ở các vị trí tương ứng trong f_0 . Sau khi tính xong, thêm lần lượt các thừa số nguyên tố đó vào.

Khi thêm số nguyên tố p , giả sử đáp án trước đó là d , thì đáp án mới là

$$d'(n) = \sum_{k=0}^{\log_p n} f(p^k) d\left(\left\lfloor \frac{n}{p^k} \right\rfloor\right).$$

Chi phí cần thiết là $\sqrt{n} \log_p n$. Tổng chi phí thêm vào là

$$\sum_{\substack{p \leq L \\ p \in \mathbb{P}}} \frac{\sqrt{n} \log n}{\log p} = O\left(\frac{L \sqrt{n} \log n}{\log L}\right).$$

Sau khi loại bỏ các thừa số nguyên tố $\leq L$, mỗi m ở trên chỉ còn thừa số nguyên tố $> L$. Nếu

$$m = \prod_{i=1}^k p_i^{\alpha_i},$$

thì

$$\sum_{i=1}^k \alpha_i \leq \log_L n.$$

Vì vậy độ dài đoạn có thể ước lượng sai giảm xuống

$$O\left(\frac{n \log n}{S \log L}\right).$$

Khi đó độ phức tạp tính $\exp(f_0)$ trở thành

$$O\left(\frac{n \log n \log_L n}{S} + S \log^2 n\right).$$

Lấy

$$S = \sqrt{\frac{n}{\log L}},$$

ta được độ phức tạp

$$O\left(\frac{\sqrt{n} \log^2 n}{\sqrt{\log L}}\right).$$

Lấy $L = \log n$, độ phức tạp thời gian là

$$O\left(\frac{\sqrt{n} \log^2 n}{\sqrt{\log \log n}}\right).$$

4.2. Tối ưu ln

Khi tính ln, ta cần biến một hàm nhân tính thỏa

$$f(p^k) = \frac{f(p)^k}{k!}$$

thành một hàm số học g chỉ có giá trị tại p và thỏa $g(p) = f(p)$.

Từ tính chất ở trên có thể trực tiếp lấy $\sqrt{n}$ hạng đầu của $g = \ln(f)$. Gọi hàm số học chỉ giữ $\sqrt{n}$ hạng đầu của g là g_0 , và đặt $g_1 = g - g_0$. Khi đó

$$\exp(g_0 + g_1) = \exp(g_0) \exp(g_1).$$

Chú ý rằng tích chập của hai g_1 có hạng thấp nhất $> n$, nên mọi giá trị trên sàng khối đều bằng 0 và không cần tính. Vì vậy

$$f = \exp(g_0) + \exp(g_0)g_1.$$

Sau khi tính $h = \exp(g_0)$, chỉ cần giải

$$f - h = hg_1.$$

Bây giờ ta cần giải một bài toán chia trên sàng khối. Với các hàm số học A, B, C thỏa $B(1) = 1$ và $A * B = C$, làm sao lấy sàng khối của A từ sàng khối của B, C ?

Trước hết, $\sqrt{n}$ hạng đầu của A dễ giải trong $O(\sqrt{n} \log n)$. Đặt $A = A_0 + A_1$, trong đó A_0 chỉ giữ giá trị của A tại các vị trí $\leq \sqrt{n}$, còn A_1 chỉ giữ giá trị tại các vị trí $> \sqrt{n}$. Khi đó

$$C = A_0B + A_1B.$$

Dùng một lần tích chập sàng khối để lấy A_0B , đặt $D = C - A_0B$; ta chỉ cần giải $A_1B = D$.

Với mọi k ,

$$D\left(\left\lfloor \frac{n}{k} \right\rfloor\right) = \sum_{i=1}^{\infty} B(i) S_{A_1}\left(\left\lfloor \frac{n}{ik} \right\rfloor\right).$$

Vì $S_{A_1}(\lfloor n/(ik) \rfloor)$ chỉ có giá trị khi $ik \leq \sqrt{n}$, tổng số i cần liệt kê là

$$\sum_{k=1}^{\sqrt{n}} \left\lfloor \frac{\sqrt{n}}{k} \right\rfloor = O(\sqrt{n} \log n).$$

Do đó thời gian là $O(\sqrt{n} \log n)$.

Tới đây, ta đã thực hiện được phép ln trên sàng khối bằng một lần exp của hàm chỉ có giá trị tại số nguyên tố, một lần tích chập sàng khối, và thêm $O(\sqrt{n} \log n)$ độ phức tạp phụ.

4.3. Tối ưu tích chập sàng khối

Lúc này nút thắt độ phức tạp lại nằm ở tích chập sàng khối. Tích chập sàng khối có thể làm tốt hơn không? Cũng có thể.

Trong phần sửa sai của ước lượng tích chập sàng khối, với mỗi $m \in [n - L, n]$, ta cần liệt kê mọi bộ ba (x, y, t) thỏa $xyt = m$. Cố định m , liệu phần này có thể tốt hơn $d_3(m)$?

Quan sát các giá trị ta thật sự quan tâm:

- giá trị của xy và t , vì chúng ảnh hưởng tới vị trí cần sửa;
- giá trị của $\lfloor S \ln x \rfloor + \lfloor S \ln y \rfloor$, vì nó ảnh hưởng tới việc vị trí xy có được cập nhật hay không.

Chú ý tính chất

$$S \ln(xy) \leq \lfloor S \ln x \rfloor + \lfloor S \ln y \rfloor < S \ln(xy) + 2.$$

Vì vậy khi đã xác định xy , giá trị $\lfloor S \ln x \rfloor + \lfloor S \ln y \rfloor$ chỉ có hai khả năng. Do đó với x, y , ta chỉ cần biết

$$w_x = \lfloor S \ln x \rfloor \bmod 2, \quad w_y = \lfloor S \ln y \rfloor \bmod 2.$$

Ta cần làm một phép chập để các đóng góp x, y đi vào xy . Giả sử phân tích thừa số nguyên tố của m là

$$m = \prod_{i=1}^k p_i^{\alpha_i}.$$

Phép chập ở trên thực chất là một phép chập k -chiều; miền giá trị của chiều thứ i là $[0, \alpha_i]$. Tuy phép chập còn chịu ảnh hưởng của w , chỉ cần liệt kê bốn trường hợp $w_x = 0/1$, $w_y = 0/1$ và làm phép chập nhiều chiều thông thường, hoặc dùng phản diễn đơn vị căn bậc hai: với $c = 1, -1$, dùng

$$c^{w_x} c^{w_y} = c^{(w_x + w_y) \bmod 2}$$

để giảm còn hai phép chập.

Bài toán chập đa thức nhiều biến có thể tham khảo luận văn đội tuyển tập huấn năm 2021 của Li Baitian [2]. Thời gian một lần là

$$d(m) \omega(m) \log d(m).$$

Xét cách phân tích độ phức tạp. Chọn một hằng số l . Số thừa số nguyên tố $> n^{1/l}$ chỉ là $O(l)$, và đóng góp của chúng vào $\omega(m)$ cũng như $\log d(m)$ đều không quá hằng số, nên có thể bỏ qua.

Với hạng $\log d(m)$, có thể dùng

$$\sum_{i=1}^k \sum_{e=1}^{\alpha_i} \frac{1}{e}$$

để ước lượng. Ngoài ra, nếu $p^t \parallel m$, thì

$$(t+1)d\left(\frac{m}{p^t}\right) \leq d(m),$$

nên có thể ước lượng $d(ipq^e)$ với $p, q \in \mathbb{P}$ bởi $O(d(i)e)$.

Với một m , phần $d(m) \log d(m) \omega(m)$ có thể viết ở cấp

$$O\left(d(m) \left(\sum_{\substack{p \mid m \\ p \in \mathbb{P}}} 1\right) \left(\sum_{\substack{q \in \mathbb{P}, e \geq 1 \\ q^e \parallel m}} \frac{1}{e}\right)\right).$$

Trường hợp $p = q$ có thể bỏ qua trong phân tích độ phức tạp. Ta có thể ước lượng tiếp bởi

$$O\left(\sum_{p|m, p \in \mathbb{P}} \sum_{\substack{q \in \mathbb{P}, e \geq 1 \\ q^e \parallel m, p \neq q}} e \cdot d\left(\frac{m}{pq^e}\right)\right).$$

Do đó có thể phân tích bằng tổng

$$\begin{aligned} & \sum_{m \in [n-L, n]} \sum_{\substack{p < n^{1/l} \\ p \in \mathbb{P}, p|m}} \sum_{\substack{q < n^{1/l} \\ q \in \mathbb{P}, e \geq 1, p \neq q}} d\left(\frac{m}{pq^e}\right) \\ & \leq \sum_{\substack{p \leq n^{1/l} \\ p \in \mathbb{P}}} \sum_{\substack{q \leq n^{1/l} \\ q \in \mathbb{P}, e \geq 1}} \sum_{i \in \left[\frac{n-L}{pq^e}, \frac{n}{pq^e}\right]} d(i). \end{aligned}$$

Chú ý rằng p, q, e chỉ có $O(n^{2/l} / \ln n)$ khả năng, nên tiếp tục dùng ước lượng tổng tiền tố hàm ước trong số học giải tích [1]:

$$\sum_{i \in [L, R]} d(i) = (R - L + 1) \log R + O(R^{131/416}).$$

Chỉ cần hằng số l thỏa

$$\frac{2}{l} + \frac{131}{416} < \frac{1}{2},$$

ta có thể bỏ qua $O(R^{131/416})$.

Vì vậy có thể ước lượng độ phức tạp tiếp thành

$$\sum_{\substack{p \leq n^{1/l} \\ p \in \mathbb{P}}} \sum_{\substack{q \leq n^{1/l} \\ q \in \mathbb{P}, e \geq 1}} \frac{L}{pq^e}.$$

Do

$$\sum_{\substack{p \leq S \\ p \in \mathbb{P}}} \frac{1}{p} = O(\log \log S),$$

và

$$\sum_{\substack{p \leq S \\ p \in \mathbb{P}, e \geq 1}} \frac{1}{p^e}$$

không vượt quá hai lần tổng trước, phần độ phức tạp này ước lượng được là

$$L(\log \log n)^2.$$

Tổng độ phức tạp là

$$\frac{n \log^2 n}{S} + S(\log \log n)^2.$$

Khi lấy

$$S = \frac{\sqrt{n} \log n}{\log \log n},$$

độ phức tạp thời gian là

$$O(\sqrt{n} \log n \log \log n).$$

5. Tổng kết

Trong bài viết này, tác giả thu được một phương pháp sàng có tính mở rộng khá tốt, đưa tích chập sàng khối, phép chia sàng khối, tổng tiền tố sàng khối của hàm nhân tính và bài toán thống kê tiền tố số nguyên tố về độ phức tạp thời gian $O(\sqrt{n} \text{ polylog } n)$. Hai bài toán sau cần thỏa một số điều kiện, chẳng hạn $f(p)$ là đa thức bậc hằng theo p , hoặc một vài tính chất đặc biệt khác. Bài viết cũng thảo luận tối ưu phương pháp này, đưa tích chập sàng khối và phép chia sàng khối của hàm số học về

$$O(\sqrt{n} \log n \log \log n),$$

đồng thời đưa tổng tiền tố sàng khối của hàm nhân tính và thống kê tiền tố số nguyên tố về

$$O\left(\frac{\sqrt{n} \log^2 n}{\sqrt{\log \log n}}\right).$$

Tuy độ phức tạp thời gian tốt hơn, do hằng số lớn và có nhiều thừa số log, cách làm này trong phạm vi dữ liệu OI hiện nay vẫn không thể vượt qua sàng Min_25. Tác giả hy vọng người đọc tiếp tục suy nghĩ và nghiên cứu bài toán tính tổng hàm nhân tính, và mong sẽ xuất hiện các cách làm thực dụng hơn.

6. Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn các huấn luyện viên đội tuyển tập huấn quốc gia, Peng Sijin và Yang Yaoliang, đã hướng dẫn.

Cảm ơn các thầy ở Trường Trung học Học Quân, trong đó có Xu Xianyou, đã quan tâm và hướng dẫn.

Cảm ơn gia đình đã đồng hành và ủng hộ.

Cảm ơn các bạn học và anh chị trong đội bạn đã giúp đỡ và đồng hành.

Cảm ơn các bạn học Zhang Yongxuan, Zhang Hengyi và những người khác đã đọc bản thảo và góp ý cho bài viết này.

Cảm ơn các thầy cô và bạn học khác đã giúp đỡ tác giả.

Tài liệu tham khảo

1. [Divisor summatory function](#).
2. Li Baitian. *Khung lý thuyết tính toán hàm sinh trong thi tin học*. Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2021, 2021.

Bài 2. Bàn về bài toán parity của matroid tuyến tính

Guo Yuchong, Trường Trung học số 2 trực thuộc Đại học Sư phạm Hoa Đông

Tóm tắt

Bài toán parity của matroid tuyến tính (*Linear Matroid Parity Problem*, LMPP) định nghĩa một lớp bài toán tối ưu hóa trên không gian tuyến tính. Mô hình này tạo cầu nối giữa phương pháp đại số và tối ưu tổ hợp: nhiều bài toán kinh điển như matching cực đại trên đồ thị tổng quát hoặc luồng mạng đều có thể quy về nó. Đổi lại, độ phức tạp thường tăng lên, nhưng ta thu được một góc nhìn thống nhất cho các bài toán bề ngoài rất khác nhau.

1. Ký hiệu và quy ước

Ký hiệu e_i là vector đơn vị có thành phần thứ i bằng 1, các thành phần còn lại bằng 0. Với ma trận A và hai tập chỉ số S, T , ký hiệu $A_{S,T}$ là ma trận con chỉ giữ các hàng trong S và các cột trong T . Ký hiệu $\deg_x(F)$ là bậc cao nhất theo biến x trong đa thức nhiều biến F .

2. Bài toán parity của matroid tuyến tính

2.1. Định nghĩa

Cho m cặp vector (a_i, b_i) , trong đó a_i, b_i đều là vector n -chiều. Cần chọn nhiều cặp nhất sao cho mọi vector xuất hiện trong các cặp được chọn đều độc lập tuyến tính.

2.2. Dạng thừa và dạng gọn

Gọi v là đáp án của bài toán gốc. Với các biến chưa xác định $x'_1, \dots, x'_m$, dựng

$$A = (a_1, b_1, a_2, b_2, \dots, a_m, b_m),$$

và

$$X = \text{diag} \left(\begin{pmatrix} 0 & x'_1 \\ -x'_1 & 0 \end{pmatrix}, \begin{pmatrix} 0 & x'_2 \\ -x'_2 & 0 \end{pmatrix}, \dots, \begin{pmatrix} 0 & x'_m \\ -x'_m & 0 \end{pmatrix} \right).$$

Xét các ma trận trên trường phân thức hữu tỉ của $x'_1, \dots, x'_m$, ta có

$$2v + 2m = \text{rank} \begin{pmatrix} 0 & A \\ -A^T & X \end{pmatrix}.$$

Đây là cách dựng dạng thừa.

Với các biến chưa xác định $x_1, \dots, x_m$, dựng ma trận

$$M = \sum_{i=1}^m x_i (a_i b_i^T - b_i a_i^T).$$

Khi xét trên trường phân thức hữu tỉ của $x_1, \dots, x_m$, ta có

$$2v = \text{rank}(M).$$

Đây là cách dựng dạng gọn. So với dạng thừa, dạng gọn cho ma trận nhỏ hơn nên có lợi về thời gian tính toán; các thuật toán phía sau đều dùng cách dựng này. Tính đúng của hai cách dựng đã được chứng minh chi tiết trong tài liệu tham khảo [1], nên bài viết không lặp lại.

2.3. Thuật toán

Để có độ phức tạp tốt, ta xử lý đặc biệt các biến $x_1, \dots, x_m$. Chọn một số nguyên tố lớn p , lấy ngẫu nhiên các x_i trong $[0, p) \cap \mathbb{Z}$, rồi thế vào M để được ma trận M' trên $\mathbb{F}_p$. Dùng khử Gauss tính $\text{rank}(M')$ trong $O(n^3)$, và coi

$$\text{rank}(M) = \text{rank}(M')$$

để suy ra đáp án.

2.4. Phân tích xác suất đúng

Thuật toán trên có xác suất sai. Hiển nhiên $\text{rank}(M') \leq \text{rank}(M)$; cần chứng minh hai giá trị này bằng nhau với xác suất lớn.

Đặt $r = \text{rank}(M)$. Khi đó M có một ma trận con khả nghịch cấp r , ký hiệu $M_{S,T}$. Định thức $\det(M_{S,T})$ là một đa thức nhiều biến khác 0 theo $x_1, \dots, x_m$, còn $\det(M'_{S,T})$ là giá trị sau khi thế các biến ngẫu nhiên vào đa thức ấy.

Bổ đề Schwartz–Zippel. Nếu $f(x_1, \dots, x_m)$ là đa thức nhiều biến khác 0 có tổng bậc không quá d , và mỗi x_i được chọn ngẫu nhiên từ tập hữu hạn S , thì

$$\Pr[f(x_1, \dots, x_m) = 0] \leq \frac{d}{|S|}.$$

Chứng minh. Với $m = 1$, đa thức một biến bậc không quá d có không quá d nghiệm. Giả sử mệnh đề đúng với $m - 1$ biến, tách riêng biến x_m :

$$f(x_1, \dots, x_m) = \sum_{i=0}^d x_m^i f_i(x_1, \dots, x_{m-1}).$$

Gọi k là bậc cao nhất theo x_m . Sau khi cố định $x_1, \dots, x_{m-1}$, nếu đa thức một biến thu được là đa thức không thì

$$f_k(x_1, \dots, x_{m-1}) = 0,$$

mà f_k có bậc không quá $d - k$, nên xác suất trường hợp này không vượt quá $(d - k)/|S|$. Ngược lại, đa thức một biến không không có quá k nghiệm, nên xác suất chọn phải nghiệm không vượt quá $k/|S|$. Tổng lại là $d/|S|$.

Áp dụng bổ đề cho $\det(M_{S,T})$, xác suất $\det(M'_{S,T}) = 0$ không vượt quá r/p . Vì thế thuật toán cho đáp án đúng với xác suất ít nhất $1 - r/p$.

2.5. Dựng phương án

Xét lần lượt từng cặp vector. Nếu xóa cặp hiện tại mà đáp án không giảm thì xóa, ngược lại giữ lại. Cuối cùng sẽ còn đúng v cặp, tạo thành một phương án tối ưu. Cách làm trực tiếp cần tính lại $\text{rank}(M)$ sau mỗi lần xóa, nên mất $O(mn^3)$.

Vì

$$\text{rank}(a_i b_i^T - b_i a_i^T) \leq 2,$$

có thể tăng tốc bằng kỹ thuật nhiều hạng thấp của ma trận.

Bổ đề Sherman–Morrison–Woodbury. Với ma trận khả nghịch A, B , ma trận U cỡ $n \times m$ và ma trận V cỡ $m \times n$, ta có

$$(A - UB^{-1}V)^{-1} = A^{-1} + A^{-1}U(B - VA^{-1}U)^{-1}VA^{-1}.$$

Đồng thời $A - UB^{-1}V$ khả nghịch khi và chỉ khi $B - VA^{-1}U$ khả nghịch.

Trong bài toán hiện tại, ma trận M dựng được chưa chắc khả nghịch nên chưa thể dùng trực tiếp bổ đề. Do M phản đối xứng, nếu $r = \text{rank}(M)$ thì M có một ma trận con chính $M_{S,S}$ khả nghịch cấp r . Chỉ giữ các thành phần vector thuộc S sẽ đưa bài toán về trường hợp M khả nghịch mà không làm mất phương án đúng; một tập S hợp lệ có thể tìm trong $O(n^3)$.

Giả sử từ đây M khả nghịch và duy trì $N = M^{-1}$. Với cặp đang xét, đặt

$$U = (a_i, -b_i), \quad V = (b_i, a_i)^T,$$

khi đó $a_i b_i^T - b_i a_i^T = UV$. Việc kiểm tra $M - UV$ khả nghịch được chuyển thành kiểm tra ma trận 2×2

$$I - VNU.$$

Ta tính nó trong $O(r^2)$ và kiểm tra định thức trong $O(1)$. Nếu khả nghịch, cập nhật

$$N' = (M - UV)^{-1} = N + NU(I - VNU)^{-1}VN,$$

cũng trong $O(r^2)$. Tổng thời gian dựng phương án giảm còn $O(mr^2)$, sau phần tiền xử lý $O(n^3)$.

2.6. Mở rộng

Định nghĩa LMPP yêu cầu mỗi nhóm có đúng hai vector. Với trường hợp có nhóm một vector, cách trực tiếp là mở rộng vector lên $n + m_1$ chiều, và với nhóm một vector thứ i ghép thêm $b_i = e_{n+i}$. Cách này đúng nhưng làm tăng số chiều khi m_1 lớn.

Nếu viết ma trận sau mở rộng dưới dạng

$$M = \begin{pmatrix} M_2 & A_1 \\ -A_1^T & 0 \end{pmatrix},$$

trong đó M_2 là ma trận chỉ xét các cặp và A_1 chứa các vector của nhóm một phần tử, thì ta chỉ quan tâm tới $\text{rank}(M)$. Vì vậy có thể chỉ giữ một cơ sở cột của A_1 , kích thước không quá n , rồi quy về trường hợp $m_1 \leq n$. Khi ấy cách mở rộng trực tiếp không làm tăng cấp độ phức tạp.

2.7. Tiểu kết

Phần này đã trình bày thuật toán ngẫu nhiên tổng quát để giải và dựng phương án cho LMPP, đồng thời nêu cách mở rộng sang nhóm một vector mà không tăng cấp độ phức tạp. Các thành phần thời gian chính là:

- dựng $M = \sum_{i=1}^m x_i(a_i b_i^T - b_i a_i^T)$, mất $O(mn^2)$;
- tính $\text{rank}(M)$, mất $O(n^3)$;
- nếu cần dựng phương án, tính M^{-1} , mất $O(n^3)$;
- sau mỗi nhiều hạng thấp, kiểm tra khả nghịch và cập nhật N , tổng $O(mr^2)$.

Trong bài toán cụ thể, cấu trúc đặc biệt của a_i, b_i thường còn cho phép tối ưu thêm.

3. Bài toán parity có trọng số của matroid tuyến tính

3.1. Giới thiệu

Bài toán parity có trọng số là phiên bản có trọng số của LMPP. Tài liệu tham khảo [4] đưa ra thuật toán $O(\text{poly}(n, m))$ cho bài toán này, nhưng thuật toán khá phức tạp. Phần này trình bày một thuật toán đơn giản hơn với thời gian cao hơn, không phải đa thức theo độ dài biểu diễn tổng quát.

3.2. Định nghĩa

Cho m cặp vector (a_i, b_i) , trong đó a_i, b_i là vector n -chiều. Cặp thứ i có trọng số nguyên dương w_i . Cần chọn một số cặp có tổng trọng số lớn nhất sao cho mọi vector trong các cặp được chọn độc lập tuyến tính.

3.3. Dạng thừa và dạng gọn

Gọi v là đáp án, và giả sử $w_i \in [1, V] \cap \mathbb{Z}$. Thêm biến chưa xác định z , dùng bậc theo z để biểu diễn tổng trọng số. Tương tự dạng thừa không trọng số, đặt

$$A = (a_1 z^{w_1}, b_1, a_2 z^{w_2}, b_2, \dots, a_m z^{w_m}, b_m),$$

$$X = \text{diag} \left(\begin{pmatrix} 0 & x'_1 \\ -x'_1 & 0 \end{pmatrix}, \dots, \begin{pmatrix} 0 & x'_m \\ -x'_m & 0 \end{pmatrix} \right), \quad Y = \text{diag}(y_1, \dots, y_n).$$

Khi đó

$$2v = \deg_z \det \begin{pmatrix} Y & A \\ -A^T & X \end{pmatrix}.$$

Ý tưởng chứng minh là khai triển định thức theo các phần tử đường chéo của Y , rồi dùng tính chất của Pfaffian:

$$\det(B) = \text{pf}(B)^2$$

với B phản đối xứng. Các tiểu thức khác 0 của X buộc hai chỉ số của mỗi cặp phải cùng được chọn hoặc cùng không được chọn; bậc cao nhất theo z khi đó đúng bằng hai lần tổng trọng số lớn nhất của một tập vector độc lập tuyến tính.

Để đưa về dạng gọn, dùng bổ đề Schur. Nếu D khả nghịch thì

$$\det \begin{pmatrix} A & B \\ C & D \end{pmatrix} = \det(A - BD^{-1}C) \det(D).$$

Áp dụng vào ma trận trên:

$$2v = \deg_z \det(Y + AX^{-1}A^T).$$

Đặt $x_i = -1/x'_i$, ta có

$$Y + AX^{-1}A^T = Y + \sum_{i=1}^m x_i (a_i b_i^T - b_i a_i^T) z^{w_i}.$$

Gọi ma trận này là M , ta thu được dạng gọn của bài toán có trọng số:

$$2v = \deg_z \det(M).$$

3.4. Thuật toán

Chỉ cần tính $\det(M)$. Tương tự trường hợp không trọng số, các biến x_i, y_i được gán ngẫu nhiên trên $\mathbb{F}_p$. Mỗi phần tử của M là đa thức theo z bậc không quá V , nên $\det(M)$ có bậc không quá Vn . Chọn $Vn + 1$ giá trị khác nhau $z_1, \dots, z_{Vn+1}$, tính $\det(M)$ sau khi thế từng z_i , rồi dùng nội suy Lagrange để khôi phục đa thức $\det(M)$.

3.5. Dựng phương án

Tương tự phần không trọng số, duy trì các ma trận sau khi thế $z_1, \dots, z_{V_{n+1}}$:

$$M_1, \dots, M_{V_{n+1}}$$

cùng nghịch đảo của chúng

$$N_1, \dots, N_{V_{n+1}}.$$

Ta cần hỗ trợ nhiều hạng thấp và duy trì định thức.

Bổ đề định thức ma trận. Nếu A, B khả nghịch thì

$$\det(A - UB^{-1}V) = \frac{\det(A)}{\det(B)} \det(B - VA^{-1}U).$$

Chúng minh tương tự bổ đề Sherman–Morrison–Woodbury.

Nếu mọi M_i luôn khả nghịch thì công thức trên đủ để cập nhật định thức. Trong thực tế có thể gặp thời điểm $\det(M_i) = 0$. Vì $\det(M)$ có bậc không quá Vn theo z , nếu các z_i được chọn ngẫu nhiên trong $[0, p) \cap \mathbb{Z}$, thì theo Schwartz–Zippel, số lần kỳ vọng gặp $\det(M_i) = 0$ là

$$O\left(\frac{V^2 mn^2}{p}\right).$$

Khi xảy ra, chọn lại z_i ngẫu nhiên cho tới khi nó khác mọi z_j còn lại và $\det(M_i) \neq 0$. Với p đủ lớn, chi phí dựng lại có thể xem là không đáng kể. Sau đó, mỗi lần nội suy lại $\det(M)$ từ các giá trị $\det(M_1), \dots, \det(M_{V_{n+1}})$, ta quyết định được cặp hiện tại có nằm trong phương án tối ưu hay không.

3.6. Tiểu kết

Phần này trình bày thuật toán ngẫu nhiên tổng quát cho phiên bản có trọng số. Các thành phần thời gian chính là:

- tính giá trị của từng phần tử ma trận tại $z_1, \dots, z_{V_{n+1}}$ bằng đánh giá nhiều điểm: $O(Vn^3 \log^2(Vn))$;
- tính định thức tại từng z_i : $O(Vn^4)$;
- nội suy $\det(M)$: $O(Vn \log^2(Vn))$.

Nếu cần dựng phương án, cần thêm:

- tính $N_1, \dots, N_{V_{n+1}}$: $O(Vn^4)$;
- sau mỗi nhiều hạng thấp, nội suy lại $\det(M)$: $O(Vmn \log^2(Vn))$;
- cập nhật các $\det(M_i)$ và N_i : $O(Vmn^3)$.

4. Ứng dụng của bài toán parity matroid tuyến tính

4.1. Giới thiệu

Phần này nêu một số ví dụ quy các bài toán tối ưu tổ hợp về LMPP.

4.2. Matching cực đại trên đồ thị tổng quát

Định nghĩa. Cho một đồ thị vô hướng, cần chọn nhiều cạnh nhất sao cho không có hai cạnh được chọn chung đỉnh.

Thuật toán. Cách cổ điển để giải bài toán này là thuật toán blossom. Với cạnh thứ i là (u_i, v_i) , đặt

$$a_i = e_{u_i}, \quad b_i = e_{v_i}.$$

Nếu hai cạnh được chọn chung đỉnh u , vector e_u xuất hiện ít nhất hai lần nên phụ thuộc tuyến tính. Ngược lại, nếu không chung đỉnh thì mỗi vector đơn vị xuất hiện nhiều nhất một lần, nên độc lập tuyến tính. Do đó bài toán matching cực đại được quy về LMPP. Ma trận M thu được trùng với ma trận Tutte, một công cụ kinh điển khác cho matching tổng quát.

Nếu số cạnh là m , số đỉnh là n , dùng trực tiếp thuật toán tổng quát để dựng phương án cho thời gian $O(mn^2)$.

Tối ưu. Ta dùng cấu trúc đặc biệt của a_i, b_i . Bổ đề Harvey nói rằng nếu A khả nghịch và A, B chỉ khác nhau ở ma trận con chính ứng với tập S , đặt $D = B - A$, thì

$$(B^{-1})_{T,T} = (A^{-1})_{T,T} - (A^{-1})_{T,S}(I + D_{S,S}(A^{-1})_{S,S})^{-1}D_{S,S}(A^{-1})_{S,T}.$$

Đồng thời B khả nghịch khi và chỉ khi

$$I + D_{S,S}(A^{-1})_{S,S}$$

khả nghịch. Bổ đề này cho thấy chỉ cần thông tin cục bộ để xử lý một sửa đổi cục bộ.

Trong matching tổng quát, cặp (a_i, b_i) chỉ ảnh hưởng hai vị trí M_{u_i, v_i} và M_{v_i, u_i} . Vì vậy có thể dùng chia để trị. Gọi $\text{solve}(S, T)$ là thủ tục xử lý mọi cặp có $u_i \in S, v_i \in T$, quyết định chúng có thuộc đáp án hay không. Đặt

$$X = S \cup T,$$

ta chỉ cần quan tâm $M_{X,X}$ và $(M^{-1})_{X,X}$. Nếu $|S| = |T| = 1$, xử lý trực tiếp trong $O(1)$. Ngược lại, chia đôi S, T thành S_1, S_2, T_1, T_2 , đệ quy trên bốn cặp (S_i, T_j) ; sau mỗi đệ quy, các cặp bị loại chỉ gây thay đổi trong ma trận con chính ứng với X , nên dùng bổ đề Harvey cập nhật ảnh hưởng lên $(M^{-1})_{X,X}$ trong $O(|X|^3)$.

Phân tích đệ quy cho hai trường hợp $S = T$ và $S \cap T = \emptyset$ dẫn tới

$$T_2(n) = O(n^3) + 4T_2(n/2) = O(n^3),$$

$$T_1(n) = 2T_1(n/2) + 2T_2(n) + O(n^3) = O(n^3).$$

Vì thế $\text{solve}(\{1, \dots, n\}, \{1, \dots, n\})$ chạy trong $O(n^3)$.

4.3. Cây khung có mex lớn nhất

Bài toán này xuất phát từ QOJ 6402, *MEXimum Spanning Tree*.

Định nghĩa. Cho đồ thị vô hướng liên thông có trọng số cạnh là số nguyên không âm. Cần tìm cây khung sao cho mex của tập trọng số cạnh là lớn nhất. Với tập S , $\text{mex}(S)$ là số nguyên không âm nhỏ nhất không thuộc S .

Thuật toán. Diễn đạt lại: cần tìm k lớn nhất sao cho có thể chọn một đồ thị con không chu trình, trong đó với mỗi trọng số thuộc $[0, k) \cap \mathbb{Z}$ có đúng một cạnh được chọn. Khi cố định k , đây là giao của matroid đồ thị (các rừng) và matroid màu (các tập có màu đôi một khác nhau); cả hai đều là matroid tuyến tính, và giao matroid tuyến tính yếu hơn LMPP.

Cụ thể, nếu số đỉnh là n , với cạnh (u_i, v_i, w_i) thỏa $w_i \in [0, k) \cap \mathbb{Z}$, đặt

$$a_i = e_{u_i} - e_{v_i}, \quad b_i = e_{n+w_i}.$$

Cách dựng này cũng cho thấy quy nạp tổng quát từ giao hai matroid tuyến tính về LMPP. Ma trận tương ứng có dạng

$$M = \begin{pmatrix} 0 & B \\ -B^T & 0 \end{pmatrix},$$

nên $\text{rank}(M) = 2 \text{rank}(B)$.

Trong bài toán này, duyệt k tăng dần cho tới khi $\text{rank}(B) < k$. Khi chuyển từ $k - 1$ sang k , cột thứ k của B đổi từ toàn 0 thành

$$\sum_{i=1}^m [w_i = k] a_i.$$

Giống duy trì cơ sở tuyến tính, vừa thêm vector vừa khử Gauss; mỗi vector mất $O(n^2)$, tổng thời gian $O(n^3)$.

4.4. Cây khung với giới hạn số cạnh mỗi màu

Định nghĩa. Cho đồ thị vô hướng liên thông, mỗi cạnh có một màu là số nguyên dương. Cần tìm một rừng có nhiều cạnh nhất sao cho với mọi màu i , số cạnh màu i không vượt quá lim_i .

Thuật toán. Xem bài toán như giao matroid tuyến tính. Điểm chính là biểu diễn điều kiện “màu i xuất hiện không quá lim_i lần” bằng một matroid tuyến tính. Với màu i , cấp lim_i chiều độc lập với các màu khác. Bài toán quy về việc dựng m vector $k = \text{lim}_i$ chiều trên $\mathbb{F}_p$ sao cho bất kỳ k vector nào cũng độc lập tuyến tính.

Chọn một số nguyên tố $p > m$, và đặt thành phần thứ j của vector thứ i là i^j . Tính độc lập của mọi k vector được chứng minh trực tiếp bằng định thức Vandermonde.

4.5. Ví dụ khác

Bài toán này xuất phát từ UOJ 785, *Goodbye Renyin E: Tìm cặp năm mới*.

Định nghĩa. Cho đồ thị vô hướng, đỉnh u có màu w_u . Cần chọn nhiều đường đi đôi một không giao nhau nhất. Với mỗi đường đi P , gọi hai đầu mút là u, v , cần thỏa:

- $w_u \neq 0, w_v \neq 0$, và $w_u \neq w_v$;
- với mọi $x \in P, x \neq u, x \neq v$, ta có $w_x = 0$.

Thuật toán. Đổi cách nhìn: chọn nhiều cạnh nhất sao cho trong đồ thị chỉ giữ các cạnh ấy, mỗi thành phần liên thông có nhiều nhất hai đỉnh màu khác 0; nếu có hai đỉnh như vậy thì màu của chúng phải khác nhau. Chuyển đổi này đúng vì nghiệm tối ưu không có thành phần toàn màu 0 trừ trường hợp mọi đỉnh

đều màu 0; nếu có thể gộp thành phần ấy vào thành phần kề thì nghiệm tốt hơn. Nếu gọi c_0 là số đỉnh màu khác 0, và c_2 là số thành phần có đúng hai đỉnh màu khác 0, thì

$$|V| - \#E_{\text{chọn}} = \#\text{thành phần liên thông} = c_0 - c_2.$$

Hai đại lượng đầu và số đỉnh màu khác 0 là hằng số, nên chỉ cần tối đa hóa số cạnh.

Gán cho mỗi màu i một vector hai chiều a_i , sao cho các vector này đôi một độc lập tuyến tính. Với cạnh (u_i, v_i) , đặt

$$a_i = e_{2u_i-1} - e_{2v_i-1}, \quad b_i = e_{2u_i} - e_{2v_i}.$$

Cách này bảo đảm tập cạnh được chọn tạo thành rừng. Với đỉnh u có $w_u \neq 0$, đặt

$$a_u = \alpha_{w_u,1} e_{2u-1} + \alpha_{w_u,2} e_{2u}.$$

Sau đó có hai cách tương đương:

- xem mỗi đỉnh màu khác 0 là một nhóm một vector có thể chọn hoặc không chọn, rồi dùng dạng mở rộng ở phần 2.6;
- bắt buộc chọn mọi đỉnh màu khác 0. Nếu V_1 là không gian sinh bởi các vector của đỉnh, phân rã $\mathbb{F}_p^n = V_1 \oplus V_2$ và chiếu mọi a_i, b_i của cạnh xuống V_2 , từ đó loại ảnh hưởng của điều kiện bắt buộc chọn đỉnh.

Trong nghiệm tối ưu, mọi đỉnh màu khác 0 đều được chọn, nên hai cách là tương đương. Để kiểm tra rằng cấu trúc trên bảo đảm mỗi thành phần liên thông thỏa yêu cầu. Do a_i, b_i có dạng giống ví dụ matching, có thể dùng tiếp kỹ thuật chia để trị dựa trên bổ đề Harvey để đạt tổng thời gian $O(n^3)$.

5. Tổng kết

Bài viết giới thiệu thuật toán tổng quát cho bài toán parity của matroid tuyến tính, các mở rộng và một số ứng dụng; qua đó khảo sát bước đầu việc dùng phương pháp đại số trong tối ưu tổ hợp. Do giới hạn năng lực của tác giả, bài viết không đi vào các nội dung như thuật toán đa thức cho phiên bản có trọng số hoặc bài toán parity của matroid phi tuyến tính.

Những năm gần đây, học thuật có nhiều kết quả mới theo hướng này, đồng thời còn nhiều phương pháp đại số khác đang phát triển. Hiện tại các phương pháp ấy chưa được cộng đồng OI tiếp nhận rộng rãi. Tác giả hy vọng bài viết có thể gợi hứng thú tìm hiểu phương pháp đại số và đưa những lớp bài toán, kỹ thuật mới vào OI.

6. Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn các thầy Jin Jing, Gao Kai và Wu Shenguang đã hướng dẫn tác giả.

Cảm ơn cha mẹ đã nuôi dưỡng và dạy dỗ.

Cảm ơn các anh Chang Ruinian, Wan Chengzhang, cùng các bạn Ke Yisi và Ke Yijing đã giúp đỡ trong quá trình viết bài.

Cảm ơn mọi thầy cô và bạn học đã giúp đỡ tác giả.

Tài liệu tham khảo

1. Chang Ruinian. *Bàn về ứng dụng đại số tuyến tính trong OI*. Tập luận văn đội tuyển quốc gia IOI 2022.

2. Ren Qingyu. *Goodbye Renyin E: lời giải bài Tìm cặp năm mới*.
3. Ho Yee Cheung, Lap Chi Lau, Kai Man Leung. *Algebraic Algorithms for Linear Matroid Parity Problems*.
4. Satoru Iwata, Yusuke Kobayashi. *A Weighted Linear Matroid Parity Algorithm*.

Bài 3. Bàn về hợp nhất và tách cấu trúc dữ liệu

Huang Luotian, Trường Trung học trực thuộc Đại học Nhân dân Trung Quốc

Tóm tắt

Bài viết thảo luận việc hợp nhất và tách các cấu trúc dữ liệu. Trước hết ta giới thiệu hợp nhất và tách cây đoạn, cây cân bằng, rồi mở rộng sang cây phân trị đỉnh, cây phân trị cạnh và static Top Tree; đồng thời đưa ra cách hợp nhất và tách static Top Tree. Ngoài ra, bài viết cũng trình bày một số ứng dụng của những kỹ thuật này trong OI.

1. Hợp nhất và tách cấu trúc dữ liệu tĩnh

1.1. Dẫn nhập

Các cấu trúc dữ liệu tĩnh quen thuộc có nhiều loại: cây đoạn, cây phân trị đỉnh, cây phân trị cạnh, static Top Tree. Do đặc điểm tĩnh của chúng, khi cần hợp nhất hoặc tách, thao tác thường khá thuận tiện: thông tin tại các nút tương ứng có thể được hợp nhất trực tiếp; khi tách, chỉ cần tách một số nút thành hai nút nằm ở cùng vị trí. Điều này cũng thể hiện rõ trong hợp nhất cây đoạn.

1.2. Hợp nhất cây đoạn

Trước hết xét hợp nhất cây đoạn không có lazy tag, quan sát thuật toán cơ bản rồi thử mở rộng.

Hợp nhất cây đoạn nghĩa là hợp nhất hai cây đoạn mở nút động thành một cây, đồng thời hợp nhất thông tin tại các vị trí tương ứng. Ý tưởng thô bạo là đệ quy hợp nhất mọi nút xuất hiện đồng thời trong hai cây; nếu một bên là nút rỗng và bên kia là một cây con, thì trả thẳng cây con ấy.

Để duy trì thông tin, với nút được hợp nhất thô bạo ta có hai lựa chọn:

1. sau khi hợp nhất xong các cây con, kéo thông tin từ hai con mới lên;
2. hợp nhất trực tiếp thông tin của hai nút tương ứng.

Hai cách này có ưu và nhược điểm riêng. Cách thứ nhất đòi hỏi thông tin trên cây đoạn hỗ trợ hợp nhất từ các con. Trong đa số tình huống đây là điều tự nhiên; ví dụ với dãy nhị phân, ta có thể duy trì trong mỗi đoạn số lần xuất hiện của các dãy con $[010, 01, 10, 0, 1]$.

Nhưng có những thông tin không hỗ trợ kéo từ con lên. Chẳng hạn ta hợp nhất cây đoạn lồng cây cân bằng, tức mỗi nút cây đoạn duy trì một cây cân bằng. Nếu hợp nhất cây cân bằng từ hai con rồi cần sao chép một bản làm thông tin của nút hiện tại, không phải mọi loại cây cân bằng hoặc thao tác đều có thể được làm bền vững. Trong khi đó, hợp nhất theo vị trí các cây cân bằng có thể xem như thực hiện đồng thời nhiều phép hợp nhất cây cân bằng, nên có thể dùng lại phân tích độ phức tạp của hợp nhất cây cân bằng.

Trong phạm vi xử lý được bởi hai cách trên, ta có thể hợp nhất hai cây trong thời gian

$$O(|T_1 \cap T_2|).$$

Sau n lần sửa đổi và hợp nhất, tổng độ phức tạp thời gian là $O(n \log n)$.

Với cây đoạn có lazy tag, do cây đoạn mở nút động có thể tạo nút mới khi đẩy tag xuống, ảnh hưởng tới độ phức tạp, ta cần dùng một số cách để hạn chế số nút mới.

Với cây đoạn mở nút động đang được duy trì, luôn giữ tính chất: mỗi nút hoặc không có con, hoặc có đủ hai con. Khi đó đẩy tag ở nút không phải lá sẽ không tạo nút mới. Bài toán trở thành: xử lý thế nào khi hợp nhất một nút lá với một cây con cây đoạn. Một ý tưởng là nút lá thường biểu diễn một đoạn có tính chất đồng nhất, nên trích ra tính chất đồng nhất đó và đánh thành tag lên cây con còn lại. Ví dụ nếu sửa đổi là cộng đoạn, thì mọi giá trị trong đoạn do nút lá biểu diễn đều giống nhau. Nếu hợp nhất hai cây là cộng theo vị trí, ta có thể xem giá trị đồng nhất ấy là một tag cộng cho cây con của cây kia.

Ví dụ 1.1. Cho một cây nhị phân gốc 1, mỗi đỉnh có trọng số a_i . Mỗi đỉnh hoặc là lá, hoặc có đúng hai con. Nếu đỉnh u là lá thì $b_u = a_u$; ngược lại

$$b_u = |b_{\text{lson}} - b_{\text{rson}}| + a_u,$$

trong đó lson, rson là hai con trái phải của u . Hỗ trợ sửa đổi điểm a_u , sau mỗi lần hỏi b_1 , với

$$n, q \leq 10^5, \quad 0 \leq a_u \leq 20.$$

Xét xử lý offline, dùng cây đoạn duy trì giá trị b_u của mỗi đỉnh u tại các thời điểm khác nhau. Với đỉnh không phải lá cần hợp nhất hai cây đoạn; với mọi đỉnh cần thực hiện một số phép cộng đoạn.

Vấn đề chính là xử lý nhanh việc hợp nhất một lá với một cây con cây đoạn mở nút động. Khi đó thao tác cần làm là biến mọi giá trị x trong cây con thành

$$x \mapsto |x - v|,$$

trong đó v là tag cộng đoạn trên nút lá. Rõ ràng lấy trị tuyệt đối theo v không phải một tag hợp nhất được, nên ta dùng phân tích thế năng.

Gọi $\min_u, \max_u$ lần lượt là giá trị nhỏ nhất và lớn nhất của b trong đoạn thời gian do nút cây đoạn u biểu diễn. Khi cần lấy trị tuyệt đối theo v trên cây con u , có hai nhánh cắt tĩa:

1. nếu $v \leq \min_u$, đánh tag $x \mapsto x - v$;
2. nếu $v \geq \max_u$, đánh tag $x \mapsto v - x$.

Vì phép $x \mapsto kx + b$ có thể hợp nhất, hai nhánh trên xử lý được bằng tag. Thiết kế thế năng

$$\Phi = \sum_u (\max_u - \min_u + 1).$$

Nếu không rơi vào hai nhánh cắt tĩa, đệ quy thô bạo sẽ làm $\max - \min$ giảm ít nhất 1. Phép cộng đoạn chỉ làm thay đổi thế năng của $O(\log n)$ đoạn, và giá trị v hiển nhiên thay đổi nhiều nhất chưa tới 20. Vì thế tổng độ phức tạp thời gian là $O(nv \log n)$. Ví dụ này minh họa một cách khác dùng thế năng khi hợp nhất nút lá và cây con cây đoạn.

1.3. Tối ưu độ phức tạp bộ nhớ

Nếu cài đặt trực tiếp, hợp nhất cây đoạn dùng $O(n \log n)$ bộ nhớ. Nếu các phép hợp nhất có thể xử lý offline, có thể tối ưu xuống $O(n)$.

Xét dựng cây từ quá trình hợp nhất, rồi phân rã nặng nhẹ trên cây hợp nhất này; chạy hợp nhất cây đoạn theo thứ tự ưu tiên đi vào con nặng, đồng thời viết thu hồi rác cho cây đoạn. Ta chứng minh bộ nhớ của cách này là $O(n)$.

Bổ đề 1.1. Một cây đoạn mở nút động sau m thao tác có

$$O(m(\log n - \log m))$$

nút.

Xét phân loại nút cây đoạn, lần lượt với $\text{dep} \leq \log m$ và $\text{dep} > \log m$. Với loại đầu, ngay cả khi xây đầy đủ cũng chỉ có $O(m)$ nút. Vì sửa đổi cây đoạn có dạng hai chuỗi, nên nhiều nhất có $O(m)$ chuỗi. Với loại sau, mỗi chuỗi chỉ có thể tạo mới $O(\log n - \log m)$ nút, nên tổng cộng có

$$O(m(\log n - \log m))$$

nút.

Bổ đề 1.2. Với một nút bất kỳ u , đường từ u tới gốc sẽ được chia thành $k \leq \log n$ tiền tố chuỗi nặng. Giả sử đỉnh đầu của các chuỗi nặng này, theo thứ tự từ trên xuống, là $z_1, z_2, \dots, z_k$, thì

$$\text{size}(z_i) \leq \frac{n}{2^{i-1}}.$$

Mỗi lần nhảy lên một chuỗi nặng, kích thước cây con ít nhất nhân đôi, suy ra bổ đề trên. Kết hợp hai bổ đề, tại mọi thời điểm tổng kích thước các cây đoạn bị chặn bởi

$$\sum_{i=1}^k O\left(\frac{n}{2^{i-1}} \left(\log n - \log \frac{n}{2^{i-1}}\right)\right) = \sum_{i=1}^k O\left(\frac{n}{2^i} i\right) = O(n).$$

Do đó với các bài toán ở phần sau có thể offline, bộ nhớ đều có thể tối ưu xuống $O(n)$.

Với một lớp cây đoạn mở nút động đặc biệt khác, mỗi lần sửa đổi đều là sửa điểm. Lợi dụng tính chất này cũng có thể tối ưu bộ nhớ xuống $O(n)$. Cụ thể, duy trì cây ảo của mọi nút lá trên cấu trúc cây đoạn và chỉ duy trì các nút thuộc cây ảo. Sau m thao tác, cây đoạn mở nút động hiển nhiên có $O(m)$ nút cây ảo. Tuy nhiên cách này khá rườm rà khi cài đặt, nên thông thường dùng phương pháp thứ nhất.

1.4. Hợp nhất thêm các cấu trúc dữ liệu tĩnh

1.4.1. Hợp nhất cây phân trị đỉnh, cây phân trị cạnh. Ta thấy hợp nhất cây đoạn có khả năng mở rộng rất mạnh.

Tương tự cây đoạn mở nút động, ta có thể định nghĩa cây phân trị đỉnh, cây phân trị cạnh mở nút động. Để tiện trình bày, ta chỉ bàn cây phân trị đỉnh; cây phân trị cạnh là tương tự.

Vì mỗi nút trong cây phân trị đỉnh có nhiều con, ta khó hỗ trợ tìm kiếm từ trên xuống, và một số thông tin cũng khó duy trì. Do đó xét xây cây Huffman cho các con của cây phân trị đỉnh; không khó chứng minh chiều cao là $O(\log n)$.

Trong một lớp bài toán cây phân trị đỉnh, ta thường cần duy trì thông tin từ trọng tâm tới các đỉnh khác trong thành phần liên thông hiện tại. Với mỗi đỉnh u , các nút bị nó ảnh hưởng chính là các tổ tiên của nó trên cây phân trị đỉnh. Vì vậy nếu ta làm sửa điểm thì chỉ có $O(\log n)$ nút có thông tin thay đổi trên cây phân trị đỉnh.

Khác cây đoạn, cây phân trị đỉnh không hỗ trợ hợp nhất thông tin duy trì ở các con lên nút hiện tại, nên chỉ có thể dùng cách hợp nhất thứ hai ở trên.

Ví dụ 1.2 (CTSC 2018, Viết sai bằng bạo lực). Cho hai cây có trọng số cạnh T, T' , trọng số cạnh có thể âm. Cần tìm hai đỉnh x, y để tối đa hóa

$$\text{depth}(x) + \text{depth}(y) - (\text{depth}(\text{LCA}(x, y)) + \text{depth}'(\text{LCA}'(x, y))).$$

Biến đổi phần đóng góp để loại LCA trong cây thứ nhất, thu được

$$\frac{1}{2}(\text{depth}(x) + \text{depth}(y) + \text{dis}(x, y) - 2 \text{depth}'(\text{LCA}'(x, y))).$$

Sau đó liệt kê $\text{LCA}'(x, y)$ trong T' , tức DFS trên T' . Giả sử liệt kê tới u ; mỗi lần hợp nhất một con v của u vào cây con của u và tính đóng góp.

Khi đó bài toán chuyển thành: ban đầu có nhiều tập, mỗi tập là một đỉnh; cần hỗ trợ hợp nhất tập. Khi hợp nhất S_1, S_2 , cần tính

$$\max_{u \in S_1, v \in S_2} \text{dis}(u, v) + \text{depth}(u) + \text{depth}(v).$$

Dùng cây phân trị cạnh để xử lý đóng góp. Với mỗi tập, mở một cây phân trị cạnh; mỗi nút lưu $\text{info}_{u,0/1}$, biểu thị giá trị lớn nhất của $\text{dep} + \text{dis}$ từ hai phía tới cạnh phân trị. Với mỗi tập, dùng cây phân trị cạnh mở nút động rồi hợp nhất các cây này. Vì thông tin của nút rỗng là đơn vị, nên hợp nhất nút rỗng với cây con thì trả thẳng cây con. Với hai nút trùng nhau, hợp nhất $\text{info}_{u,0/1}, \text{info}_{v,0/1}$ bằng phép lấy max. Tổng thời gian là $O(n \log n)$, bộ nhớ $O(n)$.

1.4.2. Hợp nhất static Top Tree. Trước hết ta giới thiệu static Top Tree.

Định nghĩa 1.1. Với cây vô hướng T , định nghĩa một *cluster* trên T là một bộ ba

$$C = (V, E, B),$$

trong đó (V, E) liên thông và là đồ thị con của T , tập $B \subseteq V, |B| \leq 2$, và với mọi $x \in V$, nếu tồn tại $y \notin V$ sao cho $(x, y) \in E(T)$, thì $x \in B$. Các phần tử trong B gọi là điểm biên của C . Ta dùng $V(C), E(C)$ lần lượt biểu thị tập đỉnh, tập cạnh của C , và $B(C)$ biểu thị tập điểm biên của C .

Định nghĩa 1.2. Thao tác *compress* là thao tác hợp nhất hai cluster thành một cluster. Trong đó, hai cluster có giao của hai tập điểm biên bằng 1 được hợp nhất thành một cluster mới; tập điểm biên mới là hiệu đối xứng của hai tập điểm biên cũ.

Định nghĩa 1.3. Thao tác *rake* cũng hợp nhất hai cluster thành một cluster. Trong đó, hai cluster có giao của hai tập điểm biên bằng 1 được hợp nhất thành một cluster mới; tập điểm biên mới là tập điểm biên của một trong hai cluster.

Với một cây, ta có thể dùng hai thao tác trên để co toàn bộ cây. Cụ thể, xem một cluster như một cạnh bị co giữa hai điểm biên (u, v) ; ban đầu mỗi cạnh (u, v) là một cluster, rồi liên tục hợp nhất các cluster bằng hai thao tác trên, cuối cùng hợp nhất được cả cây thành một cluster. Trong quá trình co, cấu trúc hợp nhất của các cluster tạo thành một cây, gọi là Top Tree.

Không khó thấy Top Tree như vậy có mỗi nút nhiều nhất hai con. Tiếp theo ta đưa ra một cách xây cây sao cho độ sâu của Top Tree là $\Theta(\log n)$.

Phân rã nặng cây gốc, rồi DFS xử lý từ dưới lên từng chuỗi nặng, giả sử mỗi cây con nhẹ đã được hợp nhất thành một cluster. Với mỗi đỉnh trên chuỗi nặng, lấy các cluster của những cây con nhẹ của cha đỉnh này, dùng chia để trị để *rake* chúng lại; cluster tổng nhận được lại được *rake* một lần nữa vào cạnh cha của đỉnh này. Để bảo đảm độ sâu không quá lớn, mỗi lần chia đôi, *rake* xong hai nửa rồi mới *rake* thành một cluster. Cây do thao tác này tạo ra gọi là *rake tree*. Sau đó ta thu được các cluster xếp thành một chuỗi, rồi *compress* chúng thành một cluster; tương tự cần dùng chia để trị để khống chế độ sâu. Cây nhận được gọi là *compress tree*.

Bổ đề 1.3. Nếu chọn điểm giữa có trọng số theo kích thước cluster để chia để trị, thì Top Tree xây ra theo cách này có độ sâu $\Theta(\log n)$.

Dù là rake tree hay compress tree, cứ nhảy lên trên một số tầng hằng số thì kích thước cluster ít nhất nhân đôi, nên độ sâu là $O(\log n)$.

Sau đây ta bàn việc hợp nhất Top Tree được xây bằng thuật toán trên.

Ví dụ 1.3. Định nghĩa một thao tác: chọn một đoạn $[l, r]$, rồi đổi mọi ký tự trong đoạn từ ngoặc vuông sang ngoặc tròn và từ ngoặc tròn sang ngoặc vuông.

Định nghĩa trọng số $\text{val}(S)$ của một xâu ngoặc: nếu xâu ngoặc này có thể biến thành hợp lệ bằng các thao tác trên, trọng số là số thao tác ít nhất; nếu không thì là 0.

Cho q lần sửa đổi và truy vấn, có hai loại:

1. sửa đổi: cho đoạn $[l, r]$, đổi mọi ký tự trong đoạn từ ngoặc vuông sang ngoặc tròn và ngược lại;
2. truy vấn: cho đoạn $[l, r]$, tính

$$\sum_{[l', r'] \subseteq [l, r]} \text{val}(s[l', r']).$$

Bài này cần quan sát và biến đổi sơ bộ trọng số trong đề; phần ấy không liên quan nhiều tới bài viết nên được lược qua. Cấu trúc dữ liệu cần hỗ trợ: với một chuỗi đỉnh dọc trong cây, từ nông tới sâu là $a_1, a_2, \dots, a_k$, tính thông tin nửa nhóm của các cây con trong những con của a_i có chỉ số lớn hơn a_{i+1} , đồng thời hỗ trợ sửa đổi chuỗi trong cây.

Vì các thao tác không có tính chất thuận lợi, ta xét xây static Top Tree đầy đủ. Do thứ tự giữa các con là quan trọng trong bài này, mỗi nút compress cần nối hai cây con rake, lần lượt biểu thị thông tin các con nhẹ bên trái và bên phải của con nặng.

Khi định vị trên rake tree hoặc compress tree, cần chú ý các trường hợp sau. Giả sử cần định vị đoạn giữa u và v :

1. một trong u, v là tổ tiên của đỉnh kia;
2. không đỉnh nào trong u, v là tổ tiên của đỉnh kia;
3. u, v lần lượt nằm trong hai rake tree trái phải của con nặng;
4. một trong u, v là con nặng, tức không nằm trong rake tree;
5. u, v nằm trong cùng một rake tree.

Năm trường hợp này mô tả các tình huống gặp khi định vị trong static Top Tree; cách tính đóng góp cho từng loại không khó, nên bài viết không triển khai chi tiết. Thời gian là $O(n \log n)$, bộ nhớ $O(n)$.

Tương tự cây đoạn mở nút động, ta có thể có static decomposition tree mở nút động. Mỗi lần sửa đổi, ta định vị tới $O(\log n)$ nút bị ảnh hưởng, rồi xây các nút ấy và toàn bộ con của chúng. Vì mỗi nút hoặc là lá, hoặc có đủ mọi con, nên miễn là không đẩy tag xuống lá, số nút của cây sẽ không đổi.

Quá trình định vị khác với tìm kiếm từ gốc xuống của cây đoạn: chỉ cần ghi lại điểm biên tương ứng của cluster hiện tại. Ta không thể chịu chi phí mỗi lần tìm điểm biên mới, vì điều đó cần một phép nhị phân $O(\log n)$. Do đó ở đây thường xây trước một static Top Tree đầy đủ; khi định vị, duy trì đồng thời vị trí trên cây mở nút động và trên cây đầy đủ, rồi nhờ cây đầy đủ biết nút cần định vị nằm ở đâu. Cụ thể, trước hết định vị tới hai đầu của hai chuỗi sửa đổi, tức các đầu mút của phép sửa chuỗi. Sau đó liên tục nhảy lên cha trong static Top Tree đầy đủ và ghi lại thuộc về con nào. Chỉ cần một mảng phụ $O(\log n)$ để ghi đường đi. Rồi từ gốc đi xuống theo hai đường đi, vừa đi vừa mở nút động. Như vậy, với chi phí bộ nhớ phụ $O(\log n)$, ta giải quyết vấn đề định vị với hằng số nhỏ.

Sau khi định vị xong, tùy yêu cầu ta đánh tag chuỗi, tag cây con hoặc thực hiện truy vấn.

Bây giờ đã có static Top Tree mở nút động, xét cách hợp nhất hai static Top Tree mở nút động. Ta vẫn hợp nhất T_1, T_2 từ trên xuống.

Nếu không có lazy tag, có thể dùng đệ quy thô bạo rất đơn giản trên các nút ở vị trí tương ứng, và trả về tại nút rỗng. Ở đây thông tin khi hợp nhất cũng có vấn đề giống cây đoạn: cần chọn một trong hai cách duy trì thông tin. Giả sử có thể hợp nhất thông tin trong $O(1)$, ta nhận được thuật toán $O(n \log n)$.

Nếu có lazy tag, ta phải đối mặt với việc hợp nhất một nút lá và một cây con. Khi ấy trên nút lá có hai loại tag: tag trên đường giữa các điểm biên của cluster và tag trên toàn cluster trừ đường đó. Trường hợp thường gặp là có thể đánh thẳng hai loại tag này lên nút tương ứng của static Top Tree bên kia. Nếu không được, cần thiết kế một thể năng cho hai bộ thông tin: thông tin trên đường giữa các điểm biên của cluster, và thông tin của toàn cluster ngoài phần đường đó. Vì hai bộ thông tin có liên hệ với nhau, cụ thể thông tin đường giữa các điểm không biên duy trì trên compress tree là tổng của hai phần trên rake tree, nên khi dùng thể năng và đệ quy thô bạo, cần chuyển đổi giữa hai loại thông tin.

Ví dụ 1.4. Cho một cây có gốc 1, gồm n đỉnh. Mỗi cạnh có một ký tự $c \in \{0, 1\}$. S_u là xâu tạo bởi việc viết lần lượt các ký tự trên đường từ gốc tới u . Bảo đảm các cạnh từ cùng một đỉnh tới các con có ký tự đôi một khác nhau.

Với mỗi đỉnh u , có một giá trị val_u và một giới hạn a_u . Với mỗi đỉnh u , cần tính

$$ans_u = \max_{\substack{0 \leq i \leq |S_u| \\ i \geq a_u, S_u = S_v[|S_v| - |S_u| + 1, |S_v|] \\ S_u[1, i] = S_v[1, i]}} val_v.$$

Xây automaton AC trên cây trie đầu vào. Bài toán chuyển thành: với mỗi nút u trên automaton AC, duy trì một cây trie có trọng số đỉnh. Cần hỗ trợ truy vấn tổng trên chuỗi và lấy max trên chuỗi. Khi hợp nhất hai cây trie, lấy max trọng số ở các nút tương ứng.

Vì đề bảo đảm cây là nhị phân và không có sửa/truy vấn cây con, ta có thể lược bỏ rake tree và việc duy trì thông tin đường giữa các điểm không biên.

Xét tới việc lấy max trên đoạn và hỏi tổng đoạn, đây là ứng dụng kinh điển của segment tree beats. Vì vậy ở mỗi nút duy trì giá trị nhỏ nhất, số lượng giá trị nhỏ nhất, giá trị nhỏ thứ hai và tổng đoạn; đồng thời duy trì tag lười biểu thị toàn bộ giá trị nhỏ nhất đã được cộng thêm bao nhiêu. Khi làm sửa chuỗi, trước hết định vị tới nút, rồi mô phỏng beats: nếu giá trị nhỏ thứ hai không nhỏ hơn val, thì xử lý bằng cắt tĩa; nếu không thì đệ quy theo thông tin sau đó. Để chứng minh phần này có thời gian $O(n \log n)$.

Với trường hợp hợp nhất cây, vẫn dùng khung hợp nhất static Top Tree tổng quát ở trên. Điểm duy nhất cần xử lý là khi hợp nhất một nút lá với một cây con static Top Tree: khi đó tương đương với sửa các nút trên compress tree bằng phép lấy max với val, trong đó val là tag cộng trên lá. Phép sửa lấy max này có thể tiếp tục dùng phương pháp beats. Vì thế bài toán được giải trong $O(n \log n)$ thời gian và $O(n)$ bộ nhớ.

1.5. Tách cây đoạn

Tách cây đoạn hỗ trợ tách một cây đoạn thành hai phần có miền giá trị không giao nhau. Chú ý rằng tách cây đoạn và hợp nhất cây đoạn không phải là hai phép ngược nhau.

Cụ thể, nếu cần chia dãy a_i thành

$$b_i = \begin{cases} a_i, & i \leq p, \\ 0, & i > p, \end{cases} \quad c_i = \begin{cases} 0, & i \leq p, \\ a_i, & i > p, \end{cases}$$

đồng thời duy trì thông tin đoạn của b_i, c_i , ta có thể thực hiện bằng cách tách cây đoạn đang duy trì dãy a_i .

Nếu một nút của cây đoạn biểu diễn đoạn $[l, r]$ thỏa $r \leq p$, thông tin của đoạn này có thể được sao chép nguyên vẹn sang nút tương ứng trong cây đoạn của dãy b . Tương tự, nếu $l > p$, thông tin đoạn có thể sao chép sang cây đoạn của dãy c . Các nút không thỏa cả hai điều kiện hiển nhiên chỉ có $O(\log n)$ nút; ta sao chép chúng thành hai bản và đặt vào vị trí tương ứng.

Cách làm này chỉ tăng thêm $O(\log n)$ nút, nên phân tích thể năng ban đầu vẫn đúng.

Ví dụ 1.5. Duy trì một dãy $a_1, a_2, \dots, a_n$ gồm các số nguyên không âm, hỗ trợ ba loại thao tác:

1. cho đoạn $[l, r]$, xor mọi số trong đoạn với x ;
2. cho đoạn $[l, r]$, sắp xếp các số trong đoạn theo thứ tự tăng dần;
3. cho đoạn $[l, r]$, hỏi xor của các số trong đoạn.

Vì thao tác sắp xếp đoạn khó duy trì trực tiếp, ta dùng phương pháp amortized theo các đoạn màu để né nó. Tại mọi thời điểm, viết a thành dạng: với mỗi tập S_i , sắp xếp các phần tử trong S_i , lần lượt xor với x_i , rồi nối các đoạn lại.

Dùng một 01-trie, tức cây đoạn trên miền giá trị, để duy trì tập S_i ; dùng một cây đoạn để duy trì toàn cục các x_i và tổng xor của mỗi tập sau khi xor với x_i . Mỗi khi gặp một đoạn thao tác $[l, r]$, tách các tập cắt qua $l-1, l$ và qua $r, r+1$. Ở đây cần hỗ trợ tách trie.

Thao tác xor đoạn chỉ cần sửa đoạn trên các x_i . Thao tác sắp xếp đoạn cần đánh tag lên các trie này rồi hợp nhất chúng. Truy vấn đoạn được thực hiện trên cây đoạn hỏi tổng xor đoạn. Do đó trong 01-trie chỉ cần duy trì kích thước cây con và xor của cây con.

Vì cần tách, không thể xây sẵn cây thao tác. Nhưng mọi sửa đổi trong bài này đều là sửa điểm, nên có thể dùng cây đoạn mở nút động dạng nén để đưa bộ nhớ về $O(n)$. Thời gian là

$$O(n(\log n + \log v)).$$

1.6. Tách thêm các cấu trúc dữ liệu tĩnh

Sau khi tách cấu trúc dữ liệu, việc lấy thông tin từ con hợp nhất lên là đơn giản. Nhưng nếu muốn tách trực tiếp từ thông tin cũ, thông tin được duy trì bắt buộc phải hỗ trợ tách. Vì vậy việc tách cây phân trị đỉnh/cạnh yêu cầu cao đối với thông tin và phạm vi ứng dụng hẹp, nên không phải trọng tâm của bài viết.

Xét tách một cây con và phân bù cây con trên static Top Tree.

Giả sử cần tách cây con của u . Ta có thể định vị phần cây con của u tương ứng trên static Top Tree: đó là các nút trong compress tree nơi u nằm có độ sâu phía sau u , cùng với các cây con rake tree của chúng. Phần này có thể được tách nguyên vẹn. Với các nút tổ tiên của u trên static Top Tree, do một phần của chúng nằm trong cây con của u trên cây gốc, phần còn lại không nằm trong đó, nên cần sao chép sang hai cây. Mỗi lần chỉ tạo mới $O(\log n)$ nút.

Ví dụ 1.6. Cho một cây T . Duy trì nhiều đa tập S_i , các phần tử thuộc $\{1, \dots, n\}$. Hỗ trợ:

1. tách mọi phần tử trong S_p có chỉ số nằm trong cây con của u sang S_q ;
2. với mọi đỉnh x trên đường từ u tới v , chèn x vào S_p đúng k lần;
3. hợp nhất hai tập;
4. hỏi tổng số lần xuất hiện trong S_p của mọi đỉnh x trên đường từ u tới v .

Dùng static Top Tree duy trì số lần xuất hiện của mỗi số. Sửa đổi tương đương cộng trên chuỗi, truy vấn tương đương hỏi tổng trên chuỗi, đồng thời cần hỗ trợ hợp nhất và tách.

Hợp nhất chỉ cần cộng theo vị trí, nên hợp nhất lá và cây con rất đơn giản: đánh lazy tag trực tiếp. Khi tách, trước hết định vị tới compress tree chứa u , rồi theo vị trí của u tách nút hiện tại thành hai phần và chọn một phía để đệ quy.

Thiết kế thế năng

$$\Phi = \sum_i |T_i|,$$

tức tổng số nút của các static Top Tree. Có thể phân tích được thời gian là $O(n \log n)$.

Sau khi xử lý tách cây con, một ý tưởng khác là xử lý tách đường. Nghĩa là sao chép một phần compress tree trong static Top Tree sang vị trí tương ứng của T' , nhưng các rake tree nối với những compress tree này không nên được sao chép sang vị trí tương ứng của T' ; chúng phải được giữ ở vị trí cũ. Do đó các nút T' cần sao chép không có dạng một số cây con; chi phí thô bạo bóc các rake tree nối với compress tree là không chấp nhận được.

Tuy vậy, gặp bài toán tách đường không có nghĩa là hết cách. Xét ví dụ sau.

Ví dụ 1.7. Cho một cây T . Duy trì nhiều đa tập S_i , các phần tử thuộc $\{1, \dots, n\}$. Hỗ trợ:

1. tách mọi phần tử trong S_p có chỉ số nằm trên đường từ u tới v sang S_q ;
2. với mọi đỉnh x trên đường từ u tới v , chèn x vào S_p đúng k lần;
3. hợp nhất hai tập;
4. hỏi tổng số lần xuất hiện trong S_p của mọi đỉnh x trên đường từ u tới v .

Vì chính cấu trúc của static Top Tree gây ra vấn đề trên, ta cần mở rộng thành một cấu trúc mới. Bài này không cần duy trì thông tin cây con, nên thông tin trong rake tree con của một compress node không quan trọng. Nói cách khác, không nhất thiết để rake tree làm con của compress node hiện tại.

Xét duy trì static Top Tree hai tầng: tách mỗi nút ban đầu thành u và u' , lần lượt biểu thị tầng thứ nhất và tầng thứ hai. Cấu trúc bên trong tầng thứ nhất giống static Top Tree thường. Bên trong tầng thứ hai chỉ xây compress tree. Với mỗi gốc rt của một compress tree, có thêm một cạnh trở tới rt' .

Chỉ tầng thứ hai duy trì thông tin chuỗi; tầng thứ nhất chỉ dùng để định vị và cân bằng trọng số. Rõ ràng cấu trúc trên có số con của mỗi nút là $O(1)$ và chiều cao $O(\log n)$.

Với static Top Tree hai tầng mở nút động, định vị như sau. Trước hết xây một static Top Tree đầy đủ, rồi định vị các nút tương ứng với đường u tới v trên đó. Sau đó tìm các nút tương ứng của những vị trí ấy trong static Top Tree tầng thứ hai, rồi mở nút động tới các nút được định vị và tổ tiên của chúng, bao gồm cả các nút ở tầng thứ nhất.

Khi đã có static Top Tree hai tầng mở nút động, ta có thể hợp nhất và hỗ trợ tách ra một đường. Thao tác hợp nhất tương tự static Top Tree thường. Với thao tác tách, đường cần tách nằm trên một số đoạn của các compress tree ở tầng thứ hai; vì compress tree tầng thứ hai không chứa rake tree dư thừa, các cây con này có thể được sao chép trực tiếp thành các nút tương ứng của T' . Tổng số nút dùng chung trên những compress tree này, kể cả các nút dùng chung trên static Top Tree tầng thứ nhất, là $O(\log n)$, nên có thể tạo mới chúng. Các nút khác vẫn giữ thông tin ngoài đường.

Thiết kế thế năng

$$\Phi = \sum_i |T_i|,$$

tức tổng số nút của các static Top Tree. Có thể phân tích được thời gian là $O(n \log n)$.

2. Hợp nhất và tách cấu trúc dữ liệu động

2.1. Dẫn nhập

Cấu trúc dữ liệu động có phạm vi ứng dụng rất rộng; do đặc điểm động, chúng có thể ứng phó với nhiều loại thao tác. Nhưng chính sự bất định của cấu trúc cũng làm việc hợp nhất trở nên khó khăn. Hợp nhất heuristic là cách ứng phó phổ biến nhất, nhưng vì có thêm một thừa số log, lại không hỗ trợ tách, nên chưa thỏa đáng. Phần sau giới thiệu một lớp bài toán trên cây cân bằng.

2.2. Hợp nhất cây cân bằng

Ví dụ 2.1. Cần duy trì nhiều dãy số nguyên tăng dần, hỗ trợ:

1. cộng mọi phần tử trong một dãy với cùng một hằng số;
2. đổi dấu mọi phần tử trong một dãy và đảo ngược cả dãy;
3. trộn hai dãy thành một dãy;
4. hỏi tổng một đoạn của một dãy.

Có nhiều loại cây cân bằng, và cách hợp nhất cũng khác nhau. Bài viết chỉ giới thiệu hợp nhất treap. Trước hết chứng minh một số bổ đề về treap.

Bổ đề 2.1. Treap có n nút có độ sâu kỳ vọng $O(\log n)$.

Giả sử sau khi sắp xếp mọi phần tử theo key, các giá trị ngẫu nhiên lần lượt là $b_1, b_2, \dots, b_n$ (không mất tổng quát, giả sử mọi phần tử đôi một khác nhau). Với x và y , xét hai trường hợp:

1. nếu $x \leq y$, thì x là tổ tiên của y khi và chỉ khi

$$b_x < \min_{x < j \leq y} b_j;$$

2. nếu $x > y$, thì x là tổ tiên của y khi và chỉ khi

$$b_x < \min_{y \leq j < x} b_j.$$

Xác suất của sự kiện này chính là xác suất x có giá trị ưu tiên nhỏ nhất, tức

$$\frac{1}{|x - y| + 1}.$$

Độ sâu của x có thể định nghĩa là số tổ tiên của x , nên kỳ vọng số tổ tiên là

$$\sum_{i=1}^n \frac{1}{|x - i| + 1} \sim O(\log n).$$

Định nghĩa 2.1. *Finger search* là thao tác trong một cấu trúc dữ liệu mà nếu gọi $d(x, y)$ là số phần tử giữa x và y (bao gồm x, y), thì sau khi đã xác định vị trí của x , có thể truy cập y trong thời gian

$$O(f(d(x, y))).$$

Hiển nhiên nếu $f(x) > \log x$ thì không có ý nghĩa, vì tìm kiếm lại từ đầu còn tốt hơn. Vì vậy phần sau mặc định $f(x) = \log x$.

Bổ đề 2.2. Trên treap, độ dài đường đi từ x tới y có kỳ vọng

$$O(\log d(x, y)).$$

Đường đi từ x tới y có độ dài bằng: số nút là tổ tiên của x nhưng không là tổ tiên của y , cộng với số nút là tổ tiên của y nhưng không là tổ tiên của x . Theo đối xứng, chỉ cần xét về đầu.

Giả sử mọi phần tử của treap theo thứ tự tăng dần là

$$a_1, a_2, \dots, a_n,$$

trong đó $a_i = x, a_j = y$. Rõ ràng $d(x, y) = j - i + 1$. Xét một nút a_k và tính xác suất nó thỏa “là tổ tiên của x nhưng không là tổ tiên của y ”, rồi cộng lại. Ta phân loại theo k :

1. $1 \leq k \leq i$. Điều này tương đương k là giá trị nhỏ nhất trên $[k, i]$, và giá trị nhỏ nhất trên $[k, i]$ lớn hơn giá trị nhỏ nhất trên $(i, j]$. Xác suất phần trước là $1/(i - k + 1)$, phần sau là $(j - i)/(j - k + 1)$. Tích lại được

$$\frac{j - i}{(i - k + 1)(j - k + 1)} = \frac{1}{i - k + 1} - \frac{1}{j - k + 1}.$$

Cộng lại cho k cho ra $O(\log d(i, j))$.

2. $i < k < j$. Tương tự, xác suất là

$$\frac{j - k}{(k - i + 1)(j - i + 1)} = \frac{1}{k - i + 1} - \frac{1}{j - i + 1}.$$

Cộng lại cũng được $O(\log d(i, j))$.

Từ đó thấy phương pháp finger search trên treap là: từ x liên tục nhảy lên cho tới $\text{lca}(x, y)$, rồi đi xuống định vị tới y .

Xét hợp nhất hai treap T_1, T_2 , kích thước lần lượt là n, m . Không mất tổng quát giả sử $m \leq n$. Gọi

$$p_1 < p_2 < \dots < p_m$$

là thứ hạng của các phần tử trong T_2 giữa $n + m$ phần tử. Nếu chèn lần lượt theo thứ tự, thời gian là

$$O\left(\sum_{i=1}^m \log(p_i - p_{i-1})\right), \quad p_0 = 0.$$

Do tính lồi của $\log$, giá trị lớn nhất của biểu thức trên là

$$O\left(m \log \frac{n}{m}\right).$$

Thiết kế thế năng

$$\Phi = \sum_i |T_i| \log |T_i|.$$

Mỗi lần hợp nhất làm thế năng thay đổi

$$\begin{aligned} & (n + m) \log(n + m) - n \log n - m \log m \\ & \geq m \log(n + m) - m \log m \geq m \log \frac{n}{m}. \end{aligned}$$

Tức ta dùng thời gian $O(m \log(n/m))$ để làm thế năng tăng ít nhất $m \log(n/m)$. Vì thế năng có chặn trên $n \log n$, tổng thời gian là $O(n \log n)$.

2.3. Tách cây cân bằng

Ví dụ 2.2. Cần duy trì nhiều dãy số nguyên tăng dần, hỗ trợ:

1. với một dãy a , cho trọng số k , tách a thành hai dãy gồm các phần tử nhỏ hơn k và các phần tử không nhỏ hơn k ;
2. cộng mọi phần tử trong một dãy với cùng một hằng số;
3. đổi dấu mọi phần tử trong một dãy và đảo ngược cả dãy;
4. trộn hai dãy thành một dãy;
5. hỏi tổng một đoạn của một dãy.

Tách cây cân bằng cũng nghĩa là chia theo miền giá trị thành hai phần không giao nhau. Với treap, việc này có thể xử lý trong $O(\log n)$.

Với thao tác hợp nhất, ta xét trộn thô bạo hai dãy. Mỗi lần lấy phần tử đầu dãy, tức giá trị nhỏ nhất; giả sử là $x \leq y$. Sau đó tách khỏi T_1 mọi phần tử có giá trị $\leq y$; lúc này nhất định có $x > y$, rồi tách khỏi T_2 mọi phần tử có giá trị $\leq x$. Lặp lại như vậy cho tới khi hoàn tất trộn. Tiếp theo chứng minh độ phức tạp của cách làm này.

Với một cây T , viết trọng số các nút theo thứ tự tăng dần là

$$a_1, \dots, a_n.$$

Xét thế năng

$$\Phi = \sum_T \sum_{i=1}^{n+1} \log(a_i - a_{i-1}),$$

trong đó $a_0 = 0$, $a_{n+1} = v$, lần lượt biểu thị cận dưới và cận trên của miền giá trị.

Ta vẫn dùng cách làm của phần trước. Đặt

$$k = \sum_i [c_i \neq c_{i+1}],$$

trong đó $c_i \in \{0, 1\}$ biểu thị a_i thuộc T_1 hay T_2 . Từ đó có thể chứng minh thời gian hợp nhất hai cây là $O(k \log n)$, còn thế năng giảm $O(k)$. Φ có chặn trên $O(n \log v)$, và các thao tác sửa đổi không ảnh hưởng tới thế năng, nên tổng thời gian là

$$O(n \log n \log v).$$

Tổng kết

Bài viết giới thiệu việc hợp nhất và tách cấu trúc dữ liệu, chia theo cấu trúc tĩnh và động. Phần tĩnh giới thiệu tư tưởng hợp nhất và tách một lớp cấu trúc dữ liệu tĩnh: xuất phát từ cây đoạn, rồi mở rộng sang cây phân trị đỉnh, cây phân trị cạnh và static Top Tree; đồng thời đưa ra thuật toán hợp nhất và tách trên static Top Tree. Phần động giới thiệu cách cây cân bằng xử lý nhược điểm do tính động gây ra, và từ góc nhìn finger search giải quyết nhiều bài toán hợp nhất cây cân bằng. Các kỹ thuật này có triển vọng ứng dụng rộng trong thi tin học.

Tuy nhiên, trong lý thuyết cấu trúc dữ liệu vẫn còn nhiều vấn đề bỏ ngỏ. Tác giả hy vọng bài viết có thể gợi mở để độc giả tiếp tục nghiên cứu sâu hơn các lý thuyết liên quan và làm chúng trưởng thành hơn.

Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn các thầy Ye Jinyi, Liang Lei và Gu Duoyu của Trường Trung học trực thuộc Đại học Nhân dân Trung Quốc đã quan tâm và hướng dẫn.

Cảm ơn gia đình và bạn bè đã ủng hộ, động viên tác giả.

Cảm ơn các bạn Lou Yuzhan và Hu Zhaoyang đã thảo luận và đem lại gợi ý cho tác giả.

Cảm ơn các bạn Lou Yuzhan và Ye Liqi đã đọc duyệt bài viết.

Tài liệu tham khảo

1. Cheng Siyuan. *Bàn về ứng dụng của static Top Tree trên cây và đồ thị series-parallel tổng quát*. Tập luận văn đội tuyển quốc gia, 2023.
2. Zhao Yuyang. *Lý thuyết, cách dùng và cài đặt các nội dung liên quan tới Top tree*. <https://uoj.ac/blog/4912>.
3. Wikipedia. *Finger search*. https://en.wikipedia.org/wiki/Finger_search.
4. Hu Zhaoyang. *Cây cân bằng và finger search*. cnblogs.com/PYWBKTDA/p/14576937.
5. Zhou Xin. *Về một bài toán liên quan tới cây cân bằng*. <https://www.luogu.com.cn/blog/user19567/guan-yu-yi-ge-ping-heng-shu-xiang-guan-di-wen-ti>.

Bài 4. Bàn về nhân tử Lagrange và đối ngẫu trong OI

Shi Kaicheng, Trường Trung học Zhenhai, Ninh Ba

Tóm tắt

Trong các bài toán tối ưu, nhân tử Lagrange là các biến tham số được đưa vào khi cần tìm cực trị của một hàm dưới ràng buộc phương trình; bài toán đối ngẫu Lagrange là một bài toán tối ưu khác tương ứng với bài toán tối ưu ban đầu. Bài viết này giới thiệu phương pháp nhân tử Lagrange, đồng thời chỉ ra mối liên hệ nội tại giữa đối ngẫu Lagrange, đối ngẫu quy hoạch tuyến tính và bài toán cân bằng Nash.

1. Dẫn nhập

Nhân tử Lagrange là một công cụ do Lagrange đưa ra để xử lý các bài toán tối ưu có ràng buộc. Ý tưởng của nó là thêm các biến phụ để đưa ràng buộc vào biểu thức tối ưu, từ đó cung cấp một phương pháp hiệu quả cho loại bài toán này và dẫn tới phương pháp nhân tử Lagrange cũng như bài toán đối ngẫu Lagrange.

Đối ngẫu Lagrange là một phương pháp đối ngẫu cho bài toán tối ưu. Bản thân nó không thường xuất hiện trực tiếp trong tin học, nhưng một trường hợp đặc biệt của nó là đối ngẫu quy hoạch tuyến tính; xuất phát từ nhân tử Lagrange, ta có thể nhanh chóng suy ra dạng đối ngẫu của bài toán quy hoạch tuyến tính. Với bài toán đối ngẫu Lagrange còn có định lý minimax, đóng vai trò quan trọng trong cân bằng Nash của trò chơi hai người tổng bằng không. Bài viết sẽ xoay quanh nhân tử Lagrange và đối ngẫu Lagrange, rồi trình bày quan hệ giữa các lớp bài toán trên.

2. Ký hiệu cơ bản

$\partial f / \partial x$ biểu thị đạo hàm riêng của hàm f theo biến x .

$\nabla f(x_1, \dots, x_n)$ biểu thị gradient của f tại $x_1, \dots, x_n$, tức vectơ gồm mọi đạo hàm riêng của f .

$(x_1, x_2, \dots, x_n)$ biểu thị một vectơ n chiều, trong đó tọa độ thứ i là x_i .

$\mathbb{R}^n$ biểu thị không gian Euclid n chiều.

$\max f$ và $\min f$ lần lượt biểu thị giá trị lớn nhất và nhỏ nhất của hàm f . Nếu f không bị chặn trên thì xem $\max f = +\infty$; nếu f không bị chặn dưới thì xem $\min f = -\infty$.

$\sum a + b$ tương đương với $(\sum a) + b$; $\max a + b$ tương đương với $\max(a + b)$, và $\min$ được hiểu tương tự $\max$.

Với $x, y \in \mathbb{R}^n$, $x \leq y$ đúng khi và chỉ khi $\forall i \in [1, n] \cap \mathbb{Z}$, $x_i \leq y_i$. Các ký hiệu $x \geq y$, $x < y$, $x > y$ được hiểu tương tự.

Với ma trận A kích thước $n \times m$, A_i biểu thị hàng thứ i của ma trận, $1 \leq i \leq n$, tức một vectơ hàng m chiều.

3. Nhân tử Lagrange

3.1. Dẫn nhập

Xét một lớp bài toán tối ưu có ràng buộc đẳng thức:

$$\begin{aligned} \max \quad & f(x) \\ \text{s.t.} \quad & g_1(x) = 0, \\ & g_2(x) = 0, \\ & \dots \\ & g_m(x) = 0. \end{aligned} \tag{1}$$

Trong đó $x = (x_1, x_2, \dots, x_n)$ là một vectơ thực n chiều, còn $f, g_1, \dots, g_m$ đều là các hàm liên tục khả vi từ $\mathbb{R}^n$ tới $\mathbb{R}$.

Với loại bài toán này, ta có thể thêm hệ số vào trước các ràng buộc đẳng thức, biến bài toán ban đầu thành dạng

$$\max_x \min_{\lambda_i} f(x) + \lambda_1 g_1(x) + \dots + \lambda_m g_m(x). \tag{2}$$

Các hệ số $\lambda_1, \dots, \lambda_m \in \mathbb{R}$ được gọi là nhân tử Lagrange; hàm

$$F(x, \lambda_1, \dots, \lambda_m) = f(x) + \lambda_1 g_1(x) + \dots + \lambda_m g_m(x)$$

được gọi là hàm Lagrange.

Định lý 3.1. Công thức (1) và (2) là tương đương.

Chứng minh. Chú ý rằng nếu $g_i(x) \neq 0$, chỉ cần cho λ_i tiến tới $+\infty$ hoặc $-\infty$ là sẽ có

$$\min_{\lambda_i} f(x) + \lambda_1 g_1(x) + \dots + \lambda_m g_m(x) = -\infty.$$

Vì vậy, sau khi bọc thêm một lớp $\max_x$ ở ngoài, mọi x đạt giá trị lớn nhất đều phải thỏa $g_i(x) = 0$.

3.2. Định lý nhân tử Lagrange

Định lý 3.2 (định lý nhân tử Lagrange). Giả sử $f, g_1, \dots, g_m : \mathbb{R}^n \rightarrow \mathbb{R}$ đều là các hàm liên tục khả vi. Khi đó, với mọi điểm cực trị x^* của f dưới điều kiện $\forall i \in [1, m] \cap \mathbb{N}, g_i(x^*) = 0$, các vectơ

$$\nabla f(x^*), \nabla g_1(x^*), \dots, \nabla g_m(x^*)$$

đều tuyến tính phụ thuộc.

Do khuôn khổ bài viết có hạn, và chứng minh định lý cần dùng nội dung giải tích như định lý hàm ẩn, bài viết không trình bày chứng minh ở đây. Chứng minh đầy đủ có thể xem trong tài liệu [1].

Hệ quả 3.1. Với mọi điểm cực trị x^* của f dưới các điều kiện

$$g_i(x_1, \dots, x_n) = 0, \quad 1 \leq i \leq m,$$

nếu $\nabla g_1(x^*), \dots, \nabla g_m(x^*)$ tuyến tính độc lập, thì tồn tại duy nhất một bộ hệ số $\lambda_1, \dots, \lambda_m$ sao cho

$$\nabla f(x^*) = \lambda_1 \nabla g_1(x^*) + \dots + \lambda_m \nabla g_m(x^*).$$

Hệ quả này là một dạng tương đương của định lý 3.2. Kết luận của nó thường được viết thành: với hàm Lagrange

$$F(x_1, \dots, x_n, \lambda_1, \dots, \lambda_m) = f(x_1, \dots, x_n) + \lambda_1 g_1(x_1, \dots, x_n) + \dots + \lambda_m g_m(x_1, \dots, x_n),$$

mọi đạo hàm riêng theo các x_i và λ_i đều bằng 0.

Điều kiện $\nabla g_1(x^*), \dots, \nabla g_m(x^*)$ tuyến tính độc lập không thể bỏ qua. Nếu bỏ qua điều kiện này, hãy xét

$$f(x, y) = x + y, \quad g(x, y) = x^2 + y^2,$$

và tìm giá trị lớn nhất của $f(x, y)$ khi $g(x, y) = 0$. Hàm Lagrange khi đó là

$$F(x, y, \lambda) = x + y + \lambda(x^2 + y^2),$$

không có điểm dừng nào, nhưng dưới điều kiện $g(x, y) = 0$, giá trị lớn nhất của $f(x, y)$ phải là $f(0, 0) = 0$. Nguyên nhân là tại $x = 0, y = 0$ ta có $\nabla g(x, y) = 0$, không thỏa điều kiện tuyến tính độc lập của ∇g . Dạng chặt chẽ của định lý nhân tử Lagrange chưa phổ biến trong OI; khi sử dụng định lý này, người đọc nên lưu ý điều kiện trên có được thỏa hay không.

3.3. Phương pháp nhân tử Lagrange

Với bài toán

$$\begin{aligned} \max \quad & f(x) \\ \text{s.t.} \quad & g_1(x) = 0, g_2(x) = 0, \dots, g_m(x) = 0, \quad m < n, \end{aligned}$$

nếu đáp án tồn tại, ta có thể dùng phương pháp nhân tử Lagrange để giải. Quy trình cụ thể như sau.

Định nghĩa hàm Lagrange

$$F(x, \lambda_1, \dots, \lambda_m) = f(x) + \lambda_1 g_1(x) + \dots + \lambda_m g_m(x).$$

Lập $n + m$ phương trình

$$\frac{\partial F}{\partial x_i}(x) = 0, \quad \frac{\partial F}{\partial \lambda_i}(x) = 0,$$

trong đó $\partial F / \partial \lambda_i = 0$ tương đương $g_i(x) = 0$. Lấy tất cả x là nghiệm của hệ, cùng mọi x làm cho $\nabla g_1(x), \dots, \nabla g_m(x)$ tuyến tính phụ thuộc nhưng vẫn thỏa $g_i(x) = 0$. Thay từng x đó vào $f(x)$ rồi lấy giá trị lớn nhất; giá trị này chính là cực đại của $f(x)$ dưới các ràng buộc $g_i(x) = 0$.

Ví dụ 3.1. Tìm giá trị lớn nhất của

$$f(x, y) = -x^2 + 4x + y$$

dưới điều kiện $x^2 + 2y^2 = 30$.

Lời giải. Đặt

$$g(x, y) = x^2 + 2y^2 - 30,$$

và xét

$$F(x, y, \lambda) = f(x, y) + \lambda g(x, y) = -x^2 + 4x + y + \lambda(x^2 + 2y^2 - 30).$$

Nếu $\nabla g(x, y)$ tuyến tính phụ thuộc, tức $\nabla g(x, y) = 0$, thì

$$\frac{\partial g}{\partial x} = 2x = 0, \quad \frac{\partial g}{\partial y} = 4y = 0,$$

suy ra $x = 0, y = 0$. Khi đó $g(x, y) = -30$, không thỏa ràng buộc. Do đó ta có thể xem $\nabla g(x, y)$ tuyến tính độc lập tại cực trị của $f(x, y)$, và dùng định lý nhân tử Lagrange.

Khi $\nabla F(x, y, \lambda) = 0$, ta có hệ

$$\frac{\partial F}{\partial x} = 2(\lambda - 1)x + 4 = 0, \quad \frac{\partial F}{\partial y} = 4\lambda y + 1 = 0, \quad \frac{\partial F}{\partial \lambda} = x^2 + 2y^2 - 30 = 0.$$

Hệ phương trình này có bốn nghiệm. Thay vào $f(x, y)$, nghiệm tối ưu là

$$x \approx 1.8715, \quad y \approx 3.6399, \quad \lambda \approx -0.0687, \quad f(x, y) \approx 7.6234.$$

Vì x, y bị chặn khi $g(x, y) = 0$, giá trị lớn nhất của $f(x, y)$ tồn tại. Do đó thảo luận trên bao quát toàn bộ trường hợp, và cực đại toàn cục của $f(x, y)$ dưới điều kiện $g(x, y) = 0$ là khoảng 7.6234.

3.3.1. [NOI 2012] Đạp xe Tứ Xuyên–Tây Tạng

Mô tả bài toán. Cho các số thực s_i, k_i, v_i', E . Cần tối thiểu hóa

$$f(v_1, \dots, v_n) = \sum_{i=1}^n \frac{s_i}{v_i},$$

trong đó $v_1, \dots, v_n$ đều là số thực dương và cần thỏa

$$\sum_{i=1}^n k_i(v_i - v_i')^2 s_i \leq E.$$

Phạm vi dữ liệu.

$$1 \leq n \leq 10000, \quad 0 \leq E \leq 10^8, \quad s_i \in [0, 10^5], \quad k_i \in (0, 15], \quad v_i' \in (-100, 100).$$

Phân tích. Phân tích sơ bộ cho thấy khi $f(v_1, \dots, v_n)$ đạt giá trị nhỏ nhất dưới điều kiện của đề, nhất định có

$$\sum_{i=1}^n k_i(v_i - v_i')^2 s_i = E \quad \text{và} \quad v_i \geq v_i'.$$

Do đó định nghĩa

$$g(v_1, \dots, v_n) = \sum_{i=1}^n k_i(v_i - v_i')^2 s_i - E,$$

và dùng phương pháp nhân tử Lagrange để tìm điểm dừng của

$$F(v_1, \dots, v_n, \lambda) = f(v_1, \dots, v_n) - \lambda g(v_1, \dots, v_n),$$

đồng thời xử lý riêng trường hợp ∇g tuyến tính phụ thuộc. Tuy đề có ràng buộc $0 < v_i$, phương pháp nhân tử Lagrange sẽ tìm mọi điểm cực trị; ta chỉ cần xét các điểm cực trị thỏa $0 < v_i$.

Lập phương trình:

$$\frac{\partial F}{\partial v_i} = -\frac{s_i}{v_i^2} + 2\lambda s_i k_i (v_i - v'_i) = 0, \quad \frac{\partial F}{\partial \lambda} = \sum_{i=1}^n k_i (v_i - v'_i)^2 s_i - E = 0.$$

Sắp xếp lại được

$$2\lambda s_i k_i (v_i - v'_i) = \frac{s_i}{v_i^2}, \quad (3)$$

$$\sum_{i=1}^n k_i (v_i - v'_i)^2 s_i = E. \quad (4)$$

Với một λ xác định, có thể giải mọi v_i từ (3), rồi thay vào (4) để kiểm tra. Xét đồ thị của hai vế trong (3): khi $\lambda > 0$, hai hàm có đúng một giao điểm trên $v_i > 0$, và nhất định $v_i \geq v'_i$; khi $\lambda \leq 0$, hai vế không có giao điểm trên miền $v_i \geq v'_i$. Vì vậy nhất định $\lambda > 0$ và $v_i \geq v'_i$. Từ đồ thị cũng thấy khi λ tăng, v_i giảm, nên vế trái của (4) cũng giảm.

Do đó có thể nhị phân λ trên miền số thực dương, dùng (3) để tính v_i rồi thay vào (4). Nếu vế trái của (4) lớn hơn E , tăng λ ; ngược lại giảm λ . Trường hợp ∇g tuyến tính phụ thuộc xảy ra khi và chỉ khi mọi $v_i = v'_i$; khi đó xử lý riêng.

3.3.2. [CCPC 2023 Harbin] Energy Distribution

Mô tả bài toán. Cho w_{ij} với $1 \leq i < j \leq n$. Cần dựng một bộ x_i thỏa

$$0 \leq x_i \leq 1, \quad \sum_{i=1}^n x_i = 1,$$

và tối đa hóa

$$\sum_{i=1}^n \sum_{j=i+1}^n x_i x_j w_{ij}.$$

Phạm vi dữ liệu.

$$n \leq 10, \quad 0 \leq w_{ij} \leq 1000,$$

trong đó w_{ij} và x_i đều là số thực.

Phân tích. Đặt

$$f(x_1, \dots, x_n) = \sum_{i=1}^n \sum_{j=i+1}^n x_i x_j w_{ij}, \quad g(x_1, \dots, x_n) = \sum_{i=1}^n x_i - 1.$$

Đáp án là giá trị lớn nhất của $f(x_1, \dots, x_n)$ dưới điều kiện $g(x_1, \dots, x_n) = 0$ và $0 \leq x_i$.

Nếu không có ràng buộc $0 \leq x_i$, ta có thể dùng trực tiếp phương pháp nhân tử Lagrange để tìm cực đại. Khi có các ràng buộc bất đẳng thức $0 \leq x_i$, nếu chỉ dùng phương pháp nhân tử Lagrange và kiểm tra mọi điểm cực trị thỏa $0 \leq x_i$, có thể bỏ sót đáp án cuối cùng. Nguyên nhân là phương pháp này chỉ tìm cực trị trong toàn miền định nghĩa; nếu tại một điểm nào đó có $x_i = 0$, điểm đó có thể không phải cực trị

trong toàn miền, nhưng lại là cực trị trong miền thực tế. Ở đây, toàn miền là miền không có ràng buộc $0 \leq x_i$, còn miền thực tế là miền có ràng buộc này.

Vì vậy, ta có thể liệt kê những x_i bằng 0, đưa các điều kiện $x_i = 0$ đó vào ràng buộc đẳng thức, rồi dùng phương pháp nhân tử Lagrange và khử Gauss để giải. Cách này bảo đảm tìm được mọi điểm cực trị trong miền thực tế; lấy điểm tốt nhất trong số đó là đáp án. Khi khử Gauss không có nghiệm duy nhất, nghĩa là tồn tại biến tự do; có thể điều chỉnh để một x_i nào đó trở thành 0 trong khi giá trị hàm không đổi, nên có thể bỏ qua các trường hợp không có nghiệm duy nhất. Có thể thấy chỉ cần các x_i không đồng thời bằng 0, ta luôn có ∇g tuyến tính độc lập, vì thế phương pháp nhân tử Lagrange không bỏ sót đáp án nào. Tổng độ phức tạp là $O(2^n n^3)$.

4. Đối ngẫu Lagrange

4.1. Định nghĩa

Với $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$ và $h : \mathbb{R}^n \rightarrow \mathbb{R}^p$, xét bài toán tối ưu

$$\begin{aligned} \max \quad & f(x) \\ \text{s.t.} \quad & g(x) \geq 0, \\ & h(x) = 0. \end{aligned} \tag{5}$$

Định nghĩa hàm Lagrange

$$F : \mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R}^p \rightarrow \mathbb{R}$$

của nó là

$$F(x, \lambda, \nu) = f(x) + \lambda^T g(x) + \nu^T h(x).$$

Định nghĩa hàm đối ngẫu Lagrange

$$L : \mathbb{R}^m \times \mathbb{R}^p \rightarrow \mathbb{R}$$

là

$$L(\lambda, \nu) = \max_x F(x, \lambda, \nu).$$

Nếu $F(x, \lambda, \nu)$ không có chặn trên theo x , đặt $L(\lambda, \nu) = +\infty$.

Bài toán đối ngẫu Lagrange của (5) được định nghĩa là

$$\begin{aligned} \min \quad & L(\lambda, \nu) \\ \text{s.t.} \quad & \lambda \geq 0. \end{aligned} \tag{6}$$

Với bài toán tìm $\min f(x)$ dưới ràng buộc, cũng có thể định nghĩa bài toán đối ngẫu bằng cách tương tự, chẳng hạn xét đối ngẫu của $\max -f(x)$.

4.2. Tính chất

4.2.1. Đối ngẫu yếu.

Định lý 4.1 (bất đẳng thức min–max). Với $F : D_x \times D_y \rightarrow \mathbb{R}$, ta có

$$\max_{y \in D_y} \min_{x \in D_x} F(x, y) \leq \min_{x \in D_x} \max_{y \in D_y} F(x, y).$$

Chứng minh. Với mọi i, j ,

$$\begin{aligned} F(i, j) &\leq F(i, j), \\ \min_x F(x, j) &\leq F(i, j), \\ \max_y \min_x F(x, y) &\leq \max_y F(i, y). \end{aligned}$$

Lấy $\min_i$ ở vế phải cho ra

$$\max_y \min_x F(x, y) \leq \min_x \max_y F(x, y).$$

Định lý 4.2 (định lý đối ngẫu yếu). Gọi giá trị tối ưu của (5) là s^* , và giá trị tối ưu của bài toán đối ngẫu Lagrange là t^* . Khi đó luôn có $s^* \leq t^*$.

Chứng minh. Tương tự định lý 3.1, ta có thể thêm nhân tử Lagrange để đưa các ràng buộc $g(x) \geq 0$ và $h(x) = 0$ vào hàm tối ưu:

$$\begin{aligned} s^* &= \max_{g(x) \geq 0, h(x)=0} f(x) \\ &= \max_x \min_{\lambda \geq 0, \nu} f(x) + \lambda^T g(x) + \nu^T h(x) \\ &\leq \min_{\lambda \geq 0, \nu} \max_x f(x) + \lambda^T g(x) + \nu^T h(x) \\ &= \min_{\lambda \geq 0, \nu} \max_x F(x, \lambda, \nu) \\ &= \min_{\lambda \geq 0, \nu} L(\lambda, \nu) \\ &= t^*. \end{aligned}$$

Quan sát chứng minh này, ta thấy bài toán đối ngẫu chính là bài toán thu được bằng cách hoán đổi thứ tự của $\min$ và $\max$.

4.2.2. Đối ngẫu mạnh. Nếu dưới ký hiệu của định lý 4.2 có $s^* = t^*$, ta nói đối ngẫu mạnh được thỏa.

Đối ngẫu mạnh không phải lúc nào cũng đúng, nhưng khi các hàm có một số tính chất tốt, nó sẽ đúng; ví dụ bài toán quy hoạch tuyến tính có đối ngẫu mạnh. Đối ngẫu Lagrange và đối ngẫu mạnh có ứng dụng rất quan trọng trong tối ưu lồi, và từ đó có thể suy ra điều kiện tối ưu Karush–Kuhn–Tucker (KKT) [6], vốn rất giống định lý nhân tử Lagrange. Tuy nhiên phần này không liên quan nhiều tới OI, nên bài viết lược bỏ; chi tiết có thể xem [2].

Dù vậy, có một định lý minimax có dạng tương tự đối ngẫu Lagrange và khá thuận tiện trong OI.

Bổ đề 4.1. Nếu $D_x \subseteq \mathbb{R}^n$, $D_y \subseteq \mathbb{R}^m$ là các tập lồi compact, hàm liên tục $F : D_x \times D_y \rightarrow \mathbb{R}$ là hàm lồi theo x và là hàm lõm theo y , thì tồn tại $x_0 \in D_x$, $y_0 \in D_y$ sao cho

$$\min_{x \in D_x} F(x, y_0) = F(x_0, y_0) = \max_{y \in D_y} F(x_0, y).$$

Ở đây, một tập con của $\mathbb{R}^n$ là compact khi và chỉ khi nó đóng và bị chặn; tập $S \subseteq \mathbb{R}^n$ là lồi khi với mọi $p, q \in S$ và mọi $\lambda \in [0, 1]$, ta có $\lambda p + (1 - \lambda)q \in S$. Hàm $f : C \rightarrow \mathbb{R}$ là lồi nếu

$$f(\lambda p + (1 - \lambda)q) \leq \lambda f(p) + (1 - \lambda)f(q),$$

và là lõm nếu dấu bất đẳng thức trên đổi chiều.

Chứng minh bổ đề này khá khó, nên bài viết không trình bày; có thể xem [4].

Định lý 4.3 (định lý minimax). Nếu $D_x \subseteq \mathbb{R}^n$, $D_y \subseteq \mathbb{R}^m$ là các tập lồi compact, hàm liên tục $F : D_x \times D_y \rightarrow \mathbb{R}$ là hàm lồi theo x và là hàm lõm theo y , thì

$$\max_{y \in D_y} \min_{x \in D_x} F(x, y) = \min_{x \in D_x} \max_{y \in D_y} F(x, y).$$

Chứng minh. Theo bổ đề 4.1, tồn tại $x_0 \in D_x$, $y_0 \in D_y$ thỏa

$$\max_{y \in D_y} F(x_0, y) = F(x_0, y_0) = \min_{x \in D_x} F(x, y_0).$$

Do đó

$$\begin{aligned} \min_{x \in D_x} \max_{y \in D_y} F(x, y) &\leq \max_{y \in D_y} F(x_0, y) \\ &= F(x_0, y_0) \\ &= \min_{x \in D_x} F(x, y_0) \\ &\leq \max_{y \in D_y} \min_{x \in D_x} F(x, y). \end{aligned}$$

Từ định lý đối ngẫu yếu,

$$\max_{y \in D_y} \min_{x \in D_x} F(x, y) \leq \min_{x \in D_x} \max_{y \in D_y} F(x, y).$$

Suy ra hai vế bằng nhau.

4.2.3. Tính lồi.

Định lý 4.4. Hàm đối ngẫu Lagrange

$$L(\lambda, \nu) = \max_x F(x, \lambda, \nu)$$

là hàm lồi.

Chứng minh. Với $\lambda_1, \lambda_2 \in \mathbb{R}^m$, $\nu_1, \nu_2 \in \mathbb{R}^p$ và $a \in [0, 1]$, ta có

$$\begin{aligned} &L(a\lambda_1 + (1-a)\lambda_2, a\nu_1 + (1-a)\nu_2) \\ &= \max_x F(x, a\lambda_1 + (1-a)\lambda_2, a\nu_1 + (1-a)\nu_2). \end{aligned}$$

Giả sử giá trị lớn nhất đạt tại x^* . Khi đó

$$\begin{aligned} &L(a\lambda_1 + (1-a)\lambda_2, a\nu_1 + (1-a)\nu_2) \\ &= F(x^*, a\lambda_1 + (1-a)\lambda_2, a\nu_1 + (1-a)\nu_2) \\ &= f(x^*) + (a\lambda_1 + (1-a)\lambda_2)^T g(x^*) + (a\nu_1 + (1-a)\nu_2)^T h(x^*) \\ &= a(f(x^*) + \lambda_1^T g(x^*) + \nu_1^T h(x^*)) \\ &\quad + (1-a)(f(x^*) + \lambda_2^T g(x^*) + \nu_2^T h(x^*)) \\ &= aF(x^*, \lambda_1, \nu_1) + (1-a)F(x^*, \lambda_2, \nu_2) \\ &\leq aL(\lambda_1, \nu_1) + (1-a)L(\lambda_2, \nu_2). \end{aligned}$$

4.3. Ví dụ

4.3.1. [POJ Monthly 2015.5] Min-Max.

Mô tả bài toán. Đặt

$$F(x_1, \dots, x_n) = \sum_{i=1}^n \mu_i x_i.$$

Cho p_i, q_i, C . Cần dựng một bộ μ_i thỏa

$$0 \leq \mu_i \leq 1, \quad \sum_{i=1}^n \mu_i = 1, \quad F(p_1, \dots, p_n) = C,$$

và tìm giá trị lớn nhất, nhỏ nhất của $F(q_1, \dots, q_n)$ dưới các ràng buộc này.

Phạm vi dữ liệu.

$$1 \leq n \leq 50000, \quad |p_i|, |q_i| \leq 1000.$$

Phân tích. Lấy bài toán tìm giá trị lớn nhất làm ví dụ. Thêm nhân tử Lagrange cho ràng buộc $F(p_1, \dots, p_n) = C$, bài toán ban đầu trở thành

$$\max_{\mu_i} \min_{\lambda} F(q_1, \dots, q_n) + \lambda(F(p_1, \dots, p_n) - C),$$

trong đó $0 \leq \mu_i \leq 1$ và $\sum_i \mu_i = 1$. Khai triển F , lớp trong bằng

$$\sum_{i=1}^n (q_i + \lambda p_i) \mu_i - \lambda C.$$

Biểu thức này là tuyến tính theo cả μ_i và λ , còn miền xác định của μ_i là tập lồi, nên có thể dùng định lý minimax để đổi đáp án thành

$$\min_{\lambda} \max_{\mu_i} \sum_{i=1}^n (q_i + \lambda p_i) \mu_i - \lambda C.$$

Với λ cố định,

$$\max_{\mu_i} \sum_{i=1}^n (q_i + \lambda p_i) \mu_i = \max_i (q_i + \lambda p_i).$$

Vì vậy đáp án bằng

$$\min_{\lambda} \max_i (q_i + \lambda(p_i - C)).$$

Chỉ cần dựng bao lồi và giải bài toán một chiều này. Giá trị nhỏ nhất được xử lý tương tự.

5. Đối ngẫu quy hoạch tuyến tính

5.1. Dạng chuẩn

Với dạng chuẩn của quy hoạch tuyến tính

$$\begin{aligned} \max_x \quad & c^T x \\ \text{s.t.} \quad & x \geq 0, \\ & Ax \leq b, \end{aligned} \tag{7}$$

ta có thể xem đây là một trường hợp đặc biệt của (5), chỉ có ràng buộc bất đẳng thức. Định nghĩa hàm Lagrange của nó:

$$F(x, y_1, y_2) = c^T x + y_1^T (b - Ax) + y_2^T x.$$

Hàm đối ngẫu Lagrange là

$$\begin{aligned} L(y_1, y_2) &= \max_x F(x, y_1, y_2) \\ &= \max_x c^T x + y_1^T (b - Ax) + y_2^T x \\ &= \max_x (c^T - y_1^T A + y_2^T) x + y_1^T b. \end{aligned}$$

Bài toán đối ngẫu Lagrange $\min_{y_1, y_2 \geq 0} L(y_1, y_2)$ tương đương với

$$\begin{aligned} \min \quad & y_1^T b \\ \text{s.t.} \quad & y_2^T = y_1^T A - c^T, \\ & y_1, y_2 \geq 0. \end{aligned}$$

Khử y_2 khỏi hệ trên, ta được

$$\begin{aligned} \min \quad & y_1^T b \\ \text{s.t.} \quad & y_1^T A \geq c^T, \\ & y_1 \geq 0. \end{aligned}$$

Dạng này hoàn toàn trùng với đối ngẫu quy hoạch tuyến tính. Vì vậy, đối ngẫu quy hoạch tuyến tính là một lớp đặc biệt của đối ngẫu Lagrange.

5.2. Dạng tùy ý

Với một bài toán quy hoạch tuyến tính không ở dạng chuẩn, nếu ta chuyển nó về dạng chuẩn rồi mới lấy đối ngẫu theo cách thông thường, các bước sẽ rườm rà và có thể thêm rất nhiều phần tử dư thừa vào ma trận, gây khó khăn cho việc quan sát và phán đoán hình thức tổ hợp của bài toán. Trong tình huống này, ta có thể thêm nhân tử Lagrange trực tiếp vào bài toán gốc và dùng định lý minimax để lấy đối ngẫu. Cách này giữ lại đáng kể hình thức gọn gàng của bài toán ban đầu, làm cho ý nghĩa tổ hợp của bài toán đối ngẫu rõ ràng hơn.

Bài toán sau minh họa cách lấy đối ngẫu cho một bài toán quy hoạch tuyến tính tùy ý.

5.2.1. [AtCoder World Tour Finals 2022 Day 1] Welcome to Tokyo!.

Mô tả bài toán. Cho m cặp l_i, r_i . Với mỗi $k = 1, 2, \dots, n$, cần tìm: nếu đánh dấu k số trong $1, 2, \dots, n$, tối đa có bao nhiêu chỉ số i sao cho đoạn $[l_i, r_i]$ chứa ít nhất một số được đánh dấu.

Phạm vi dữ liệu.

$$1 \leq n, m \leq 10^6, \quad 1 \leq l_i, r_i \leq n.$$

Phân tích. Xét bài toán với một k cố định. Gọi a_i biểu thị vị trí i có được đánh dấu hay không, và b_i biểu thị đoạn $[l_i, r_i]$ có chứa số được đánh dấu hay không. Khi đó bài toán có thể chuyển thành tìm nghiệm nguyên của quy hoạch tuyến tính sau:

$$\begin{aligned} \max \quad & \sum_{i=1}^m b_i \\ \text{s.t.} \quad & a_i \leq 1, \quad \sum_{i=1}^n a_i = k, \quad b_i \leq 1, \quad b_i \leq \sum_{j=l_i}^{r_i} a_j, \quad 0 \leq a_i, b_i. \end{aligned}$$

Viết lại một chút:

$$\begin{aligned} & \max_{0 \leq a_i, b_i} \sum_{i=1}^m b_i \\ \text{s.t.} \quad & a_i - 1 \leq 0, \quad \sum_{i=1}^n a_i - k = 0, \quad b_i - 1 \leq 0, \quad b_i - \sum_{j=l_i}^{r_i} a_j \leq 0. \end{aligned}$$

Ma trận của bài toán quy hoạch tuyến tính này là ma trận hoàn toàn đơn mô đun; điều đó có nghĩa là luôn tồn tại nghiệm tối ưu trong đó mọi biến đều là số nguyên. Vì vậy có thể bỏ ràng buộc nguyên.

Thêm nhân tử Lagrange cho các ràng buộc, ta được

$$\begin{aligned} & \max_{0 \leq a_i, b_i} \min_{p_i, q_i, s_i \leq 0, \lambda} \left(\sum_{i=1}^m \left(b_i + q_i(b_i - 1) + s_i \left(b_i - \sum_{j=l_i}^{r_i} a_j \right) \right) \right. \\ & \quad \left. + \sum_{i=1}^n p_i(a_i - 1) + \lambda \left(\sum_{i=1}^n a_i - k \right) \right). \end{aligned}$$

Dùng định lý minimax để hoán đổi max và min, rồi sắp xếp lại:

$$\begin{aligned} & \min_{p_i, q_i, s_i \leq 0, \lambda} \max_{0 \leq a_i, b_i} \left(\sum_{i=1}^n \left(p_i + \lambda - \sum_{j|i \in [l_j, r_j]} s_j \right) a_i \right. \\ & \quad \left. + \sum_{i=1}^m (1 + q_i + s_i) b_i - \sum_{i=1}^n p_i - \sum_{i=1}^m q_i - \lambda k \right). \end{aligned}$$

Xem a_i, b_i là nhân tử Lagrange, ta thu được

$$\begin{aligned} \min \quad & - \sum_{i=1}^n p_i - \sum_{i=1}^m q_i - \lambda k \\ \text{s.t.} \quad & p_i + \lambda - \sum_{j|i \in [l_j, r_j]} s_j \leq 0, \\ & 1 + q_i + s_i \leq 0, \\ & p_i, q_i, s_i \leq 0. \end{aligned}$$

Đổi mọi p_i, q_i, s_i, λ thành số đối của biến ban đầu:

$$\begin{aligned} \min \quad & \sum_{i=1}^n p_i + \sum_{i=1}^m q_i + \lambda k \\ \text{s.t.} \quad & p_i + \lambda \geq \sum_{j|i \in [l_j, r_j]} s_j, \\ & q_i + s_i \geq 1, \\ & p_i, q_i, s_i \geq 0. \end{aligned}$$

Chú ý $s_i > 1$ không có ý nghĩa, nên có thể thêm ràng buộc $s_i \leq 1$ và đặt

$$p_i = \max \left(0, \sum_{j|i \in [l_j, r_j]} s_j - \lambda \right), \quad q_i = 1 - s_i.$$

Khi đó được

$$\begin{aligned} \min_{s_i, \lambda} \quad & \sum_{i=1}^n \max \left(0, \sum_{j|i \in [l_j, r_j]} s_j - \lambda \right) - \sum_{i=1}^m s_i + \lambda k + m \\ \text{s.t.} \quad & 0 \leq s_i \leq 1. \end{aligned}$$

Khi $\lambda < 0$, nếu giữ các biến khác không đổi và thay λ bằng 0, đáp án không tệ hơn; vì vậy có thể giả sử $\lambda \geq 0$. Nếu tồn tại một i sao cho

$$\max \left(0, \sum_{j|i \in [l_j, r_j]} s_j - \lambda \right) > 0,$$

ta có thể giảm thích hợp một số s_j để giá trị max này trở thành 0 mà vẫn giữ nguyên đáp án. Do đó tồn tại nghiệm tối ưu sao cho với mọi $i \in [1, n] \cap \mathbb{Z}$,

$$\sum_{j|i \in [l_j, r_j]} s_j - \lambda \leq 0.$$

Bài toán rút gọn thành

$$\begin{aligned} & m + \min_{s_i, \lambda} \lambda k - \sum_{i=1}^m s_i \\ \text{s.t.} \quad & \lambda \geq 0, \quad 0 \leq s_i \leq 1, \quad \sum_{j|i \in [l_j, r_j]} s_j \leq \lambda. \end{aligned}$$

Từ tính nguyên đã nói ở trên, ta chỉ cần xét nghiệm nguyên tối ưu của bài toán này. Kết hợp với

$$\lambda = \max_i \sum_{j|i \in [l_j, r_j]} s_j,$$

bài toán tương đương với chọn t đoạn trong m đoạn để phủ, gọi số lần phủ lớn nhất trên mọi vị trí là λ , rồi tối thiểu hóa $m + k\lambda - t$.

Trọng tâm của bài viết là cách lấy đối ngẫu chứ không phải cách giải bài toán sau đối ngẫu, nên các bước tiếp theo được lược bỏ; có thể tham khảo lời giải chính thức của bài [8]. Các nội dung sâu hơn về cách giải bài toán đối ngẫu quy hoạch tuyến tính có thể xem luận văn đội tuyển quốc gia IOI 2021 của Ding Xiaoman [9].

6. Cân bằng Nash

6.1. Các khái niệm cơ bản trong trò chơi

Định nghĩa 6.1 (chiến lược). Chiến lược của một người tham gia biểu thị toàn bộ hành vi của người đó trong trò chơi, còn gọi là chiến lược thuần. Tập mọi chiến lược có thể của một người tham gia được gọi là tập chiến lược.

Định nghĩa 6.2 (kết quả trò chơi). Chiến lược của tất cả người tham gia tạo thành một kết quả của trò chơi; kết quả này chứa toàn bộ thông tin của lần chơi đó. Tập mọi kết quả có thể được gọi là tập kết quả của trò chơi.

Định nghĩa 6.3 (lợi ích). Với một kết quả trò chơi, mỗi người tham gia có một giá trị thực tương ứng, gọi là lợi ích. Mỗi người đều muốn tối đa hóa lợi ích của mình; hàm ánh xạ từ kết quả trò chơi tới lợi ích của một người được gọi là hàm lợi ích.

Định nghĩa 6.4 (trò chơi tĩnh). Trò chơi tĩnh là trò chơi trong đó các người tham gia ra quyết định đồng thời và chỉ quyết định một lần. Tức là khi quyết định, mỗi người đều không biết những người khác đã chọn hành động nào.

Khái niệm đối lập với trò chơi tĩnh là trò chơi động, trong đó hành động của người tham gia có thứ tự trước sau. Nếu không nói riêng, các trò chơi được mô tả dưới đây đều là trò chơi tĩnh.

Định nghĩa 6.5 (trò chơi thông tin đầy đủ). Trò chơi thông tin đầy đủ là trò chơi trong đó mỗi người tham gia hiểu đầy đủ về những người khác, tức biết không gian chiến lược và hàm lợi ích của họ.

Khái niệm đối lập là trò chơi thông tin không đầy đủ; nếu đó là trò chơi động, người tham gia có thể thu được thông tin về người khác trong quá trình chơi. Nếu không nói riêng, các trò chơi dưới đây đều là trò chơi thông tin đầy đủ.

Định nghĩa 6.6 (trò chơi phi hợp tác). Nếu giữa các người tham gia không tồn tại thỏa thuận nào có tính ràng buộc, và quyết định của mỗi người chỉ dựa trên việc tối đa hóa lợi ích của chính mình, thì trò chơi đó được gọi là trò chơi phi hợp tác.

Khái niệm đối lập là trò chơi hợp tác, trong đó các người tham gia có thể hợp tác với nhau. Nếu không nói riêng, các trò chơi dưới đây đều là trò chơi phi hợp tác.

Định nghĩa 6.7 (chiến lược hỗn hợp). Khi hành động, một người tham gia có thể gán cho mỗi chiến lược thuần một xác suất và bảo đảm tổng xác suất bằng 1. Cách hành động này được gọi là chiến lược hỗn hợp.

Bài viết chỉ xét trò chơi phi hợp tác tĩnh, thông tin đầy đủ, có đúng hai người tham gia, và tập chiến lược của mỗi người là hữu hạn. Khi đó trò chơi có thể đơn giản hóa thành mô hình sau.

Giả sử kích thước tập chiến lược của hai người lần lượt là n và m . Hàm lợi ích của hai bên có thể biểu diễn bằng hai ma trận thực $n \times m$ là A, B , trong đó A_{ij} và B_{ij} lần lượt biểu thị lợi ích của người thứ nhất và người thứ hai khi người thứ nhất chọn chiến lược i , còn người thứ hai chọn chiến lược j .

Chiến lược hỗn hợp của hai người có thể biểu diễn bằng vectơ cột thực a độ dài n và vectơ cột thực b độ dài m , thỏa

$$0 \leq a_i, b_i \leq 1, \quad \sum_{i=1}^n a_i = 1, \quad \sum_{i=1}^m b_i = 1.$$

Lợi ích kỳ vọng của người thứ nhất là $a^T A b$, còn lợi ích kỳ vọng của người thứ hai là $a^T B b$.

Định nghĩa 6.8 (cân bằng Nash). Nếu mỗi người tham gia đã chọn chiến lược hỗn hợp của mình, và không người nào có thể tăng lợi ích kỳ vọng bằng cách chỉ điều chỉnh chiến lược hỗn hợp của chính mình, thì tập chiến lược hỗn hợp hiện tại cùng kết quả trò chơi tương ứng được gọi là cân bằng Nash, hoặc điểm cân bằng Nash.

Ta có thể hiểu cân bằng Nash rõ hơn qua vài trò chơi kinh điển.

Ví dụ 6.1 (song đề tù nhân). Trong một nhà tù có hai nghi phạm. Mỗi người có hai lựa chọn: nhận tội và tố giác người kia, hoặc giữ im lặng. Nếu cả hai cùng tố giác, mỗi người bị kết án 8 năm. Nếu một người tố giác còn người kia im lặng, người tố giác được thả ngay, còn người im lặng bị kết án 10 năm. Nếu cả hai cùng im lặng, mỗi người bị kết án 2 năm. Hai tù nhân không thể trao đổi, và đều muốn tối thiểu hóa án tù của mình.

Dùng số đối của thời hạn tù làm lợi ích, ta có

$$n = m = 2, \quad A = \begin{pmatrix} -8 & 0 \\ -10 & -2 \end{pmatrix}, \quad B = \begin{pmatrix} -8 & -10 \\ 0 & -2 \end{pmatrix}.$$

Với cả hai bên, chọn tố giác luôn nghiêm ngặt tốt hơn chọn im lặng trong mọi tình huống. Vì vậy tồn tại duy nhất một điểm cân bằng Nash: cả hai cùng tố giác, tức

$$a = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Ví dụ 6.2 (trò chơi giới tính). Một đôi vợ chồng: chồng muốn xem bóng đá, vợ muốn đi mua sắm, nhưng cả hai đều muốn ở cùng nhau hơn là tách ra làm việc riêng. Cụ thể, mỗi người chọn xem bóng đá hoặc đi mua sắm. Nếu hai người chọn khác nhau, cả hai nhận mức hài lòng 0. Nếu chọn giống nhau, người có sở thích tương ứng nhận 3, người kia nhận 2.

Theo mô tả,

$$n = m = 2, \quad A = \begin{pmatrix} 3 & 0 \\ 0 & 2 \end{pmatrix}, \quad B = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}.$$

Trò chơi này có ba điểm cân bằng Nash:

$$a = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 0 \end{pmatrix},$$

khi đó lợi ích hai người là 3, 2;

$$a = \begin{pmatrix} 3/5 \\ 2/5 \end{pmatrix}, \quad b = \begin{pmatrix} 2/5 \\ 3/5 \end{pmatrix},$$

khi đó lợi ích kỳ vọng của hai người đều là 6/5; và

$$a = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad b = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

khi đó lợi ích hai người là 2, 3. Chú ý điểm cân bằng Nash thứ hai công bằng nhất cho hai bên, nhưng lợi ích kỳ vọng của nó lại nghiêm ngặt thấp hơn hai điểm còn lại.

6.2. Cân bằng Nash của trò chơi hai người tổng bằng không

Định nghĩa 6.9 (trò chơi tổng bằng không). Nếu với mọi kết quả trò chơi, tổng lợi ích của tất cả người tham gia đều bằng 0, thì trò chơi đó được gọi là trò chơi tổng bằng không.

Với trò chơi hai người tổng bằng không, nếu người thứ nhất nhận lợi ích x thì người thứ hai nhất định nhận lợi ích $-x$. Vì vậy chỉ cần một ma trận A để mô tả hàm lợi ích. Với chiến lược hỗn hợp a, b của hai bên, lợi ích kỳ vọng của người thứ nhất là $a^T A b$, còn của người thứ hai là $-a^T A b$.

Định lý 6.1. Với mọi trò chơi hai người tổng bằng không, không thể tồn tại hai điểm cân bằng Nash sao cho lợi ích kỳ vọng của người tham gia tại hai điểm đó khác nhau.

Chứng minh. Chứng minh phản chứng. Giả sử chiến lược hỗn hợp của hai bên tại điểm cân bằng Nash thứ nhất là a_1, b_1 , tại điểm thứ hai là a_2, b_2 , và người thứ nhất có lợi ích kỳ vọng cao hơn tại điểm thứ nhất:

$$a_1^T A b_1 > a_2^T A b_2.$$

Theo định nghĩa cân bằng Nash, không người chơi nào có thể tăng lợi ích kỳ vọng bằng cách thay đổi chiến lược của chính mình. Vì vậy

$$-a_1^T A b_1 \geq -a_1^T A b_2, \quad a_2^T A b_2 \geq a_1^T A b_2.$$

Suy ra

$$a_2^T A b_2 \geq a_1^T A b_2 \geq a_1^T A b_1,$$

mâu thuẫn với $a_1^T A b_1 > a_2^T A b_2$. Do đó một trò chơi hai người tổng bằng không không thể có hai điểm cân bằng Nash với lợi ích kỳ vọng khác nhau.

Định lý 6.2. Trò chơi hai người tổng bằng không luôn tồn tại điểm cân bằng Nash.

Chứng minh. Giả sử số chiến lược của hai người lần lượt là n, m , và ma trận lợi ích của người thứ nhất là A . Gọi D_a là tập mọi chiến lược hỗn hợp của người thứ nhất, D_b là tập mọi chiến lược hỗn hợp của người thứ hai. Khi đó D_a, D_b đều là các tập lồi compact. Định nghĩa

$$F : D_a \times D_b \rightarrow \mathbb{R}, \quad F(a, b) = a^T A b.$$

Theo bổ đề 4.1, tồn tại $a_0 \in D_a, b_0 \in D_b$ sao cho

$$\max_{a \in D_a} F(a, b_0) = F(a_0, b_0) = \min_{b \in D_b} F(a_0, b).$$

Vì vậy chiến lược hỗn hợp a_0, b_0 của hai bên tạo thành một cân bằng Nash.

Từ thảo luận trên, mọi trò chơi hai người tổng bằng không đều tồn tại cân bằng Nash, và mọi cân bằng Nash có cùng lợi ích kỳ vọng. Vì vậy trong ứng dụng thực tế, người ta thường xem cân bằng Nash là lời giải của trò chơi hai người tổng bằng không. Với các bài toán trò chơi hai người tổng bằng không trong OI, câu “hai bên đều dùng chiến lược tối ưu” chính là chỉ các chiến lược hỗn hợp đạt cân bằng Nash.

Hệ quả 6.1. Với một trò chơi hai người tổng bằng không, gọi D_a là tập chiến lược hỗn hợp của người thứ nhất, D_b là tập chiến lược hỗn hợp của người thứ hai, A là ma trận lợi ích của người thứ nhất, và S là lợi ích kỳ vọng của người thứ nhất tại cân bằng Nash. Khi đó

$$S = \min_{b \in D_b} \max_{a \in D_a} a^T A b = \max_{a \in D_a} \min_{b \in D_b} a^T A b.$$

Chứng minh. Giả sử chiến lược hỗn hợp tại cân bằng Nash của hai bên là a_0, b_0 . Từ định nghĩa,

$$\max_{a \in D_a} a^T A b_0 = S = \min_{b \in D_b} a_0^T A b.$$

Do đó

$$\begin{aligned} \min_{b \in D_b} \max_{a \in D_a} a^T A b &\leq \max_{a \in D_a} a^T A b_0 \\ &= S \\ &= \min_{b \in D_b} a_0^T A b \\ &\leq \max_{a \in D_a} \min_{b \in D_b} a^T A b. \end{aligned}$$

Theo định lý minimax,

$$\min_{b \in D_b} \max_{a \in D_a} a^T A b = \max_{a \in D_a} \min_{b \in D_b} a^T A b,$$

nên kết luận đúng.

Hệ quả này rất quan trọng trong OI: cân bằng Nash, vốn được định nghĩa bằng điểm cân bằng, nay được chuyển thành một công thức rõ ràng. Biểu thức

$$\min_{b \in D_b} \max_{a \in D_a} a^T A b$$

có thể hiểu là người thứ nhất đưa ra một chiến lược hỗn hợp trước, rồi người thứ hai, sau khi biết chiến lược hỗn hợp của người thứ nhất, mới chọn chiến lược hỗn hợp của mình; hai bên đều hành động tối ưu. Khi đó hình thức trò chơi chuyển từ tĩnh sang động, giống phần lớn trò chơi trong OI hơn, nên thuận tiện cho thí sinh phân tích.

Ngoài ra, hệ quả này còn cung cấp một cách chuyển bài toán cân bằng Nash của trò chơi hai người tổng bằng không thành bài toán quy hoạch tuyến tính. Lợi ích kỳ vọng của người thứ nhất tại điểm cân

bằng Nash có thể biểu diễn bằng quy hoạch tuyến tính:

$$\begin{aligned} \max \quad & t \\ \text{s.t.} \quad & x_i \geq 0, \quad \sum_{i=1}^n x_i = 1, \\ & \forall j, \quad \sum_{i=1}^n x_i A_{ij} \geq t. \end{aligned}$$

6.3. Ví dụ

6.3.1. [THUPC 2023 vòng sơ loại] Trò chơi lừa dối.

Mô tả bài toán. Hai người chơi một trò chơi tĩnh tổng bằng không. Mỗi bên có $n + 1$ chiến lược. Hàm lợi ích của người thứ nhất được biểu diễn bằng ma trận $(n + 1) \times (n + 1)$ là A , thỏa

$$A_{ij} = \begin{cases} \frac{j-1}{2}, & i < j, \\ 0, & i = j, \\ i-1, & i > j. \end{cases}$$

Cần tìm chiến lược hỗn hợp tối ưu của hai bên, lấy modulo 998244353.

Phạm vi dữ liệu.

$$1 \leq n \leq 4 \times 10^5.$$

Phân tích. Theo hệ quả 6.1, xem trò chơi này như việc hai bên lần lượt đưa ra chiến lược hỗn hợp. Khi người thứ nhất đưa ra chiến lược hỗn hợp a , lợi ích kỳ vọng của người thứ hai nếu chọn từng chiến lược có thể biểu diễn bằng một vectơ hàng $n + 1$ chiều c , thỏa

$$c = a^T A = \sum_{i=1}^{n+1} a_i A_i.$$

Khi đó chiến lược tối ưu của người thứ hai là chọn phần tử nhỏ nhất trong c . Vì vậy mục tiêu của người thứ nhất là tối đa hóa giá trị nhỏ nhất trong c .

Tính chất. Nếu tồn tại chiến lược hỗn hợp a của người thứ nhất sao cho mọi phần tử trong c bằng nhau, thì chiến lược đó là chiến lược tối ưu của người thứ nhất.

Chứng minh. Chứng minh phản chứng. Giả sử người thứ nhất có chiến lược hỗn hợp tốt hơn a' . Gọi $c' = \sum_{i=1}^{n+1} a'_i A_i$; khi đó $c' > c$, tức

$$\sum_{i=1}^{n+1} (a'_i - a_i) A_i > 0.$$

Đặt

$$t = \sum_{i=1}^{n+1} (a'_i - a_i) A_i > 0.$$

Tìm chỉ số lớn nhất i thỏa $a'_i - a_i \geq 0$. Khi đó

$$\begin{aligned}
 t_i &= \sum_{j=1}^{i-1} \frac{i-1}{2} (a'_j - a_j) + \sum_{j=i+1}^{n+1} (j-1) (a'_j - a_j) \\
 &\leq \frac{i-1}{2} \sum_{j=1}^{i-1} (a'_j - a_j) + \frac{i-1}{2} (a'_i - a_i) + i \sum_{j=i+1}^{n+1} (a'_j - a_j) \\
 &= -\frac{i-1}{2} \sum_{j=i+1}^{n+1} (a'_j - a_j) + i \sum_{j=i+1}^{n+1} (a'_j - a_j) \\
 &= \frac{i+1}{2} \sum_{j=i+1}^{n+1} (a'_j - a_j) \\
 &\leq 0.
 \end{aligned}$$

Điều này mâu thuẫn với $t > 0$. Với người thứ hai, cũng có thể dùng phương pháp tương tự để chứng minh kết luận tương ứng.

Vì vậy xét trường hợp mọi phần tử của c đều bằng nhau. Ta có thể lập hệ

$$\begin{aligned}
 \frac{1}{2}a_2 + \frac{2}{2}a_3 + \frac{3}{2}a_4 + \dots &= a_1 + \frac{2}{2}a_3 + \frac{3}{2}a_4 + \dots \\
 &= 2a_1 + 2a_2 + \frac{3}{2}a_4 + \dots \\
 &= \dots.
 \end{aligned}$$

Kết hợp với $\sum_{i=1}^{n+1} a_i = 1$, lấy hiệu các phương trình kế nhau rồi truy hồi là tính được a . Dễ thấy $a_i \geq 0$ với mọi i , nên a là chiến lược hỗn hợp tối ưu của người thứ nhất. Chiến lược hỗn hợp tối ưu của người thứ hai cũng có thể tính bằng phương pháp tương tự.

7. Tổng kết

Bài viết xoay quanh nhân tử Lagrange, giới thiệu kỹ thuật xử lý nhân tử Lagrange trong bài toán tối ưu và ứng dụng của phương pháp nhân tử Lagrange trong OI. Xoay quanh đối ngẫu Lagrange, bài viết giới thiệu phương pháp tính nhanh bài toán đối ngẫu quy hoạch tuyến tính, định lý minimax, cùng một số tính chất của cân bằng Nash trong trò chơi hai người tổng bằng không.

Nhân tử Lagrange và đối ngẫu còn có nhiều tính chất, kết luận thú vị. Do kiến thức của tác giả còn hạn chế, nội dung được giới thiệu tương đối đơn giản. Phần này chưa phổ biến nhiều trong OI, và số bài liên quan cũng chưa nhiều. Tác giả hy vọng bài viết có thể gợi mở để nhiều người biết tới phương pháp xử lý nhân tử Lagrange và định lý minimax, đồng thời mong chờ thêm nhiều bài toán liên quan và mở rộng thú vị xuất hiện.

8. Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn huấn luyện viên đội tuyển quốc gia Yang Yaoliang đã hướng dẫn.

Cảm ơn cha mẹ đã nuôi dưỡng và giáo dục tác giả.

Cảm ơn nhà trường đã bồi dưỡng; cảm ơn các thầy Fu Shuibo, Ying Ping'an, Yu Ting, Lin Naishu, Dong Er đã dạy dỗ, và cảm ơn các bạn học đã giúp đỡ.

Cảm ơn các bạn Wang Zhifang và Tian Junfeng đã trao đổi, thảo luận và gợi ý cho tác giả.

Cảm ơn các bạn Wang Zhifang, Fang Junzhe và Liu Hengniu đã đọc duyệt bài viết.

Cảm ơn những thầy cô và bạn bè khác đã giúp đỡ tác giả.

Tài liệu tham khảo

1. *Lagrange Multipliers and The Implicit Function Theorem*. Dartmouth [math_14_lagrange.pdf](#).
2. Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press.
3. Wikipedia. *Minimax theorem*. https://en.wikipedia.org/wiki/Minimax_theorem.
4. Younggeun Yoo. *Kakutani's Fixed Point Theorem and The Minimax Theorem in Game Theory*. <https://math.uchicago.edu/~may/REU2016/REUPapers/Yoo.pdf>.
5. Wikipedia. *Unimodular matrix*. https://en.wikipedia.org/wiki/Unimodular_matrix.
6. Wikipedia. *Karush–Kuhn–Tucker conditions*. Wikipedia KKT conditions.
7. Jian Li. *Selected Topics in Optimization*. <https://people.iis.tsinghua.edu.cn/~jianli/courses/ATCS2014spring/notel.pdf>.
8. AtCoder. *Editorial of D - Welcome to Tokyo!*. AtCoder editorial 7106.
9. Ding Xiaoman. *Bàn lại về ứng dụng của đối ngẫu quy hoạch tuyến tính trong thi tin học*. Tập luận văn đội tuyển quốc gia IOI, 2021.

Bài 5. Thuật toán liên quan tới bài toán cập nhật đoạn và truy vấn đoạn cùng ứng dụng

Shen Jiyu, Trường Văn Uyên thuộc Tập đoàn Giáo dục Trung học Học Quân, Hàng Châu

Tóm tắt

Bài viết tổng kết bài toán cập nhật và truy vấn phạm vi, giới thiệu một thuật toán cập nhật–truy vấn phạm vi offline tối ưu về số phép toán, cùng một số thuật toán và ứng dụng khác cho các bài toán liên quan.

1. Lời nói đầu

Các cấu trúc dữ liệu trên dãy, như cây đoạn, là một lớp thuật toán gọn gàng, có thể duy trì nhiều bài toán cập nhật và truy vấn trên dãy, nên được dùng rộng rãi trong OI.

Tuy vậy, khi mở rộng bài toán cập nhật–truy vấn từ một chiều lên số chiều cao hơn, cấu trúc dữ liệu trên dãy, chẳng hạn cây đoạn, không còn thích ứng tốt với cấu trúc phức tạp nữa; so với dãy, độ khó tăng lên đáng kể. Với lớp bài toán này, ta cần tìm các hướng giải quyết tốt hơn và tổng quát hơn.

Vì vậy, tác giả muốn dùng bài viết này để giới thiệu một thuật toán cập nhật–truy vấn phạm vi tối ưu về số phép toán, cũng như một số cách xử lý khác cho bài toán cập nhật–truy vấn trong không gian nhiều chiều.

2. Dẫn nhập

2.1. Một bài toán kinh điển

Ta bắt đầu từ một bài toán kinh điển.

Ví dụ 2.1. Duy trì một dãy, trong đó mỗi phần tử là một ma trận 2×2 , và cần thực hiện các thao tác sau:

- cập nhật: cho l, r, v , nhân mọi ma trận trong đoạn $[l, r]$ với ma trận v ;
- truy vấn: cho l, r , hỏi tổng các ma trận trong đoạn $[l, r]$.

Đây là bài toán kinh điển của cây đoạn, có thể giải bằng cây đoạn với lazy tag. Với mỗi nút trên cây đoạn, ta duy trì tổng ma trận của đoạn tương ứng và một lazy tag biểu diễn phép nhân ma trận áp dụng lên đoạn.

Bây giờ xét một bài toán cấu trúc dữ liệu trừu tượng hơn: bỏ qua loại thông tin cụ thể cần duy trì, và chỉ xét dạng bài toán tổng quát khi thông tin được duy trì thỏa một số điều kiện nhất định.

Với những bài toán cây đoạn duy trì được, dạng trừu tượng là:

Duy trì một dãy. Ta có thể dùng một loại dữ liệu để biểu diễn thông tin của một đoạn trên dãy, và dùng một loại dữ liệu để biểu diễn một phép cập nhật trên một đoạn. Chúng cần thỏa các tính chất sau:

1. Thông tin của hai đoạn $[l, mid]$ và $[mid + 1, r]$ có thể gộp trong $O(1)$ để thu được thông tin của $[l, r]$.
2. Hai phép cập nhật liên tiếp trên cùng một đoạn có thể gộp trong $O(1)$ thành một phép cập nhật.
3. Nếu biết thông tin của một đoạn, có thể tính trong $O(1)$ thông tin của đoạn đó sau một phép cập nhật.

Phép nhân ma trận thỏa tính kết hợp $(A \times B) \times C = A \times (B \times C)$ và tính phân phối $A \times (B + C) = A \times B + A \times C$, nhưng không thỏa tính giao hoán. Trong bài toán trên, phép gộp thông tin là phép cộng ma trận; còn tính kết hợp và tính phân phối lần lượt làm cho bài toán thỏa các tính chất 2 và 3.

2.2. Định nghĩa thông tin trừu tượng

Định nghĩa 2.1. Một nửa nhóm gồm một tập D và một phép toán hai ngôi

$$D \times D \rightarrow D,$$

thỏa tính kết hợp:

$$\forall a, b, c \in D, \quad (a * b) * c = a * (b * c).$$

Định nghĩa 2.2. Một monoid gồm một tập D , một phép toán hai ngôi $D \times D \rightarrow D$, thỏa tính kết hợp

$$\forall a, b, c \in D, \quad (a * b) * c = a * (b * c),$$

và có phần tử đơn vị:

$$\forall a \in D, \quad a * \epsilon = \epsilon * a = a.$$

Định nghĩa 2.3. Một nửa nhóm giao hoán gồm một tập D , một phép toán hai ngôi $D \times D \rightarrow D$, thỏa tính kết hợp và tính giao hoán:

$$\forall a, b, c \in D, \quad (a * b) * c = a * (b * c),$$

$$\forall a, b \in D, \quad a * b = b * a.$$

Thông tin được duy trì trong ví dụ phía trước có thể xem là thông tin nửa nhóm. Có thể thấy ta thật ra đang duy trì hai thông tin nửa nhóm: một loại biểu diễn thông tin của đoạn, một loại biểu diễn lazy tag cập nhật. Không khó nhận ra rằng chỉ cần cả thông tin đoạn và thông tin cập nhật đều là thông tin nửa nhóm thì có thể dùng cây đoạn để duy trì.

3. Cập nhật–truy vấn phạm vi offline

3.1. Phạm vi rộng hơn

Trong bài toán, phạm vi cập nhật và truy vấn không nhất thiết là một đoạn trên dãy, mà có thể là đường đi đơn trên cây, hình chữ nhật trong mặt phẳng hai chiều, phạm vi trục giao nhiều chiều, nửa mặt phẳng, phạm vi cong, v.v.

Ban đầu có n điểm. Một phạm vi biểu diễn một tập con của tập điểm ban đầu. Ta dùng một thông tin để biểu diễn tổng thông tin của các điểm trong phạm vi, và dùng một thông tin khác để biểu diễn một phép cập nhật trên phạm vi. Hai loại thông tin cần thỏa:

1. Thông tin S, T của hai phạm vi có thể gộp trong $O(1)$ thành $S + T$.
2. Hai phép cập nhật liên tiếp trên một phạm vi có thể gộp trong $O(1)$ thành một phép cập nhật.
3. Nếu biết thông tin của một phạm vi, có thể tính trong $O(1)$ thông tin của phạm vi đó sau một phép cập nhật.

3.2. Dạng bài toán

Ban đầu cho một tập gồm n điểm, mỗi điểm có một trọng số ban đầu. Mỗi phép cập nhật cho một phạm vi và một giá trị cập nhật, biểu thị việc lấy trọng số của các điểm trong phạm vi thực hiện một phép toán hai ngôi với trọng số cập nhật. Mỗi truy vấn cho một phạm vi và hỏi tổng trọng số của các điểm trong phạm vi. Trọng số của điểm và trọng số cập nhật thỏa tính chất “hai nửa nhóm”.

Dạng hình thức như sau. Cho nửa nhóm giao hoán $(D, +)$, nửa nhóm $(M, *)$, và một phép tác động hai ngôi

$$* : D \times M \rightarrow D.$$

Phép tác động này thỏa:

$$\forall a \in D, b, c \in M, \quad (a * b) * c = a * (b * c),$$

$$\forall a, b \in D, c \in M, \quad (a + b) * c = a * c + b * c.$$

Cho một tập ban đầu I_0 , một tập hữu hạn $I \subseteq I_0$, và một tập truy vấn Q . Mỗi điểm trong I có trọng số ban đầu $d_0(x) \in D$. Thao tác thứ t được định nghĩa như sau:

1. Cập nhật phạm vi: cho phạm vi $q_t \in Q$ và $f_t \in M$. Nếu $x \in q_t$ thì $d_t(x) = d_{t-1}(x) * f_t$, ngược lại $d_t(x) = d_{t-1}(x)$.
2. Truy vấn phạm vi: cho phạm vi $q_t \in Q$. Đặt $d_t(x) = d_{t-1}(x)$; đáp án là

$$\sum_{x \in q_t} d_t(x).$$

Ký hiệu $n = |I|$ là số trọng số ban đầu, và m là số thao tác cập nhật, truy vấn.

3.3. Thuật toán cập nhật–truy vấn phạm vi offline tối ưu

Sau đây là một thuật toán cập nhật–truy vấn phạm vi offline do Li Xinlong và Cai Chengze từ Đại học Thanh Hoa đề xuất.³ Thuật toán này cài đặt khá đơn giản, hằng số nhỏ, và có thể chứng minh là tối ưu về số phép toán.

³Xem [2], https://qoj.ac/files/ISAAC_2021_paper_100.pdf.

Định nghĩa 3.1. Định nghĩa quan hệ tương đương như sau. Giả sử có dãy thao tác $q_1, q_2, \dots, q_k$. Với hai điểm x_1, x_2 , nếu

$$\forall i \in [1, k], \quad x_1 \in q_i \iff x_2 \in q_i,$$

trong đó q_i là phạm vi của thao tác thứ i , thì gọi x_1 và x_2 là tương đương.

Ta gom các điểm tương đương vào cùng một lớp tương đương, từ đó thu được một số lớp tương đương. Giả sử xét các thao tác $q_l, q_{l+1}, \dots, q_r$, và tập các lớp tương đương sinh ra là $R_{l,r}$. Nếu $[l', r'] \subseteq [l, r]$, thì $R_{l,r}$ chia tập điểm mịn hơn $R_{l',r'}$; ký hiệu quan hệ này là $R_{l,r} \subseteq R_{l',r'}$.

Hình 1 trong bản gốc minh họa các điểm bị một số đường cong chia thành các lớp tương đương khác nhau: hai điểm thuộc cùng lớp khi chúng nằm trong cùng phía đối với mọi phạm vi đang xét.

Vì các thao tác cập nhật đã biết trước, ta thiết kế thuật toán chia để trị trên dãy thao tác cập nhật. Trước hết, với dãy thao tác hiện tại, ta có thể chia ra một số lớp tương đương. Càng có nhiều thao tác thì lớp tương đương càng mịn; phân hoạch mịn nhất có n lớp tương đương, với n là tổng số điểm.

Ta muốn xây dựng một cấu trúc dữ liệu dạng cây thỏa:

- nút lá lưu thông tin của một điểm đơn cần duy trì;
- nút không phải lá là nút phụ trợ để thuận tiện cập nhật và truy vấn, trên đó lưu thông tin của cả cây con tương ứng và tag cần đẩy xuống cây con;
- trước khi truy cập một nút lá, phải truy cập mọi tổ tiên không phải lá của nó.

Không khó thấy rằng cây đoạn trên dãy, Top Tree trên cây, và nhiều thuật toán kinh điển khác đều thỏa cấu trúc này. Trong bài toán hiện tại, ta xét việc duy trì một rừng có gốc động như sau:

- mỗi nút tương ứng với một lớp tương đương của một đoạn thao tác nào đó;
- mỗi nút lá lưu thông tin của một lớp tương đương kích thước 1, tức thông tin riêng của n điểm ban đầu.

Giả sử tập lớp tương đương của dãy thao tác hiện tại là R_1 . Khi đó mỗi gốc của rừng có gốc hiện tại tương ứng với một lớp tương đương trong R_1 . Giả sử ta muốn chuyển tập lớp tương đương hiện tại từ R_1 sang R_2 , và thỏa $R_1 \subseteq R_2$, tức R_2 có ít lớp tương đương hơn.

Nếu thực hiện một số phép gộp trên rừng có gốc hiện tại, mỗi lần gộp vài cây có gốc, ta có thể chuyển trạng thái R_1 tới trạng thái đích R_2 . Cách gộp các cây có gốc $x_1, x_2, \dots, x_k$ là tạo một nút mới y làm cha của $x_1, x_2, \dots, x_k$. Tương tự, ta có thể hủy một số phép gộp trong trạng thái R_2 , tức xóa các nút gốc, để quay về trạng thái R_1 . Gọi hai loại thao tác này là di chuyển giữa các rừng có gốc.

Để hỗ trợ cập nhật và truy vấn, xét việc duy trì thêm trường trên các nút không phải lá. Cần ghi hai giá trị phụ: một lazy tag cập nhật $u(x)$, và tổng cây con $d(x)$, tức tổng của các phần tử trong lớp tương đương. Khi thêm nút vào rừng có gốc, ta tạo nút mới y làm cha của $x_1, x_2, \dots, x_k$, rồi đặt

$$u(y) = \epsilon_M, \quad d(y) = \sum_{i=1}^k d(x_i),$$

như vậy là đã kéo thông tin lên. Khi xóa nút khỏi rừng có gốc, ta xóa nút y và tách ra k cây có gốc $x_1, x_2, \dots, x_k$; khi đó cần đẩy lazy tag của y xuống rồi xóa y .

Nếu có một cây gốc x mà các nút lá trong cây này đúng bằng những điểm nằm trong một lần cập nhật hoặc truy vấn, thì $d(x)$ chính là đáp án của truy vấn. Với cập nhật, ta gắn tag lên x :

$$(d(x), u(x)) \leftarrow (d(x) * U, u(x) * U).$$

Với bài toán gốc, xét chia để trị. Chia dãy thao tác hiện tại thành hai nửa. Vì mỗi nửa có ít thao tác hơn, số lớp tương đương của nó cũng ít hơn; điều này nghĩa là hai đoạn sau khi chia để trị chỉ cần xử lý ít thông tin hơn.

Gọi R_0 là trạng thái rừng có gốc gồm n lá và chưa thực hiện phép gộp nào. Giả sử dãy thao tác đang cần xử lý là $q_1, \dots, q_r$. Trước hết ta chuyển quan hệ tương đương từ R_0 sang $R_{l,r}$. Giả sử khi đó mọi điểm đã có giá trị sau $l-1$ thao tác đầu, tức $d_{l-1}(x)$. Sau đó chạy thuật toán chia để trị trên $[l, r]$, thu được giá trị $d_r(x)$ sau r thao tác, rồi quay lui và xử lý giai đoạn tiếp theo.

Khi cần chia để trị trên $q_l, \dots, q_r$, đặt $mid = \lfloor (l+r)/2 \rfloor$. Trước tiên chuyển tập lớp tương đương từ $R_{l,r}$ sang trạng thái của $q_l, \dots, q_{mid}$, tức $R_{l,mid}$; việc này sẽ thêm một số nút vào rừng có gốc. Sau khi xử lý xong $q_l, \dots, q_{mid}$, quay về $R_{l,r}$, rồi chuyển sang $R_{mid+1,r}$, chạy chia để trị trên $q_{mid+1}, \dots, q_r$, sau đó lại quay về $R_{l,r}$.

Khi đệ quy đến $l = r$, chỉ còn một thao tác, nên chỉ có hai lớp tương đương. Cập nhật thì gắn tag lên gốc của cây tương ứng với lớp tương đương cần thao tác; truy vấn thì lấy thông tin d của gốc đó.

Hình 2 trong bản gốc minh họa cấu trúc rừng có gốc khi quan hệ tương đương lần lượt đi qua bốn trạng thái $R_{1,4}$, $R_{1,2}$, $R_{3,4}$ và $R_{1,1}$. Khi chuyển từ $R_{1,4}$ sang $R_{1,2}$, các lớp $\{A\}$, $\{B\}$, $\{F\}$ được gộp thành $\{A, B, F\}$; $\{C\}$, $\{D, E\}$ được gộp thành $\{C, D, E\}$; và $\{G\}$, $\{J, K\}$ được gộp thành $\{G, J, K\}$. Khi quay từ $R_{1,2}$ về $R_{1,4}$, ta hủy các phép gộp đó. Khi đệ quy tới $R_{1,1}$, cây cần cập nhật là cây a ; nếu x là gốc của cây a , tương ứng lớp $\{A, B, C, D, E, F\}$, thì ta gắn tag cập nhật và truy vấn giá trị của x .

3.4. Độ phức tạp của thuật toán

Định nghĩa hàm rời rạc F , trong đó $F(x)$ là số lớp tương đương lớn nhất mà x thao tác có thể chia tập điểm thành. Không khó thấy $F(x)$ đơn điệu tăng, và x thao tác có thể chia tập I thành nhiều nhất $\min(F(x), |I|)$ lớp tương đương.

Định nghĩa hàm rời rạc G , trong đó $G(n)$ thỏa

$$F(G(n)) \leq n, \quad F(G(n) + 1) > n.$$

Nói cách khác, với phạm vi đã cho, $G(n)$ biểu diễn ít nhất cần bao nhiêu thao tác để chia n điểm thành quan hệ tương đương mịn nhất, tức n lớp tương đương.

Giả sử có n điểm và m thao tác. Chia m thao tác thành các nhóm, mỗi nhóm có $G(n)$ thao tác, rồi dùng thuật toán chia để trị trên từng nhóm. Số phép toán trên cấu trúc đại số là

$$T(n, m) = O(n) + \frac{m}{G(n)} T_1(G(n)), \quad T_1(n) = 2T_1(n/2) + O(F(n)).$$

Trong bài toán trên đây, tức thao tác cập nhật–truy vấn đoạn, có $F(n) = O(n)$, do đó $T(n, m) = O(n + m \log \min(n, m))$. Với cập nhật–truy vấn đường đi trên cây cũng có $F(n) = O(n)$, nên $T(n, m) = O(n + m \log \min(n, m))$.

Nếu phạm vi cập nhật–truy vấn là nửa mặt phẳng, đa giác, hình tròn, ellipse, hyperbola, v.v., thì $F(n) = O(n^2)$, suy ra

$$T(n, m) = O(n + m\sqrt{n}).$$

Nếu phạm vi cập nhật–truy vấn là phạm vi trong không gian d chiều, chẳng hạn nửa không gian d chiều, simplex, hình cầu, phạm vi trục giao, đa diện, v.v., với $d > 1$, thì $F(n) = O(n^d)$, suy ra

$$T(n, m) = O(n + mn^{1-1/d}).$$

Đặc biệt, nếu phạm vi cập nhật–truy vấn là tập con tùy ý của tập điểm, thì $F(n) = O(2^n)$, khi đó $G(n) = O(\log n)$, và

$$T(n, m) = O\left(n + \frac{mn}{\log n}\right).$$

Điều này nghĩa là ngay cả khi phạm vi là tập con tùy ý, ta vẫn có thể đạt độ phức tạp tốt hơn vết cạn.

Với thuật toán trên, số phép toán trên cấu trúc đại số chỉ phụ thuộc vào n, m, I_0, Q, F , không phụ thuộc vào hình dạng phạm vi cụ thể hay cấu trúc đại số cụ thể. Trong tài liệu [2], với trường hợp $F(x) = O(x^c)$, người ta chứng minh cận dưới số phép toán trên cấu trúc đại số là

$$\Omega(n + mn^{1-1/c}),$$

tức số phép toán của thuật toán này là tối ưu.

Cần chú ý rằng đây là thuật toán tối ưu về số phép toán, nhưng chưa tính độ phức tạp của phần hình học tính toán, như định vị điểm trong mặt phẳng hoặc không gian.

3.5. Tìm vùng

Với bài toán cập nhật–truy vấn trong mặt phẳng, ta chia thao tác thành các giai đoạn kích thước $O(\sqrt{n})$. Mỗi giai đoạn có $O(n)$ vùng; dùng thuật toán định vị điểm trên đồ thị phẳng là có thể hoàn thành việc di chuyển giữa các lớp tương đương. Thuật toán này có thể xử lý cập nhật–truy vấn phạm vi là hình tròn, hyperbola, ellipse, v.v. với cùng độ phức tạp.

Có một phương pháp băm tổng quát: nếu cần chuyển tới $R_{l,r}$, ta gán ngẫu nhiên cho mỗi phạm vi một số nguyên trong $[0, 2^w)$, rồi thực hiện cộng phạm vi đối với các phạm vi $q_l, q_{l+1}, \dots, q_r$. Khi đó các lớp tương đương khác nhau có xác suất cao nhận giá trị băm khác nhau, còn cùng một lớp tương đương thì có cùng giá trị băm. Với mỗi lớp tương đương của $R_{l,r}$, lấy một đại diện, rồi thực hiện chuyển tới $R_{l,mid}$ và $R_{mid+1,r}$.

Bài toán được chuyển thành: sau một số phép cộng phạm vi, xuất ra giá trị của mỗi điểm. Đây là một bài toán cấu trúc dữ liệu trừu tượng mới nhưng đơn giản hơn.

3.6. Ứng dụng

Ví dụ 3.1 (CTS2021). Đo thử cập nhật–truy vấn nửa mặt phẳng. Cho nửa nhóm giao hoán $(D, +)$, nửa nhóm $(M, *)$, và phép tác động hai ngôi $\star : D \times M \rightarrow D$, trong đó $\star$ thỏa tính kết hợp và tính phân phối.

Cho n điểm (x_i, y_i) trên mặt phẳng, mỗi điểm có trọng số ban đầu $d_i \in D$. Cần thực hiện m thao tác. Thao tác thứ j cho a_j, b_j, c_j và thông tin nửa nhóm $o_j \in M$. Với mọi i thỏa

$$a_j x_i + b_j y_i < c_j,$$

trước hết cần trả lời $\sum_i d_i$ trên các điểm đó, sau đó cập nhật mọi d_i tương ứng theo phép tác động bởi o_j . Các thao tác là offline. ⁴

Trong bài này, mỗi phạm vi thao tác là một nửa mặt phẳng. Với một dãy thao tác độ dài m , mặt phẳng bị chia thành $O(m^2)$ vùng. Chia dãy thao tác thành các nhóm độ dài B rồi chia để trị và áp dụng thuật toán trên; đặt $B = \sqrt{n}$, ta thu được độ phức tạp về số phép toán là $O(n + m\sqrt{n})$.

Tiếp theo cần xử lý phần hình học tính toán, có thể dùng thuật toán định vị điểm trên đồ thị phẳng. Cụ thể, với m đường thẳng truy vấn, tính giao điểm của mọi cặp đường thẳng, sắp xếp các giao điểm theo tọa độ x , rồi quét. Trong quá trình quét, duy trì thứ tự của mọi đường thẳng theo tọa độ y tại tọa độ x hiện tại; khi gặp một giao điểm thì hoán đổi thứ tự hai đường thẳng. Đồng thời sắp xếp mọi điểm theo tọa độ x ; khi quét tới một điểm, nhị phân để xác định điểm đó thuộc vùng nào. Như vậy ta thu được thông tin ban đầu của $O(m)$ vùng.

Sau khi khởi tạo rừng có gốc, cần xử lý thay đổi của rừng khi chuyển từ $R_{l,r}$ sang $R_{l,mid}$, tức tìm những vùng nào bị gộp. Với mỗi đường thẳng, sắp xếp các giao điểm trên đường đó theo tọa độ x tăng

⁴<https://qoj.ac/problem/9020>, <https://qoj.ac/problem/4817>.

dẫn và duy trì một danh sách liên kết; mỗi giao điểm thuộc danh sách của hai đường thẳng. Khi xóa đường thẳng đó, duyệt các giao điểm trên danh sách, xóa giao điểm khỏi danh sách của đường thẳng còn lại, rồi gộp hai vùng hai bên. Như vậy hoàn thành quá trình xóa đường thẳng và gộp vùng. Khi khôi phục đường thẳng đã xóa, cũng duyệt danh sách của đường thẳng đó và lần lượt hoàn tác các thao tác.

Độ phức tạp thời gian của cách làm này là $O(n + m\sqrt{n} \log n)$, nút thắt nằm ở sắp xếp giao điểm và nhị phân trong quét. Tuy nhiên bài này là bài tương tác, chỉ cần bảo đảm số phép toán là đúng.

Ví dụ 3.2 (Ynoi2006). rmpq. Cho monoid D , và một mặt phẳng trong đó mỗi điểm có trọng số $d(x, y) \in D$. Cần thực hiện các thao tác sau:

1. Cập nhật: cho đường thẳng $x = x_0$ hoặc $y = y_0$, cùng hai trọng số d_1, d_2 . Cập nhật $d(x, y)$ ở một phía của đường thẳng thành $d(x, y) * d_1$, và phía còn lại thành $d(x, y) * d_2$.
2. Truy vấn: cho x, y , hỏi giá trị $d(x, y)$.

Các thao tác bắt buộc online.⁵

Trong bài này, mỗi thao tác có phạm vi là phạm vi trục giao hai chiều. Dưới một dãy thao tác, điều này tương đương với việc mặt phẳng bị một số đường thẳng ngang và dọc chia thành lưới hình chữ nhật, trong đó mỗi ô là một lớp tương đương. Nếu dãy thao tác có độ dài m , số lớp tương đương là $O(m^2)$.

Chia dãy thao tác thành các nhóm độ dài B và chia để trị. Trước hết dùng chia để trị để xử lý lưới hình chữ nhật do nửa trái và nửa phải thao tác sinh ra, cùng thông tin của mỗi ô. Khi gộp hai lưới, chỉ cần trộn riêng các đường ngang và các đường dọc, là có thể ánh xạ mỗi ô trong lưới con tới ô tương ứng trong lưới hiện tại; như vậy đã hoàn thành việc gộp các lớp tương đương.

Quá trình gộp chia để trị tương đương với việc bắt đầu từ số lớp tương đương ít nhất và lần lượt xây dựng các lớp tương đương cho từng đoạn, tức xây dựng từ gốc của rừng có gốc, đồng thời tính được quan hệ tương ứng giữa các lớp tương đương.

Vì thao tác bắt buộc online, mỗi khi tích lũy đủ B thao tác rồi rạc, ta đóng chúng thành một khối và chia để trị xử lý. Khi truy vấn, định vị điểm trong lưới của m/B khối phía trước để lấy thông tin, còn với khối cuối cùng có ít hơn B thao tác thì truy vấn vết cạn.

Nếu dùng kỹ thuật nhóm nhị phân, mỗi lần thêm ở cuối một khối kích thước 1; khi hai khối cuối có cùng kích thước và chưa vượt ngưỡng B , liên tục gộp hai khối cuối. Cách này cũng đạt hiệu quả chia để trị, và khi truy vấn trên các khối rồi chỉ cần định vị điểm trong $O(\log B)$ khối, giảm hằng số mà không đổi độ phức tạp số phép toán.

Chi phí xử lý chia để trị một dãy thao tác độ dài n là

$$T(n) = 2T(n/2) + O(n^2) = O(n^2).$$

Nếu tổng số thao tác cập nhật và truy vấn là n , chọn $B = \sqrt{n}$ sẽ được độ phức tạp số phép toán $O(n\sqrt{n})$. Định vị điểm cần nhị phân trong các tọa độ x và y của lưới, nên độ phức tạp thời gian là $O(n\sqrt{n} \log n)$, đủ để qua bài này. Có thể dùng kỹ thuật fractional cascading để tối ưu xuống $O(n\sqrt{n})$.

Ví dụ 3.3 (Ynoi2007). TB5x. Cho nửa nhóm giao hoán $(D, +)$, nửa nhóm $(M, *)$, và phép tác động hai ngôi $\star : D \times M \rightarrow D$, trong đó $\star$ thỏa tính kết hợp và tính phân phối. Ban đầu có n điểm (i, p_i) trong mặt phẳng hai chiều, với p là một hoán vị. Cần thực hiện m thao tác.

Mỗi thao tác cho x_1, y_1, x_2, y_2 thỏa

$$0 < x_1 < x_2 < n, \quad 0 < y_1 < y_2 < n.$$

⁵<https://www.luogu.com.cn/problem/P7881>.

Tọa độ lưới của một điểm được định nghĩa là

$$((x \geq x_1) + (x \geq x_2), (y \geq y_1) + (y \geq y_2)).$$

Thao tác gồm:

1. Hỏi tổng trọng số của các điểm có tọa độ lưới (X, Y) , ghi vào $\text{ans}[X][Y]$.
2. Với mỗi điểm có tọa độ lưới (X, Y) , thực hiện cập nhật $\text{o}[X][Y]$.
3. Tọa độ của mọi điểm đồng thời thay đổi. Với một điểm có tọa độ lưới (X, Y) , tọa độ của nó đổi từ (x, y) thành $(x + \text{dx}[X], y + \text{dy}[Y])$, trong đó

$$\text{dx}[0] = 0, \quad \text{dx}[1] = n - x_2, \quad \text{dx}[2] = x_1 - x_2,$$

$$\text{dy}[0] = 0, \quad \text{dy}[1] = n - y_2, \quad \text{dy}[2] = y_1 - y_2.$$

Các thao tác là offline.⁶

Trong bài này, mỗi thao tác có phạm vi là phạm vi trục giao hai chiều. Khác với bài trước, sau mỗi thao tác, tọa độ x và y của mọi điểm đều được hợp thành với một hoán vị.

Ta vẫn dùng cách gộp chia để trị của bài trước: bắt đầu từ số lớp tương đương ít nhất và lần lượt xây dựng các lớp tương đương cho từng đoạn, tức xây dựng từ các lá của chia để trị và từ gốc của rừng có gốc. Mỗi thao tác cập nhật tọa độ là một hoán vị gồm ba đoạn liên tiếp; hợp một hoán vị gồm x đoạn liên tiếp với một hoán vị gồm y đoạn liên tiếp sinh ra nhiều nhất $x + y$ đoạn, và mỗi đoạn là một lớp tương đương.

Khi gộp, dựa vào hoán vị của hai đoạn con trái và phải để tính hoán vị của đoạn hiện tại, đồng thời xác định mỗi đoạn trong hoán vị hiện tại, tức mỗi lớp tương đương, tương ứng với lớp tương đương nào trong hai con. Sau khi xây dựng quan hệ tương ứng của các lớp tương đương và cấu trúc rừng có gốc, dùng thuật toán chia để trị ở trên là giải được bài này. Số phép toán và thời gian đều là

$$O(n + m\sqrt{n}).$$

4. Cây phân hoạch tập điểm

Giả sử cần thiết kế một cây phân hoạch tập điểm sao cho mỗi cập nhật hoặc truy vấn chỉ đệ quy vào một số ít cây con. Khi đó ta có thể hỗ trợ cập nhật–truy vấn phạm vi, đồng thời có thể online.

Với đoạn, cây phân hoạch tập điểm tối ưu là cây đoạn. Độ phức tạp thao tác là $O(n + m \log n)$. Với đường đi đơn trên cây, cây phân hoạch tập điểm tối ưu là static LCT, Top Tree, v.v.; độ phức tạp thao tác cũng là $O(n + m \log n)$.

Với hình chữ nhật trong mặt phẳng hai chiều, cây phân hoạch tối ưu là KDT. Độ phức tạp thao tác là $O(n + m\sqrt{n})$. Với phạm vi trục giao nhiều chiều, cây phân hoạch tối ưu cũng là KDT; trong không gian d chiều với $d > 1$, độ phức tạp thao tác là $O(n + mn^{1-1/d})$.

Trong trường hợp nửa mặt phẳng hai chiều, Optimal Partition Tree [1] là cấu trúc dữ liệu cập nhật–truy vấn phạm vi simplex hai chiều tối ưu: tồn tại một cách phân hoạch tập điểm sao cho mọi simplex hai chiều có thể được biểu diễn bằng tổng các tập điểm của $O(\sqrt{n})$ cây con. Cấu trúc dữ liệu này có hằng số lớn và khó cài đặt.

Optimal Partition Tree có thể mở rộng lên phạm vi simplex nhiều chiều, đồng thời vẫn bảo đảm tối ưu về lý thuyết. Với bài toán d chiều, độ phức tạp thao tác là $O(n + mn^{1-1/d})$.

Lưu ý rằng trong nửa mặt phẳng hai chiều, độ phức tạp của KDT khi dùng làm cây phân hoạch tập điểm là sai; có nhiều cách dựng dữ liệu để phá, chẳng hạn đặt các điểm rời rạc trên một đường tròn và truy vấn bằng nhiều tiếp tuyến gần với đường tròn.

⁶<https://www.luogu.com.cn/problem/P7723>.

5. Bài toán truy vấn phạm vi nửa mặt phẳng

5.1. Dạng bài toán

Trong bài toán này không có cập nhật, chỉ có truy vấn: hỏi thông tin nửa nhóm giao hoán ở một phía của nửa mặt phẳng. Nếu được offline, dùng thuật toán cập nhật–truy vấn nói trên là đạt cận dưới số phép toán $O(n + m\sqrt{n})$. Tuy nhiên khi chỉ có truy vấn và không có cập nhật, còn có một số thuật toán có thể có độ phức tạp thấp hơn hoặc hỗ trợ bất buộc online. Sau đây là vài thuật toán cho bài toán này.

5.2. Đường quét xoay

Chia tập điểm tùy ý thành các khối kích thước B .

Định nghĩa dạng chuẩn của một nửa mặt phẳng như sau: trước hết tịnh tiến nửa mặt phẳng truy vấn xuống dưới cho tới khi chạm điểm đầu tiên, rồi xoay nửa mặt phẳng đó theo chiều kim đồng hồ cho tới khi chạm điểm đầu tiên. Có thể chứng minh phép biến đổi này không làm thay đổi tập điểm ở phía được hỏi của nửa mặt phẳng.

Dưới phép biến đổi này, với B điểm chỉ tồn tại $O(B^2)$ dạng chuẩn nửa mặt phẳng khác nhau. Nếu hai nửa mặt phẳng được biến đổi về cùng một dạng chuẩn, chúng tương ứng cùng một tập điểm và đáp án truy vấn giống nhau; vì vậy chỉ có $O(B^2)$ đáp án cần tiền xử lý.

Quá trình đường quét xoay là: giả sử hiện có một đường thẳng, độ dốc của nó dịch dần từ $-\infty$ tới $+\infty$. Với góc hiện tại θ , duy trì thứ tự của mọi điểm (x, y) theo giá trị $x \cos \theta + y \sin \theta$; cũng có thể xem là với độ dốc hiện tại k , duy trì mọi điểm theo tung độ gốc $y - kx$.

Lấy tùy ý một điểm làm gốc, bắn ra một tia, và chiếu vuông góc mọi điểm lên tia đó. Ở mọi thời điểm, các dạng chuẩn nửa mặt phẳng khác nhau chính là các đường vuông góc từ các điểm xuống tia. Với B điểm, xét nhiều nhất B^2 đường thẳng do từng cặp điểm xác định. Trong quá trình xoay, khi độ dốc trùng với độ dốc đường thẳng nối một cặp điểm, hai điểm đó sẽ đổi thứ tự, và xuất hiện thêm một dạng chuẩn nửa mặt phẳng. Có tổng cộng B^2 lần đổi chỗ, nên có B^2 dạng chuẩn nửa mặt phẳng khác nhau.

Hình 3 trong bản gốc minh họa đường quét xoay: một tia quay quanh gốc, thứ tự các hình chiếu của điểm trên tia chỉ thay đổi khi tia vuông góc với đường nối của một cặp điểm.

Với một truy vấn phạm vi nửa mặt phẳng, khi đường quét tới độ dốc tương ứng, ta nhị phân trong dãy điểm để tìm tiền nhiệm và hậu nhiệm. Khi đó bài toán tương đương với hỗ trợ truy vấn tiền tố, hậu tố trên một dãy. Ta duy trì động dãy điểm của đường quét xoay hiện tại; mỗi lần đổi chỗ đều là đổi hai phần tử kề nhau, nên có thể cập nhật trong $O(1)$.

Chọn $B = \sqrt{m}$, số phép toán trên cấu trúc đại số là $O(n\sqrt{m})$, tổng thời gian là $O(n\sqrt{m} \log n)$. Nút thắt thời gian nằm ở sắp xếp độ dốc và nhị phân định vị điểm. Nếu tử số và mẫu số của các phân số có miền giá trị V , có thể dùng số thực độ chính xác $\text{poly}(V)$ để biểu diễn độ dốc. Khi đó mọi độ dốc đều có thể biểu diễn bằng một số thực, và có thể thực hiện một lần radix sort theo góc cực cho các đường nối từng cặp điểm, làm cho phần sắp xếp không mang thừa số log.

Cần chú ý khác biệt về độ phức tạp so với bài toán có cập nhật: bài toán cập nhật–truy vấn phạm vi có cận dưới $O(n + m\sqrt{n})$, không giảm khi số thao tác tăng; còn nếu chỉ hỗ trợ truy vấn thì có thể đạt $O(n\sqrt{m})$, và độ phức tạp giảm khi số thao tác nhiều hơn.

Nếu mỗi khối tiền xử lý sẵn B^2 kết quả, và dùng cách hợp lý để nhị phân xác định mỗi truy vấn thuộc dạng chuẩn nửa mặt phẳng nào, đường quét xoay có thể online.

5.3. Cây phân hoạch tập điểm

Có thể dùng Optimal Partition Tree đã nêu ở trên để đạt độ phức tạp tối cận dưới, và đây là thuật toán online. Kết hợp với đường quét xoay, ta thu được một số cách làm tốt hơn về lý thuyết:

Trước hết dùng cây phân hoạch tập điểm tối ưu để chia để trị trên tập điểm, sau đó dùng đường quét xoay trên các cây con nhỏ hơn để tiền xử lý mọi trường hợp có thể. Đặt

$$B = m^{2/3}n^{-1/3}.$$

Với các cây con kích thước không quá B , chạy đường quét xoay để tiền xử lý $O(m^{4/3}n^{-2/3})$ dạng chuẩn nửa mặt phẳng khác nhau. Khi truy vấn trên cây phân hoạch tập điểm, đệ quy truy vấn; đến cây con kích thước không quá B thì truy vấn và dừng đệ quy.

Số phép toán trên cấu trúc đại số là $O(n^{2/3}m^{2/3})$, tổng thời gian là $O(n^{2/3}m^{2/3} \log^* n)$. Tuy vậy cách này khó cài đặt, *non-practical*.

5.4. Mo trên nửa mặt phẳng

Mo trên nửa mặt phẳng là thuật toán offline, có thể mở rộng thuật toán Mo sang các truy vấn nửa mặt phẳng.

Ví dụ 5.1 (Ynoi2003). Helesta. Cho n điểm phân biệt (x_i, y_i) , $1 \leq i \leq n$, và m tập

$$S_j = \{(x_i, y_i) \mid A_j x_i + B_j y_i + C_j > 0\}, \quad 1 \leq j \leq m.$$

Cần tìm một hoán vị $p_1, p_2, \dots, p_m$ của $1, 2, \dots, m$, sao cho

$$|S_{p_1}| + \sum_{i=2}^m |S_{p_i} \oplus S_{p_{i-1}}| \leq M.$$

Ở đây M là hằng số cho trước, và

$$A \oplus B = (A \cup B) - (A \cap B).$$

7

Bài toán chuyển thành: duy trì tập điểm, ban đầu rỗng, hỗ trợ thêm điểm và xóa điểm, sao cho mọi tập điểm được một nửa mặt phẳng biểu diễn đều xuất hiện ít nhất một lần trong quá trình. Nói cách khác, cần xây dựng một thứ tự Mo trên nửa mặt phẳng sao cho số lần thêm và xóa điểm không vượt quá M .

Sau đây là một thứ tự Mo nửa mặt phẳng dựa trên ngẫu nhiên, tối ưu theo kỳ vọng. Chọn ngẫu nhiên B điểm. Với mỗi điểm được chọn, dùng điểm đó làm tâm để chạy đường quét xoay; phần này có chi phí $B \times 2n$, vì cần duy trì cả hai phía trên và dưới. B đường quét xoay đều bắt đầu và kết thúc ở cùng một độ dốc, nên chi phí nối chúng lại không vượt quá n .

Với mỗi nửa mặt phẳng truy vấn, chọn tâm xoay gần nó nhất theo chi phí, rồi di chuyển từ tập của tâm xoay đó tới nó để trả lời truy vấn. Do các điểm được rải ngẫu nhiên, khoảng cách di chuyển kỳ vọng mỗi lần là n/B , nên chi phí kỳ vọng của mỗi truy vấn là $2n/B$.

Tổng chi phí là

$$n + 2nB + \frac{2mn}{B}.$$

Chọn $B = \sqrt{m}$, ta được chi phí $4n\sqrt{m}$ nếu bỏ qua hạng n nhỏ hơn. Cài đặt trực tiếp cần tìm vết cận điểm gần nhất cho mỗi truy vấn, thời gian $O(m\sqrt{m})$. Với phép chuyển trong Mo nửa mặt phẳng, cần dùng cấu trúc dữ liệu đếm điểm nửa mặt phẳng, nhưng chi tiết này không xuất hiện trong bài trên.

⁷<https://www.luogu.com.cn/problem/P8529>.

5.5. Trường hợp đặc biệt dữ liệu ngẫu nhiên

Nếu đề bài bảo đảm mọi điểm được sinh ngẫu nhiên trong một hình chữ nhật cho trước, hoặc truy vấn được sinh ngẫu nhiên theo một cách nào đó, và vẫn chỉ có thao tác truy vấn thông tin ở một phía nửa mặt phẳng, ta có một số thuật toán đặc biệt để giải.

Khi tập điểm trong dữ liệu là ngẫu nhiên, KD-Tree có thể xem gần đúng là cây phân hoạch tập điểm tối ưu, nên cây phân hoạch tập điểm tối ưu trong phương pháp trên có thể được cài bằng KDT.

Một phương pháp khác là chia khối mặt phẳng. Giả sử có n điểm, chia mặt phẳng thành $\sqrt{n} \times \sqrt{n}$ khối. Vì tập điểm ngẫu nhiên đều, một khối kỳ vọng chỉ có hằng số điểm, nên độ phức tạp kỳ vọng của truy vấn trong khối là $O(1)$.

Một nửa mặt phẳng có thể xem như $O(\sqrt{n})$ truy vấn tiền tố hoặc hậu tố theo hàng, cùng $O(\sqrt{n})$ truy vấn trong khối. Ta có thể tiền xử lý mọi thông tin tiền tố và hậu tố, rồi vét cạn các điểm lẻ.

Số phép toán trên cấu trúc đại số là $O(n + m\sqrt{n})$, tổng thời gian cũng là $O(n + m\sqrt{n})$. Chia khối mặt phẳng là cách làm chạy nhanh nhất.

5.6. Ứng dụng

Ví dụ 5.2 (UNR #4). Tập axit caproic. Cho n điểm (x_i, y_i) trên mặt phẳng. Có m truy vấn; mỗi truy vấn cho đường tròn tâm $(0, z_i)$, bán kính R_i , và hỏi số điểm nằm trong đường tròn.⁸

Điều kiện có thể chuyển thành

$$x_i^2 + y_i^2 + z_i^2 - 2y_i z_i \leq R_i^2,$$

tức

$$2z_i y_i + R_i^2 - z_i^2 \geq x_i^2 + y_i^2.$$

Nếu ánh xạ (x_i, y_i) thành $(y_i, x_i^2 + y_i^2)$, bài toán chuyển thành hỏi số điểm nguyên nằm dưới đường thẳng

$$y = 2z_i x + R_i^2 - z_i^2,$$

và có thể dùng đường quét xoay để giải.

Ví dụ 5.3 (CTS2022). Tất. Cho n điểm (x_i, y_i, c_i) . Có m truy vấn; mỗi truy vấn cho A, B, C , hỏi số cặp có thứ tự (i, j) thỏa

$$Ax_i + By_i + C < 0, \quad Ax_j + By_j + C < 0, \quad c_i = c_j.$$

Bảo đảm tập điểm được sinh ngẫu nhiên, tức c_i, x_i, y_i lần lượt được chọn ngẫu nhiên trong các khoảng cho trước.⁹

Bài này là truy vấn tổng bình phương số điểm mỗi màu trên một nửa mặt phẳng.

Solution 1. Đường quét xoay. Sắp xếp điểm theo trọng số màu c_i , rồi chia khối kích thước B . Với mỗi truy vấn, hỏi thông tin trong từng khối: số lần xuất hiện của màu đầu tiên và màu cuối cùng trong nửa mặt phẳng, cũng như tổng bình phương số lần xuất hiện của các màu ở giữa. Thông tin này là thông tin nửa nhóm có thể gộp, nên gộp tuần tự đáp án của từng khối sẽ thu được đáp án.

Dùng đường quét xoay để xử lý truy vấn trong mỗi khối. Bài toán chuyển thành duy trì một dãy gồm các điểm, thực hiện $O(B^2)$ lần đổi chỗ hai vị trí kề nhau, và duy trì thông tin. Có thể trực tiếp duy trì rank hiện tại của mỗi điểm trong dãy và dãy điểm hiện tại.

⁸<https://uoj.ac/problem/553>.

⁹<https://www.luogu.com.cn/problem/P8261>.

Độ phức tạp thời gian là

$$O\left(\frac{n}{B}(B^2 + m) \log n\right).$$

Chọn $B = \sqrt{m}$ thu được $O(n\sqrt{m} \log n)$.

Solution 2. Cây phân hoạch tập điểm. Đảo vai trò tập điểm và phạm vi truy vấn, tính đóng góp của tập điểm cho truy vấn. Điểm (a, b) nằm một phía của nửa mặt phẳng $cx + dy < e$. Giả sử e không âm, điều kiện có thể chuyển thành

$$\frac{c}{e}x + \frac{d}{e}y < 1,$$

tức điểm $(c/e, d/e)$ nằm một phía của nửa mặt phẳng $ax + by < 1$. Như vậy mỗi truy vấn được chuyển thành một điểm, và ta xây KDT trên mọi điểm truy vấn.

Với mỗi màu, xét đóng góp của màu đó cho mọi truy vấn. Trên mỗi điểm của KDT duy trì một biến cnt. Liệt kê các điểm có màu x ; với nửa mặt phẳng tương ứng của điểm đó, trong KDT sẽ truy cập một số cây con, và ta gắn tag cộng 1 vào cnt của các cây con đó.

Sau khi thực hiện xong mọi thao tác của màu này, ta thăm tất cả nút KDT vừa được truy cập và cộng đáp án của cây con tương ứng. Do dữ liệu ngẫu nhiên, độ phức tạp của KDT là đúng. Nếu dùng cây phân hoạch tập điểm tối ưu, độ phức tạp thời gian là $O(n\sqrt{m})$.

Ví dụ 5.4 (Ynoi2000). hpi. Cho n điểm phân biệt (x_i, y_i) . Có m truy vấn; mỗi truy vấn cho A, B, C , hỏi số cặp có thứ tự (i, j) thỏa

$$\begin{aligned}x_i < x_j, \quad y_i < y_j, \\ Ax_i + By_i + C > 0, \quad Ax_j + By_j + C > 0.\end{aligned}$$

Bảo đảm dữ liệu tập điểm là ngẫu nhiên.¹⁰

Bài này là đếm cặp thống trị trong nửa mặt phẳng. Ta phân loại hướng của nửa mặt phẳng truy vấn. Trước hết xử lý riêng nửa mặt phẳng có $A = 0$ hoặc $B = 0$.

Nếu $A \times B > 0$, nửa mặt phẳng trong hệ tọa độ Descartes hướng về trái dưới hoặc phải trên; hai phần đối xứng nhau, nên chỉ xét trường hợp hướng trái dưới. Đặt trọng số điểm f_i là số điểm ở trái dưới nó, tức số điểm thỏa

$$x_j < x_i, \quad y_j < y_i.$$

Nếu nửa mặt phẳng hướng về trái dưới, chỉ cần thống kê tổng f_i của các điểm trong nửa mặt phẳng, trở thành bài toán đếm điểm nửa mặt phẳng.

Nếu $A \times B < 0$, nửa mặt phẳng trong hệ tọa độ Descartes hướng về trái trên hoặc phải dưới; hai phần đối xứng nhau, nên chỉ xét trường hợp hướng phải dưới. Đặt trọng số điểm f_i là số điểm ở phải dưới nó, tức số điểm thỏa

$$x_j > x_i, \quad y_j < y_i.$$

Khi đó chỉ cần thống kê tổng số cặp điểm trong nửa mặt phẳng, rồi trừ tổng f_i của mọi điểm trong nửa mặt phẳng là được đáp án; đây cũng là bài toán đếm điểm nửa mặt phẳng.

Vì dữ liệu của bài là ngẫu nhiên, có thể dùng KDT hoặc chia khối mặt phẳng; trong đó chia khối mặt phẳng chạy hiệu quả hơn các thuật toán khác.

¹⁰<https://www.luogu.com.cn/problem/P9996>.

6. Tổng kết

Bài viết trước hết giới thiệu thuật toán chia để trị cho cập nhật–truy vấn phạm vi offline tối ưu. Thuật toán này có tính mở rộng khá mạnh, giải được một lớp bài toán có thông tin thỏa tính chất hai nửa nhóm.

Bài viết cũng giới thiệu cây phân hoạch tập điểm, đường quét xoay, chia khối mặt phẳng và một số thuật toán khác. Các thuật toán này cũng có thể giải một số bài toán trong nửa mặt phẳng hoặc không gian nhiều chiều, đồng thời có thể đạt độ phức tạp thấp hơn hoặc hỗ trợ bất buộc online.

Do quy mô chấm hiện nay, trong các bài truyền thống đôi khi chưa phân biệt được thuật toán độ phức tạp thấp với thuật toán vét cạn. Bài toán cập nhật–truy vấn trong không gian nhiều chiều hiện vẫn xuất hiện rất ít trong OI, và ứng dụng của nó còn chờ được khai thác thêm.

Tác giả hy vọng bài viết này có thể gợi mở và đem lại thêm cảm hứng cho các bài toán liên quan trong OI.

7. Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn các huấn luyện viên đội tuyển quốc gia Peng Sijin và Yang Yaoliang đã hướng dẫn.

Cảm ơn gia đình đã đồng hành và ủng hộ.

Cảm ơn các thầy Xu Xianyou, Wang Jiahong và Zhou Bang của Trường Trung học Học Quân đã quan tâm và hướng dẫn.

Cảm ơn các anh Li Xinlong và Cai Chengze đã trao đổi, thảo luận và gợi ý cho tác giả.

Cảm ơn các bạn học ở Trường Trung học Học Quân đã đọc duyệt bài viết.

Cảm ơn các bạn và anh chị trong đội bạn đã giúp đỡ và đồng hành.

Cảm ơn những thầy cô và bạn học khác đã giúp đỡ tác giả.

Tài liệu tham khảo

1. Timothy M. Chan. *Optimal partition trees*. Proceedings of the 26th ACM Symposium on Computational Geometry, pages 1–10, 2010.
2. Xinlong Li and Chengze Cai. *Offline optimal range query and update algorithm*. 2021.

Bài 6. Bàn về một số phương pháp giải bài toán quy hoạch động trong OI

Cao Li, Trường Trung học Dư Diêu, Chiết Giang

Tóm tắt

Bài toán quy hoạch động giữ vị trí quan trọng trong OI, nhưng đường lối giải lại biến hóa rất nhiều. Bài viết này liệt kê các nguyên tắc thiết kế quy hoạch động, thảo luận những cách nghĩ và thủ pháp xử lý có tính phổ biến trong thực hành giải bài, rồi thông qua ví dụ minh họa tác dụng và hoàn cảnh áp dụng của chúng.

1. Đặc trưng của quy hoạch động

Trong tin học, quy hoạch động (*dynamic programming*, viết tắt là dp) là phương pháp chia bài toán ban đầu thành một số bài toán con rồi giải đệ quy. Các bài toán tổ hợp có thể giải tốt bằng quy hoạch động

thường có những tính chất sau.

- Nếu là bài toán tối ưu, nghiệm tối ưu của bài toán gốc được ghép từ nghiệm tối ưu của các bài toán con. Đây là điều kiện cần cho tính đúng đắn.
- Các bài toán con lặp lại. Nhờ ghi nhớ kết quả của một bài toán con, ta tránh giải lại nhiều lần và giảm độ phức tạp so với đệ quy hay chia để trị trực tiếp.
- Không có hậu hiệu: cách tách và giải bài toán gốc không được phụ thuộc trực tiếp hoặc gián tiếp vào chính nó, để thuật toán cuối cùng vận hành bình thường.

Trong thi tin học, ta thường gặp một lớp bài toán định nghĩa một loại đối tượng tổ hợp có cấu trúc đặc biệt, rồi yêu cầu thống kê một thông tin nào đó của các đối tượng ấy, chẳng hạn tính tồn tại, giá trị lớn nhất hoặc tổng. Một cách nhìn hiệu quả là tìm một trình tự khắc họa dần đối tượng: mỗi bước chỉ quyết định một phần cục bộ của cấu trúc. Khi đó đối tượng gốc được ghép từ một dãy bước, và một đoạn tiền tố hoặc một số bước con trong dãy đó chính là một giai đoạn của bài toán.

Từ cách khắc họa này, một thuật toán quy hoạch động thường gồm các phần:

- mô tả đặc trưng cấu trúc của bài toán con, tức những thông tin cần ghi lại tại mỗi giai đoạn, gọi là **trạng thái**;
- đáp án của bài toán con: với mỗi giai đoạn và trạng thái, thống kê giá trị thu được từ mọi phần đã quyết định phù hợp, gọi là **giá trị**;
- quan hệ giữa các bài toán con, tức biến đổi trạng thái và giá trị khi quyết định một hoặc nhiều bước, gọi là **chuyển trạng thái**;
- trường hợp cơ sở, tức giá trị của bài toán con không thể chia nhỏ thêm;
- trường hợp mục tiêu, tức bài toán con tương ứng với bài toán ban đầu.

Tư tưởng cốt lõi là tách mô hình phức tạp thành một tổ hợp các bước đơn giản hơn. Khi thiết kế, tư tưởng này thường thể hiện theo hai hướng. Thứ nhất, với một bài toán con, ta quyết định một phần trong giai đoạn của nó để tách thành nhiều phần hoặc nhiều trường hợp độc lập nhỏ hơn. Đây là cách nghĩ đệ quy. Thứ hai, với một bài toán con đã giải, ta quyết định thêm một phần bên ngoài giai đoạn hiện tại để tạo bài toán con lớn hơn và tính đóng góp vào giá trị của nó. Đây là cách nghĩ gia tăng. Cả hai đều có thể dẫn đến một thiết kế dp; trong từng mô hình cụ thể, nên chọn hướng làm cho trạng thái và chuyển trạng thái tự nhiên hơn.

2. Thiết kế quy hoạch động

Trong phần này, để gọn, ta viết quy hoạch động là dp. Các biến trong ví dụ được xem là số nguyên không âm nếu không nói khác. Do giới hạn dung lượng, lời giải ví dụ chỉ nêu định nghĩa bài toán con và ý tưởng chuyển trạng thái; các chi tiết cài đặt còn lại để người đọc tự phân tích.

2.1. Suy ra trạng thái từ cách khắc họa từng bước

Trước hết phân tích cấu trúc đối tượng cần đếm hoặc tối ưu, xây dựng một thứ tự khắc họa đối tượng đó, rồi chọn một số trạng thái trung gian làm giai đoạn. Sau đó trừu tượng hóa đặc trưng đại số của các giai đoạn, xác định trạng thái và giá trị, cuối cùng suy ra chuyển trạng thái.

Cách này phù hợp khi cấu trúc đối tượng khá đơn giản, chẳng hạn dãy, cây có gốc, tập hợp, hoặc một số tổ hợp đơn giản của chúng. Khi thiết kế nên đi từ thô đến mịn: trước hết xác định thứ tự tổng quát,

sau đó tính chỉnh cách chọn giai đoạn và thứ tự quyết định nhiều đại lượng cục bộ, rồi mới chỉnh chi tiết trạng thái và chuyển trạng thái. Việc chọn thứ tự rất quan trọng, vì các thứ tự khác nhau có thể cho số trạng thái khác nhau rất lớn.

Ví dụ 2.1.1 (Wooden Spoon!). Cho một cây nhị phân đầy đủ có 2^n lá; trên lá ghi một hoán vị của $1, \dots, 2^n$. Mỗi đỉnh trong ghi giá trị nhỏ hơn trong hai con. Nếu các giá trị trên đường từ lá mang số i đến gốc đôi một khác nhau, gọi i là người đoạt giải. Với mọi i , cần đếm số trường hợp i đoạt giải, modulo 998244353.

Nếu xét từ dưới lên, với cây con hiện có 2^k lá, ta phải ghép nó với một cây con cùng kích thước có giá trị gốc lớn hơn. Dùng giá trị tuyệt đối buộc phải đồng thời liệt kê người đoạt giải và giá trị gốc hiện tại; dùng thứ hạng tương đối thì chuyển trạng thái trở thành dạng tích chập. Dù có thể tối ưu bằng NTT, cách này nhiều chi tiết và hằng số lớn.

Nếu xét từ trên xuống, giá trị tại một đỉnh trùng với giá trị của một con. Cây con đó không thể chứa người đoạt giải và có thể được tính trực tiếp bằng công thức; cây con còn lại mới chứa đường đi của người đoạt giải. Vì thế mỗi giai đoạn chỉ cần ghi thêm giá trị tại đỉnh hiện tại. Nếu các giá trị trên đường từ lá đoạt giải đến gốc lần lượt là $x_0, \dots, x_n$, đơn điệu giảm, số phương án có dạng

$$2^n \prod_{i=1}^n \binom{2^n - x_i - 2^{i-1}}{2^{i-1} - 1} (2^{i-1})!.$$

Đặt $f_{i,j}$ là số phương án khi đang ở đỉnh thứ $i + 1$ theo thứ tự từ gốc xuống và $x_{n-i} = j$. Dùng hậu tố để tối ưu, mỗi lần chuyển là $O(1)$; số trạng thái là $O(n2^n)$, nên tổng thời gian cũng là $O(n2^n)$.

Ví dụ 2.1.2 (MEX counting). Cho n, k và dãy $b_1, \dots, b_n$. Cần đếm số dãy $a_1, \dots, a_n$, với $a_i \in [0, n] \cap \mathbb{Z}$, sao cho

$$|\text{mex}(a_1, \dots, a_i) - b_i| \leq k$$

với mọi i , modulo 998244353.

Đi từ trái sang phải là tự nhiên vì ràng buộc nói về tiền tố. Nhưng nếu trạng thái chỉ ghi mex, để chuyển kén ta phải ghi cả những số lớn hơn mex đã xuất hiện, dẫn đến số trạng thái mũ.

Cách tránh mâu thuẫn là chưa quyết định phần lớn hơn mex cho đến khi chúng thật sự ảnh hưởng tới mex. Đặt $f_{i,j,c}$ là số phương án sau khi đã quyết định i vị trí, hiện tại mex = j , và trước đó có c số lớn hơn j . Khi quyết định $a_{i+1} = j$, mex tăng; lúc này mới tiếp tục quyết định các số chưa quyết định bằng $j + 1, j + 2, \dots$, nếu chúng tồn tại thì sẽ tạo chuỗi tăng mex. Chuyển trạng thái dạng tích chập có thể tối ưu; sửa nhẹ định nghĩa c thành “số loại giá trị khác nhau lớn hơn j ” sẽ đạt $O(n^2)$.

Thực quan hơn, nếu xem mỗi a_i là một điểm (i, a_i) , các giai đoạn của lời giải đúng chính là các hình chữ nhật có gốc tại $(0, 0)$ đã được xác định; phần phía trên hình chữ nhật chỉ ghi số loại tung độ. Chỉ đi theo một chiều đơn lẻ sẽ thất bại.

Ví dụ 2.1.3 (đường đi biểu diễn với nhiên liệu và năng lượng). Cho đồ thị vô hướng có $n + m + 1$ đỉnh và cạnh có trọng số dương. Cần đi một chu trình xuất phát và kết thúc tại đỉnh 0, lần lượt biểu diễn tại các đỉnh $1, \dots, n$, trong khi duy trì hai đại lượng nhiên liệu và năng lượng luôn không âm. Ở đỉnh 0 có thể tốn thời gian để nạp đầy cả hai; ở một số đỉnh khác có thể nạp nhiên liệu. Cần tối thiểu hóa tổng thời gian.

Vì giới hạn nhiên liệu và năng lượng đều lớn, không nên ghi trực tiếp chúng trong trạng thái. Với năng lượng, chỉ cần kiểm soát đoạn biểu diễn nằm giữa hai lần nạp tại đỉnh 0; bài toán chuyển thành, với mọi đoạn con của $1, \dots, n$, tính thời gian ít nhất để từ 0 xuất phát với năng lượng đầy, hoàn thành đoạn đó rồi quay lại 0.

Để tránh ghi nhiên liệu còn lại, giai đoạn nên là thời điểm nạp nhiên liệu: $f_{i,r}$ biểu thị thời gian ít nhất sau khi đã hoàn thành biểu diễn tại $1, \dots, r$ và vừa nạp nhiên liệu tại đỉnh i . Chuyển thẳng sẽ quá chậm. Sau khi tiền xử lý đường đi ngắn nhất giữa các đỉnh, điều kiện một trạng thái có thể chuyển đến trạng thái tiếp theo chỉ phụ thuộc vào một số khoảng cách và ngưỡng, nên có thể dùng cây đoạn theo giá trị hoặc cây cân bằng để duy trì giá trị nhỏ nhất trong các trạng thái đủ điều kiện. Phần chuyển bên trong một đoạn vẫn cần tiền xử lý mọi cặp điểm, là nút thắt $O(n^3)$ của lời giải.

Ví dụ 2.1.4 (RAY). Bài toán cho các ràng buộc giữa một hoán vị chưa biết và một dãy giá trị, yêu cầu đếm số hoán vị thỏa mãn. Cách trực tiếp theo thứ tự hạng buộc trạng thái ghi chỉ số cuối cùng và giá trị cuối cùng, dẫn đến số trạng thái quá lớn.

Sau khi không còn tối ưu rõ ràng ở thứ tự và giai đoạn, cần phân tích mô hình để bỏ đại lượng dư thừa. Quan hệ tăng có thể viết lại qua hiệu giữa hai giá trị kề nhau theo hạng, chẳng hạn dưới dạng

$$b_{p_i} - b_{p_{i-1}} = \max(a_{p_i} - a_{p_{i-1}} + [p_i < p_{i-1}], 0).$$

Do đó thay vì quyết định trực tiếp giá trị b , ta quyết định hiệu giữa hai giá trị b kề hạng. Cách đổi góc nhìn này cho phép bỏ “giá trị tổng trước đó” khỏi trạng thái. Đồng thời, nếu tổng còn thiếu nhỏ hơn giới hạn, luôn có thể cộng vào giá trị đầu để đạt tổng mục tiêu mà không phá các điều kiện khác; vì thế chỉ cần lấy hiệu bằng cận dưới. Kết quả thu được lời giải cỡ $O(n^2 m 2^n)$.

Ví dụ 2.1.5 (HD). Với mỗi $n = 2, \dots, N$, cần đếm cặp cây có gốc trên cùng tập đỉnh, trong đó một cây có cha của mỗi đỉnh nhỏ hơn chính nó, cây kia có cha lớn hơn chính nó, và với mỗi nhãn, đỉnh đó là lá trong đúng một cây.

Nếu quyết định trực tiếp, trong một cây phần đã xét tạo một khối liên thông chứa gốc và các đỉnh được chia thành nhiều loại; trong cây còn lại chúng tạo nhiều khối con. Trạng thái tự nhiên phải ghi quá nhiều số lượng, còn chuyển trạng thái phải liệt kê số con của đỉnh mới.

Với điều kiện “phải có con”, có thể dùng bao hàm–loại trừ để chuyển thành “có thể có con”. Một cách nhìn khác là chỉ ghi một biến: hiện có bao nhiêu đỉnh trong phần đã quyết định còn cần nối con ở các bước sau. Khi quyết định cha của đỉnh mới, đồng thời quyết định liệu đó có phải con cuối cùng của cha hay không.

Đối với cây còn lại, thay vì mô tả chi tiết các khối đã quyết định, có thể ghi “yêu cầu” đặt lên phần chưa quyết định, rồi dùng cùng kiểu chuyển trạng thái như cây thứ nhất. Bản chất là đổi từ quyết định tập con của mỗi đỉnh sang quyết định cha của mỗi đỉnh. Nhờ vậy trạng thái đóng dưới chuyển trạng thái và đạt độ phức tạp đa thức theo N với một thừa số lũy thừa nhỏ.

2.2. Suy ra trạng thái từ quan hệ phụ thuộc của bài toán

Cách thứ hai là bắt đầu từ một bài toán con cố định, thường là chính bài toán gốc, chọn một phần cục bộ để quyết định, rồi xét phần còn lại phụ thuộc vào quyết định ấy ra sao. Nếu phần còn lại tách thành nhiều phần hoặc nhiều trường hợp thì tiếp tục tách. Phân tích đệ quy đến khi dạng bài toán hiện tại trùng với các bài toán con phụ thuộc; khi đó trạng thái và chuyển trạng thái cùng được xác định.

Cách này hữu ích khi cấu trúc đối tượng phức tạp, điều kiện khó xử lý, hoặc định nghĩa trạng thái không hiển nhiên. Mấu chốt là chọn đúng cục bộ để quyết định và mô tả đúng cấu trúc phần còn lại.

Ví dụ 2.2.1 (Bracket Insertion). Bắt đầu từ dãy ngoặc rỗng, thực hiện n lần: chọn đều ngẫu nhiên một khe và chèn vào đó một cặp ngoặc. Cặp được chèn là $()$ với xác suất p , và $()$ với xác suất $1 - p$. Cần tính xác suất cuối cùng dãy ngoặc hợp lệ, modulo 998244353.

Điều kiện hợp lệ là mọi tiền tố có số ngoặc trái không ít hơn số ngoặc phải. Xét ngoặc đầu tiên của dãy cuối cùng, nó chắc chắn là ngoặc trái; cùng với nó có một ngoặc được chèn trong cùng thao tác. Bỏ cặp ngoặc cùng thao tác này đi, dãy bị chia thành hai đoạn. Không có cặp nào được chèn đồng thời mà nằm ở hai đoạn khác nhau; các ngoặc trong đoạn thứ nhất phải được chèn sau cặp vừa bỏ, còn thứ tự tương đối giữa hai đoạn có thể tính bằng tổ hợp.

Sau khi tách, đoạn thứ nhất phải thỏa điều kiện mọi tiền tố có số ngoặc trái không ít hơn số ngoặc phải cộng thêm một ngưỡng; đoạn thứ hai thỏa điều kiện ban đầu. Vì vậy bài toán con tự nhiên là: độ dài $2i$, mọi tiền tố có số ngoặc trái không ít hơn số ngoặc phải cộng j . Gọi giá trị là $f_{i,j}$. Chuyển trạng thái thu được từ việc chọn vị trí cặp vừa tách và nhân hệ số tổ hợp; chỉ cần xét $0 \leq j \leq n - i$. Độ phức tạp cơ bản là $O(n^2)$, có thể tối ưu bằng NTT.

Ví dụ 2.2.2 (RF). Cho một đa tập gồm các lũy thừa của 2. Cần đếm bao nhiêu số y có thể đạt được bằng cách chọn một phần tử con của đa tập rồi chia thành k nhóm có tổng bằng nhau, sao cho tổng mỗi nhóm bằng y .

Xét từng bit nhị phân. Với một y , điều kiện cần và đủ có thể diễn đạt bằng các bất đẳng thức tiền tố trên số lượng lũy thừa 2^i : nếu lấy y_i là phần thấp i bit của y , thì ở mỗi mức i , số lượng khả dụng ở các bit thấp phải đủ bù cho phần dư $k - y_i$ theo đơn vị 2^i . Tính cần là hiển nhiên; tính đủ chứng minh bằng quy nạp: lấy các phần tử lớn nhất đủ tạo $k2^h$ ở bit cao nhất của y , chia đều cho k nhóm, rồi quy về các bit thấp.

Chứng minh này cho một cách tách bài toán: liệt kê bit cao nhất của y , lấy ra phần $k2^h$, phần còn lại lại là cùng dạng bài toán nhưng với giới hạn bit nhỏ hơn. Khi một mức bit không đủ hoặc dư quá nhiều, bài toán con tách thành “đếm các số có bit cao nhất không quá j ” cộng với một bài toán đặc biệt cùng dạng. Sau $O(\log V)$ lần phân tách sẽ về trường hợp cơ sở. Vì thế, nếu trả lời nhanh truy vấn “đa tập S đạt được bao nhiêu số có bit cao nhất không quá j ”, ta tính được đáp án trong $O(\log V)$.

Điểm cần chú ý là nên tìm các bài toán con chỉ phụ thuộc vào thông tin ban đầu hoặc phụ thuộc rất ít vào quyết định cục bộ; khi đó số trạng thái mới đủ nhỏ.

Ví dụ 2.2.3 (bài toán tập độc lập trọng số lớn nhất). Cho một cây nhị phân, mỗi đỉnh có trọng số. Cần xóa lần lượt mọi cạnh theo thứ tự bất kỳ; khi xóa một cạnh, hai đầu mút đổi trọng số cho nhau và chi phí tăng bằng tổng hai trọng số ấy. Hãy tối thiểu hóa tổng chi phí.

Chỉ xét một đỉnh có đủ hai con; lá và đỉnh một con đơn giản hơn. Với một đỉnh u , gọi cạnh nối với cha là f_u , hai cạnh nối với con trái và con phải là l_u, r_u . Nếu phân tích thứ tự xóa ba cạnh kề u , các trường hợp sinh ra nhiều dạng bài toán con: có khi cần biết trọng số cha ban đầu và trọng số cha cuối cùng, có khi chỉ cần chi phí nhỏ nhất sau khi xóa toàn bộ cạnh trong cây con.

Nếu tách quá nhỏ, ta phải liệt kê đồng thời hai trọng số trong các bài toán con, dẫn đến $O(n^3)$ hoặc hơn. Cách giảm bậc là mở rộng “cục bộ” được phân tích: khi xét một trường hợp, tiếp tục xét ba cạnh kề đỉnh con liên quan. Nhờ tính cộng của chi phí và phép lấy min, phần phụ thuộc vào từng trọng số có thể tiền xử lý thành các giá trị nhỏ nhất như

$$\min_j \{f(x, j) + d_j + \text{cost}(d_i, d_j)\}.$$

Khi đó các trạng thái đóng dưới chuyển trạng thái và đạt $O(n^2)$. Ví dụ này cho thấy thay đổi phạm vi quyết định cục bộ có thể biến hai biến liệt kê lồng nhau thành hai phần độc lập.

2.3. Tiểu kết

Trong thực hành, không nên cố chấp một khuôn mẫu suy nghĩ. Kết hợp nhiều thứ tự và góc nhìn thường là cách nhanh nhất tìm ra điểm đột phá, nhất là khi chọn giai đoạn và tối ưu trạng thái.

Một dấu hiệu quan trọng của quy hoạch động là **tính độc lập**. Nó thể hiện ở hai mức. Thứ nhất, quan hệ giữa phần đã quyết định và phần chưa quyết định có thể được trừu tượng bằng ít đại lượng; nghĩa là chi tiết bên trong phần đã quyết định hầu như không ảnh hưởng đến toàn cục ngoài các đại lượng ấy. Thứ hai, nhiều bài toán con độc lập, hoặc quan hệ giữa chúng cũng có thể được mô tả bằng ít đại lượng. Tính độc lập giúp ta không cần ghi toàn bộ lịch sử, vẫn bảo đảm cấu trúc con tối ưu và không đếm trùng hoặc bỏ sót; nhờ vậy mới xuất hiện các bài toán con lặp lại và giảm được độ phức tạp.

3. Kỹ thuật chuyển hóa bài toán

Một số bài toán không lộ rõ đặc trưng quy hoạch động, hoặc không thể giải trực tiếp bằng dp, nhưng có thể chuyển hóa thành bài toán phù hợp. Các thủ pháp dưới đây là những ví dụ khá điển hình.

3.1. Liệt kê

Liệt kê để giảm số điều kiện và lượng thông tin phải xét là kỹ thuật cơ bản.

Ví dụ 3.1.1 (Infinite Game). Cho xâu s gồm các ký tự $a, b, ?$; mỗi dấu hỏi cần thay bằng a hoặc b . Xét quá trình dựa trên s^∞ : liên tục lấy ký tự đầu làm kết quả một ván giữa a và b ; bên tương ứng được một điểm, và khi một bên đạt hai điểm thì ván kết thúc, điểm được đặt lại. Gọi $r(s)$ là giới hạn của tỉ lệ số ván a thắng. Cần đếm số cách thay dấu hỏi sao cho $r(s) < 1/2$, $= 1/2$, hoặc $> 1/2$.

Chỉ cần xét dấu của hiệu số ván thắng giữa a và b . Khi chạy trên s^∞ , quá trình chắc chắn có chu kỳ, nhưng chu kỳ ván không nhất thiết trùng với chu kỳ s , vì một ván có thể băng qua cuối xâu. Giữa một ván chỉ có bốn trạng thái tỉ số điểm; vì thế với mỗi tỉ số điểm ban đầu, chạy hết một vòng s sẽ cho tỉ số cuối và hiệu số ván thắng. Điều này có thể xem như một đồ thị hàm trên bốn trạng thái.

Trạng thái vẫn nhiều nếu ghi toàn bộ. Nhưng ta chỉ cần tổng trên chu trình, nên có thể liệt kê trước tập trạng thái nằm trên chu trình, tổng cộng $2^4 - 1 = 15$ khả năng. Khi đó trạng thái chỉ cần ghi một hiệu và ánh xạ giữa bốn tỉ số điểm; về lý thuyết có $4^4 = 256$ ánh xạ nhưng thực tế chỉ 67 ánh xạ có thể xuất hiện. Cuối cùng ép ánh xạ phù hợp với tập chu trình đã liệt kê, đạt $O(n^2)$.

3.2. Nhị phân

Nhị phân là tối ưu hóa liệt kê khi một đại lượng có tính đơn điệu, thường dùng để biến bài toán tối ưu thành bài toán kiểm tra có thể xử lý bằng dp.

3.2.1. Nhị phân đáp án.

Ví dụ 3.2.1 (Assigning Fares). Cho một cây n đỉnh và m đường đi trên cây. Cần gán trọng số nguyên dương $c_1, \dots, c_n$ cho các đỉnh sao cho trên mỗi đường đi, trọng số đỉnh đều tăng nghiêm ngặt hoặc giảm nghiêm ngặt; cần tối thiểu hóa trọng số lớn nhất.

Nếu dp trực tiếp, thông tin của một cây con gồm trọng số gốc và trọng số lớn nhất trong cây con, không thể chấp nhận. Nhị phân trước giới hạn trọng số k . Khi đó giá trị một đỉnh có thể lấy được phụ thuộc vào giá trị nhỏ nhất của các con phải nhỏ hơn nó và giá trị lớn nhất của các con phải lớn hơn nó. Đặt f_u là giá trị nhỏ nhất mà c_u có thể lấy khi c_u bắt buộc nhỏ hơn cha (nếu không có đường nào cùng đi qua hai đỉnh thì không ràng buộc). Trường hợp đối xứng có giá trị lớn nhất là $k + 1 - f_u$.

Với một đỉnh u , xét các con cùng nằm với u trên ít nhất một đường đi. Ràng buộc phụ có dạng $c_v < c_u$, hoặc với hai con v, w ,

$$[c_v < c_u] \neq [c_w < c_u].$$

Chúng có thể xử lý bằng DSU có trọng số hoặc DSU phân loại. Cuối cùng, các ràng buộc trên c_u gồm cận trên, cận dưới, và một số khoảng dạng $[l, r] \cup [k + 1 - r, k + 1 - l]$; lấy giao các ràng buộc sẽ suy ra f_u .

3.2.2. WQS nhị phân. Bài toán tối ưu bằng WQS thường có dạng: biết một hàm lồi hoặc lõm $f(x)$, cho a , cần tính $f(a)$ nhưng không thể trực tiếp. Cách quen dùng trong OI là nhị phân hệ số góc k , tính nhanh

$$\max_x \{f(x) - kx\}$$

và một x_0 đạt cực trị, rồi điều chỉnh k theo quan hệ giữa x_0 và a . Tuy nhiên trong thực hành, việc phải khôi phục x_0 từ thông tin dp làm tăng đáng kể độ khó, đặc biệt khi lồng nhiều lớp WQS.

Một cách tránh là xét

$$g(k) = \max_x \{f(x) - kx\} + ka.$$

Nếu f là hàm lồi lên theo nghĩa thường dùng cho WQS trong bài viết, thì $g(k)$ là hàm lồi xuống, và giá trị nhỏ nhất của nó là $f(a)$. Vì vậy bài toán chuyển thành tìm cực trị của $g(k)$, có thể xử lý bằng tam phân hoặc nhị phân trên hệ số mà không cần khôi phục phương án. Giải thích chi tiết hơn có trong tài liệu tham khảo của bài.

Ví dụ 3.2.2 (Aliens!!!). Cho một số ô được đánh dấu trên lưới $m \times m$, và một số k . Cần chọn nhiều nhất k hình vuông con có đường chéo chính trùng hướng với đường chéo chính của cả lưới, sao cho mọi ô được đánh dấu đều được phủ, và tối thiểu hóa số ô trong hợp các hình vuông.

Bài toán chuyển thành chọn nhiều nhất k đoạn trên $[1, m]$ để phủ tất cả các đoạn đã cho. Bỏ các đoạn bị đoạn khác chứa; khi đó hai đầu mút trái và phải của các đoạn còn lại đều tăng. Trong một nghiệm, các đoạn đã cho được chia thành nhiều nhóm liên tiếp, mỗi nhóm do một đoạn chọn phủ. Đặt $w(x, y)$ là chi phí tăng thêm khi nhóm từ $x + 1$ đến y được phủ, sau khi trừ phần giao với nhóm trước. Khi đó

$$f_{c,i} = \min_j \{f_{c-1,j} + w(j, i)\}.$$

Hàm w thỏa bất đẳng thức tứ giác, nên $f_{c,i}$ có tính lồi theo số đoạn c . Nhờ vậy có thể WQS trước để bỏ chiều số đoạn khỏi trạng thái; phần dp cuối cùng dùng hàng đợi nhị phân hoặc tối ưu hóa đường thẳng, đạt $O(n \log n \log m)$ hoặc $O(n \log^2 m)$ tùy cài đặt.

3.3. Bao hàm–loại trừ

Dùng bao hàm–loại trừ để chuyển hóa bài toán đếm là một kỹ thuật rất thường gặp và nhiều tính kỹ xảo. Vì đã có nhiều bài viết chuyên sâu về bao hàm–loại trừ phức tạp, bài viết này không triển khai chi tiết.

3.4. Chuyển hóa cho một lớp bài toán đếm

Xét một lớp bài toán có hai loại đối tượng X, Y , cùng ánh xạ $A : X \times Y \rightarrow \{0, 1\}$. Cần đếm bao nhiêu $x \in X$ thỏa tồn tại $y \in Y$ sao cho $A(x, y) = 1$. Ở đây x là đối tượng cần đếm nhưng khó tính trực tiếp; y có thể hiểu là một nghiệm hoặc chứng nhận. Ta thường dễ đếm số y hợp lệ với một x , nhưng cộng trực tiếp theo mọi y sẽ đếm trùng. Một số cách chuyển hóa thường dùng như sau.

3.4.1. Đổi cách kiểm tra. Ta có thể đổi góc nhìn hoặc khai thác tính chất để biến điều kiện “ x hợp lệ” thành mô tả tính đơn giản hơn, từ đó dp trực tiếp.

Ví dụ 3.4.1 (Red and Blue Chips). Có các chồng chip đỏ và xanh theo một quy trình ghép cho trước; cần đếm những dãy màu cuối cùng có thể thu được. Điều kiện ban đầu có vẻ mang tính quá trình: khi thêm một chip mới, phải quyết định đặt nó vào đâu.

Ta mô tả quá trình theo dãy số: ban đầu có một chip xanh; mỗi lần có thể chèn một chip màu cho trước vào sau một chip xanh. Nếu số chip đỏ liên tiếp sau các chip xanh hiện tại lần lượt là $x_1, \dots, x_m$, thì chèn chip đỏ là chọn một x_i và tăng nó lên một; chèn chip xanh là chèn một số 0 vào dãy, trừ vị trí trước đầu dãy. Từ đó suy ra điều kiện cần và đủ trên dãy cuối cùng: các tổng lớn nhất, nhì lớn nhất, v.v. phải vượt các ngưỡng do thứ tự chip đỏ trong đầu vào quyết định.

Vì vậy chỉ cần quyết định đa tập $\{x_1, \dots, x_m\}$ theo thứ tự giá trị từ lớn xuống nhỏ. Đặt $f_{i,j,c}$ ghi sau khi xét các giá trị lớn, hiện còn bao nhiêu vị trí và bao nhiêu phần tử lớn hơn ngưỡng; khi chuyển trạng thái, liệt kê số phần tử bằng giá trị hiện tại và nhân tổ hợp để tính số hoán vị. Độ phức tạp $O(n^2)$.

3.4.2. Đếm có trọng số. Nếu dp theo y dễ hơn, ta có thể thiết kế trọng số $w(y)$ sao cho với mọi x ,

$$\sum_{y \in Y} A(x, y) w(y) = \exists y \in Y, A(x, y) = 1.$$

Khi đó đáp án là tổng của $w(y)$ nhân số x mà y chứng nhận được.

Một cách thiết kế là chọn đúng một đại diện y cho mỗi x hợp lệ và đặt trọng số đại diện bằng 1, các chứng nhận khác bằng 0. Một cách khác là cho một số trọng số âm, tức dùng bao hàm–loại trừ.

Ví dụ 3.4.2 (Range Argmax). Cho m đoạn con của $[1, n]$. Với một hoán vị p , đặt x_i là vị trí của phần tử lớn nhất trên đoạn thứ i . Cần đếm số dãy $(x_1, \dots, x_m)$ có thể sinh ra.

Với một dãy x khả dĩ, xét vị trí chứa giá trị lớn nhất n . Tồn tại vị trí j sao cho mọi đoạn chứa j đều có $x_i = j$. Nếu dãy khả dĩ có một hoán vị chứng nhận mà $p_j < n$, ta có thể đổi p_j với giá trị lớn hơn mà không làm thay đổi các x_i , lặp lại để đưa n về j . Vì vậy có thể chọn đại diện là vị trí cực đại ngoài cùng bên trái j có $p_j = n$.

Sau khi cố định j , mọi đoạn chứa j bị loại bỏ, hai phía trái và phải trở thành hai bài toán con độc lập; các cực đại phía trái còn phải tôn trọng điều kiện “ngoài cùng bên trái”. Đặt $f_{l,r,t}$ là đáp án chỉ xét các đoạn con trong $[l, r]$, với yêu cầu vị trí cực đại ngoài cùng bên trái phải lớn hơn t . Độ phức tạp $O(n^3)$.

3.4.3. Quy hoạch động trên quá trình kiểm tra. Xét một thuật toán kiểm tra một x có hợp lệ hay không. Thường thuật toán nhận x từng phần và lưu một số biến. Nếu trừu tượng thuật toán này thành một DFA, các giá trị trung gian của biến là trạng thái tự động, còn phản ứng với ký tự đầu vào là chuyển trạng thái. Khi đó chỉ cần dp theo phần x đã xác định và trạng thái hiện tại của tự động. Trường hợp đơn giản nhất chỉ cần thêm một biến Boolean; phức tạp hơn, thuật toán kiểm tra chính nó cũng là dp, thường gọi là dp lồng dp.

Không phải mọi bài toán đếm đều có DFA nhỏ. Khi DFA lớn, có thể cắt bỏ trạng thái không thể đạt hoặc trạng thái chắc chắn không thể được chấp nhận, gộp các lớp trạng thái tương đương, hoặc dùng ý nghĩa tổ hợp để phát hiện những chi tiết không cần phân biệt.

Ví dụ 3.4.3 (Phone Numbers). Cho xâu s độ dài n gồm các chữ số 1 đến 9. Cần đếm số xâu t cùng giới hạn sao cho tồn tại một cách chia đoạn chung, trong đó mỗi cặp đoạn tương ứng của s và t có cùng tập chữ số. Trong mỗi đoạn không có chữ số lặp, và các chữ số tạo thành một hình chữ nhật kích thước không quá 2 trên bàn phím số.

Một kiểm tra trực tiếp dùng các giá trị f_i cho biết tiền tố có thể chia hợp lệ hay không, và khi thêm t_i thì chuyển từ vài giá trị gần nhất. Nếu ghi thô, trạng thái gồm một số bit khả thi và các chữ số gần nhất của t , có hơn một vạn khả năng.

Quan sát rằng chỉ trong vài trường hợp đặc biệt mới cần ghi đủ hai hoặc ba chữ số gần nhất của t : cụ thể khi một tập chữ số gần đây của s thuộc một trong các tập hình chữ nhật hợp lệ và các chữ số của t nằm trong tập đó. Ngoài các trường hợp ấy có thể rút gọn xuống rất ít loại. Hoặc đơn giản hơn, dùng chương trình sinh DFA tối thiểu sau khi đưa cả vài chữ số gần nhất của s vào trạng thái. Với mỗi bộ chữ số gần đây của s , số trạng thái hiệu dụng chỉ cỡ vài chục, đủ để chạy trên n lớn.

Ví dụ 3.4.4 (Hoàng lạn kê). Cho các tập $A_i \subseteq \{1, \dots, m\}$. Cần đếm số dãy $a_i \in A_i$ sao cho với mọi đoạn con không rỗng, trọng số tập độc lập lớn nhất trên đường đi đoạn đó không bằng một nửa tổng trọng số.

Với một dãy đơn, có thể kiểm tra bằng hai giá trị f_0, f_1 : hiệu lớn nhất giữa tổng đã chọn và tổng không chọn, khi vị trí cuối không chọn hoặc được chọn. Nếu $\max(f_0, f_1) = 0$ thì không hợp lệ; giá trị này cũng không thể âm. Khi thêm trọng số w , chuyển trạng thái là

$$f'_0 = \max(f_0, f_1) - w, \quad f'_1 = f_0 + w.$$

Nếu quyết định dãy a từ trái sang phải và duy trì mọi đoạn kết thúc tại vị trí hiện tại, số trạng thái quá lớn. Ta bỏ qua mọi cặp (f_0, f_1) mà dù nối tiếp thế nào cũng không thể tạo đoạn không hợp lệ. Phân tích cho thấy chỉ cần xét các cặp dạng $(-1, 1), \dots, (-m, m)$, tổng cộng 2^m tập trạng thái. Đặt $g_{i,S}$ là số cách sau i phần tử, với tập các cặp còn cần theo dõi là S . Khi chọn a , tập mới có dạng

$$S' = \{a - x \mid x \in S, x < a\} \cup \{a\}.$$

Những giá trị lớn hơn a không ảnh hưởng đến chuyển trạng thái, nên có thể cộng gộp trước; độ phức tạp tối ưu còn $O(n2^m)$.

4. Tổng kết

Quá trình giải một bài toán quy hoạch động có thể chia đại khái thành bốn bước: quan sát tính chất, chuyển hóa mô hình, thiết kế dp, và tối ưu dp. Bốn bước này liên hệ và ảnh hưởng lẫn nhau. Do giới hạn dung lượng, bài viết chỉ nêu một phần phương pháp ở bước thứ hai và thứ ba; đây chỉ là phần rất nhỏ của bức tranh toàn cảnh.

Tác giả tự nhận khuôn khổ trong bài còn mơ hồ và thô. Không có phương thuốc vạn năng cho mọi bài toán, nhưng phân tích từng mô hình cụ thể cũng không phải là việc hoàn toàn vô chương pháp. Hy vọng bài viết có thể gợi mở để người đọc tự rút ra thêm những phương pháp có phạm vi áp dụng nhất định và tìm thấy những “tia sáng tư tưởng” trong thiết kế thuật toán.

Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn các thầy Wang Xinbo và Zhu Yixing đã hướng dẫn.

Cảm ơn mẹ đã nuôi dưỡng, cảm ơn gia đình đã quan tâm và ủng hộ.

Cảm ơn anh Chang Ruinian, bạn Zhu Juzhe và bạn Lin Shangyu đã thảo luận với tác giả về các chủ đề liên quan trong bài và đem lại gợi ý. Cảm ơn anh Chang Ruinian và bạn Zhu Juzhe đã đưa ra những góp ý quý báu cho bài viết.

Cảm ơn tất cả các bạn học và phụ huynh đã giúp đỡ, đồng hành trên con đường luyện tập.

Tài liệu tham khảo

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*, third edition, chapter on dynamic programming, pages 204–236. China Machine Press, 2009.
2. *Dynamic programming*. Wikipedia, https://en.wikipedia.org/wiki/Dynamic_programming.
3. Oleksandr Kulkov (adamant). *Theoretical grounds of lambda optimization*. Codeforces, <https://codeforces.com/blog/entry/98334>, 2021.
4. *Tối ưu hóa bằng bất đẳng thức tứ giác*. OI Wiki, <https://oi-wiki.org/dp/opt/quadrangle/>.
5. FOP. *Bàn về tính lồi của hàm số trong OI*. Tập luận văn đội tuyển dự tuyển quốc gia Trung Quốc năm 2023, 2023.
6. Xu Zhe'an. *Bàn về tự động hữu hạn và ứng dụng*. Tập luận văn đội tuyển dự tuyển quốc gia Trung Quốc năm 2021, 2021.
7. Cao Li. *Tổng hợp phương pháp cho bài toán dp*. WA, <https://www.luogu.com.cn/blog/yeah-potato/dp-ti-fang-fa-zong-hui>, 2023.

Bài 7. Bàn về một số cách xử lý hợp nhất thông tin

Feng Zhengwei, Trường Trung học Ba Thục, Trùng Khánh

Tóm tắt

Hợp nhất thông tin là một bộ phận rất quan trọng trong thi tin học. Có nhiều cách xử lý khác nhau cho quá trình hợp nhất này. Bài viết nhìn vấn đề từ hai góc độ: hợp nhất thông tin một cách trực tiếp, và hợp nhất thông tin thông qua cấu trúc dữ liệu. Từ các ví dụ cụ thể trong thuật toán thi đấu, bài viết thảo luận những thủ pháp thường gặp, cách phân tích độ phức tạp và các chi tiết dễ bị bỏ sót.

Tác giả cố gắng trình bày từ góc nhìn mộc mạc và thực dụng: chọn một số ví dụ có đặc điểm rõ rệt từng gặp trong quá trình luyện tập, rồi từ đó rút ra những cách xử lý và phân tích trên các cấu trúc cổ điển. Hy vọng các ví dụ này có thể gợi mở cho người đọc khi gặp những bài toán cần “gom”, “trộn” hoặc “tái cấu trúc” nhiều nguồn thông tin.

1. Kiến thức chuẩn bị

Trong bài viết, $\log n$ mặc định là logarit cơ số 2. “Thông tin” thường chỉ đối tượng cần duy trì trong bài toán cụ thể: một trọng số, một đỉnh, một cạnh, một biểu thức, một trạng thái hoặc một phần tử dữ liệu nào đó.

Quá trình hợp nhất thông tin có thể hiểu như việc xem thông tin là phần tử trong các tập hợp, rồi gộp nhiều tập theo một quy tắc nhất định để duy trì một cấu trúc mới. Cấu trúc đó phải cho phép ta tìm nhanh thông tin cần dùng. Mỗi loại thông tin kéo theo một cách hợp nhất và một kiểu phân tích khác nhau.

2. Hợp nhất thông tin trực tiếp

Hợp nhất trực tiếp là cách dùng thao tác tương đối bạo lực để gộp thông tin, nhưng kèm theo phân tích hợp lý để chứng minh độ phức tạp vẫn đúng. Phần này trình bày hai họ tư tưởng: hợp nhất theo kích thước và hợp nhất kiểu DSU on tree, sau đó dẫn tới phân nhóm nhị phân.

2.1. Hợp nhất gợi ý theo kích thước

Giả sử có nhiều tập thông tin, tổng số phần tử là m , và ta cần nhiều lần gộp hai tập thành một. Nếu có thể chèn từng phần tử vào một tập với chi phí nhỏ, ta luôn lấy tập nhỏ chèn vào tập lớn. Với hai tập kích thước $A \leq B$, ta rút lần lượt các phần tử của tập A rồi chèn vào tập B .

Phân tích rất đơn giản: mỗi khi một phần tử bị chèn lại, kích thước tập chứa nó ít nhất gấp đôi. Do đó mỗi phần tử bị chèn lại không quá $O(\log m)$ lần. Nếu một lần chèn tốn $O(F(m))$, tổng chi phí là

$$O(mF(m) \log m).$$

Ví dụ 1. Sinh thêm cạnh động. Trong một bài toán đồ thị động, mỗi lần thêm cạnh và cần biết hiện tại còn có bao nhiêu cạnh có thể được sinh thêm. Một cạnh (x, z) có thể sinh nếu ban đầu chưa tồn tại và có đỉnh y sao cho các cạnh liên quan tạo được quan hệ bắc cầu mong muốn; sau khi sinh thêm cạnh, quá trình này có thể tiếp tục.

Nhận xét chính là các thành phần liên thông bởi cạnh hai chiều có thể được bổ sung thành đồ thị đầy đủ. Ta duy trì các thành phần liên thông đó và các cạnh một chiều còn lại giữa các khối. Cạnh một chiều có thể lưu bằng bảng băm từ khối sang khối, đồng thời duy trì cả cạnh đi ra và cạnh đi vào mỗi khối. Khi thêm cạnh làm hai khối phải hợp nhất, các bảng tương ứng được gộp bằng hợp nhất gợi ý. Nhờ phân tích ở trên, tổng độ phức tạp đạt $O(n \log n)$.

Kết hợp với chia đôi toàn cục. Hợp nhất gợi ý chỉ bảo đảm chi phí trung bình; một lần hợp nhất riêng lẻ không có cận chặt. Vì vậy trong một số thủ tục không hoàn toàn “đơn điệu”, ví dụ cần hoàn tác, không thể dùng lập luận một cách máy móc. Tuy nhiên đôi khi bài toán nhìn như phá vỡ phân tích, nhưng nếu đổi góc nhìn thì toàn bộ quá trình vẫn nghiêm ngặt.

Ví dụ 2. Đường đi có trọng số là cạnh lớn thứ hai. Với nhiều truy vấn đường đi giữa x và y , trọng số đường đi được định nghĩa là cạnh lớn thứ hai trên đường đi; nếu chỉ qua một cạnh thì giá trị này là 0. Ta cần cực tiểu hóa giá trị đó. Sau khi tách riêng cạnh lớn nhất, bài toán trở thành kiểm tra liệu có thể dùng các cạnh trọng số trong $[1, \text{mid}]$ cộng thêm một cạnh ngoài để làm x, y liên thông hay không. Với nhiều truy vấn, dùng chia đôi toàn cục.

Trong mỗi bước kiểm tra, ta dùng DSU duy trì thành phần liên thông và dùng hợp nhất gợi ý duy trì quan hệ “có cạnh ngoài” giữa các thành phần. Quy trình có hoàn tác: xử lý nửa phải, quay lui rồi xử lý nửa trái. Phân tích không dựa vào từng lần hoàn tác, mà dựa vào việc mỗi đỉnh, trong một tầng chia đôi, được thêm vào theo cùng một thứ tự; toàn bộ tương đương với $O(\log n)$ lần chạy hợp nhất gợi ý trên các tiền tố. Vì vậy độ phức tạp vẫn đúng.

Tư tưởng giống hợp nhất gợi ý. Nhiều bài sau biến đổi có thể mô tả thành hợp nhất các tập thông tin, dù thao tác gộp không phải là chèn phần tử đơn thuần. Khi đó ta vẫn có thể bắt chước phân tích “mỗi phần tử trả tiền khi tập của nó tăng đáng kể”.

Ví dụ 3. Chọn DFS order trên cây nhị phân có trọng số. Cho một cây nhị phân có trọng số cạnh, mỗi nút trong có đúng hai con. Cần dựng một thứ tự DFS của các lá sao cho giá trị lớn nhất của khoảng cách giữa hai lá kề nhau trong thứ tự là nhỏ nhất. Sau khi nhị phân hóa đáp án, ta làm dp trên cây. Với một nút x , trạng thái có thể xem là tập các cặp (a, b) : a là khoảng cách đi vào cây con, b là khoảng cách rời cây con. Các cặp bị trội có thể loại bỏ: nếu (a, b) và (c, d) thỏa $a \leq c$ và $b \leq d$, thì (c, d) không cần giữ.

Khi gộp hai cây con, giả sử tập nhỏ là S , tập lớn là T . Sau rút gọn, kích thước tập mới có thể chặn bởi $2|S|$. Ta xem phần tử của S được trả một chi phí hằng và đi vào một tập có kích thước tăng gấp đôi, còn

phần tử dư trong T bị loại bỏ trả chi phí hằng. Lập luận giống hợp nhất gợi ý cho tổng chi phí $O(n \log n)$ sau khi kiểm tra một đáp án.

2.2. Hợp nhất gợi ý trên cây

DSU on tree thường dùng để duy trì thông tin của mỗi cây con. Với mỗi đỉnh, ta chọn con có cây con lớn nhất làm con nặng, các con còn lại là con nhẹ. Khi DFS, ta giữ lại thông tin của con nặng và chèn lại thông tin từ các con nhẹ vào cấu trúc chung. Một đỉnh chỉ bị chèn lại khi đường từ nó lên gốc đi qua cạnh nhẹ; sau mỗi cạnh nhẹ, kích thước cây con ít nhất gấp đôi, nên số lần chèn lại không quá $O(\log n)$.

Cũng có thể nhìn DSU on tree như việc mỗi cây con duy trì một cấu trúc riêng, rồi khi gộp hai cây con thì chèn cây nhỏ vào cây lớn. Hai cách về bản chất giống nhau, nhưng cách giữ một cấu trúc toàn cục có lợi khi thông tin được lưu bằng mảng, bộ đếm hoặc thùng tần suất, vì việc thêm và xóa dễ kiểm soát hơn.

Ví dụ 4. Đường đi có thể sắp thành palindrome. Cho cây kích thước 10^5 , mỗi cạnh mang một chữ cái trong 22 chữ cái đầu của bảng chữ cái thường. Với mỗi cây con, cần tìm đường đi dài nhất sao cho các ký tự trên đường có thể sắp lại thành palindrome.

Một đường đi hợp lệ khi nhiều nhất một ký tự xuất hiện số lần lẻ. Mã hóa trạng thái parity của 22 ký tự bằng bitmask. Với mỗi cây con, lưu cho từng mask độ sâu lớn nhất đạt được từ gốc cây con. Khi gộp một cây con nhẹ vào cấu trúc đang giữ, ta thử mask hiện tại và các mask khác đúng một bit để cập nhật đáp án. Vì dùng một mảng toàn cục cho các mask, thao tác hoàn tác và xóa thông tin rất thuận tiện. Tổng chi phí là $O(cn \log n)$, trong đó c là kích thước bảng chữ cái.

Dùng hợp nhất để liệt kê LCA. Quá trình DSU on tree có thể xem là liệt kê LCA rồi gộp các tập liên quan. Điều này hữu ích cho các bài toán cần dùng thông tin ở LCA.

Ví dụ 5. Các chuỗi trên cây. Với một họ đường đi trên cây và tham số k , cần đếm hoặc kiểm tra giao giữa các đường đi dưới một điều kiện độ dài. Nếu hai đường giao nhau có LCA khác nhau, ta liệt kê đường có LCA sâu hơn; giao luôn có dạng đoạn đi lên rồi đi xuống từ LCA với độ dài đủ lớn, có thể dùng cây chủ tịch trên thứ tự DFS để truy vấn. Nếu hai đường có cùng LCA, ta dựng cây ảo trên các đầu mút của những đường có LCA bằng đỉnh đó, rồi dùng hợp nhất gợi ý để liệt kê vị trí LCA ở phía trái. Từ đó dùng binary lifting tìm điểm tương ứng và hợp nhất cây đoạn để duy trì khoảng DFS của cây con cần hỏi. Chi tiết cài đặt có nhiều biên, nhưng điểm mấu chốt là dùng quá trình hợp nhất để kiểm soát số lần một đầu mút được xử lý.

Những cách dựng cây ảo rồi liệt kê LCA tương tự được trình bày thêm trong tài liệu tham khảo của bài gốc.

2.3. Phân nhóm nhị phân

Phân nhóm nhị phân là một cách chuyển từ hợp nhất trực tiếp sang duy trì nhiều cấu trúc tĩnh nhỏ.

Ví dụ 6. Tập chuỗi động và truy vấn xuất hiện. Có tối đa $3 \cdot 10^5$ thao tác: thêm một chuỗi, xóa một chuỗi đang tồn tại, hoặc cho chuỗi s và hỏi tổng số lần các chuỗi hiện có xuất hiện trong s . Tổng độ dài mọi chuỗi đầu vào cũng không quá $3 \cdot 10^5$, và thao tác bắt buộc online.

Trước hết, xóa có thể tách khỏi thêm: lập một tập thứ hai cho các chuỗi bị xóa, rồi đáp án bằng số lần xuất hiện trong tập thêm trừ đi số lần xuất hiện trong tập xóa. Vì vậy chỉ cần xét thêm và hỏi.

Với một tập chuỗi tĩnh, AC automaton giải trọn vẹn bài toán truy vấn. Vấn đề là AC automaton không thuận tiện cho thêm online. Một cách cân bằng đầu tiên là chia khối: giữ một nhóm chuỗi lẻ chưa xây automaton, khi tổng độ dài vượt ngưỡng thì xây lại, còn khi hồi thì kết hợp truy vấn automaton với KMP trên các chuỗi lẻ. Chọn ngưỡng thích hợp cho độ phức tạp cỡ căn.

Cách thứ hai là phân nhóm nhị phân. Mỗi nhóm có một AC automaton. Khi thêm một chuỗi, tạo nhóm mới; nếu có hai nhóm cùng số chuỗi, gộp chúng và xây lại automaton cho nhóm lớn hơn. Tại mọi thời điểm chỉ có $O(\log n)$ automaton. Mỗi chuỗi chỉ bị xây lại khi nhóm chứa nó tăng gấp đôi, nên tổng chi phí xây dựng là $O(cn \log n)$, với c là kích thước bảng chữ cái. Cách nhìn này tương đương với một cây đoạn được xây dần theo thứ tự thêm lá.

3. Hợp nhất thông tin theo cấu trúc

Sau các phương pháp trực tiếp, phần này xét những cấu trúc dữ liệu vốn đã hỗ trợ hợp nhất hoặc có thể được biến đổi để hợp nhất tốt hơn: leftist heap, cây cân bằng, treap không xoay và cây đoạn hợp nhất.

3.1. Leftist heap

Leftist heap duy trì một tập trọng số, hỗ trợ lấy phần tử cực trị và gộp nhanh hai heap. Cấu trúc vẫn là heap, tức khóa của nút cha có quan hệ thứ tự không nghiêm với khóa của nút con. Để bảo đảm độ phức tạp gộp, mỗi nút duy trì dist , với

$$\text{dist}(0) = 0, \quad \text{dist}(\text{left}(x)) \geq \text{dist}(\text{right}(x)), \quad \text{dist}(x) = \text{dist}(\text{right}(x)) + 1.$$

Khi gộp, so sánh khóa hai gốc, giữ gốc phù hợp, gộp đệ quy con phải với heap còn lại, sau đó đổi hai con nếu cần để khôi phục tính chất leftist và cập nhật dist . Với cây con kích thước A , có thể chứng minh $\text{dist} \leq \log(A + 1)$, nên gộp tốn $O(\log n)$.

Ví dụ 7. Quy hoạch động lỗi trên cây. Trong một bài toán chọn đỉnh đen trên cây, chi phí cạnh là độ chênh số đỉnh đen ở hai phía. Quy hoạch động theo cây có trạng thái $f_{x,y}$: trong cây con của x chọn y đỉnh đen. Với mỗi cạnh, ảnh hưởng lên hàm theo y có dạng lỗi; khi gộp hai cây con là min-plus convolution của hai hàm lỗi, tức dãy sai phân vẫn có thứ tự. Ta duy trì dãy sai phân bằng hai heap quanh điểm chia, một heap lớn và một heap nhỏ, đồng thời cần cộng/trừ toàn cục cho cả heap và gộp heap giữa các cây con. Leftist heap đáp ứng đúng các thao tác này.

3.2. Hợp nhất cây cân bằng

Cây cân bằng thường duy trì tập có thứ tự và hỗ trợ truy vấn, cập nhật theo khoảng, tính tổng, v.v. Hợp nhất hai cây cân bằng là lấy hợp của hai tập rồi nhận được một cây mới. Trong thực hành OI, cách phổ biến vẫn là chèn cây nhỏ vào cây lớn vì dễ viết. Tuy nhiên có những cách hợp nhất có độ phức tạp tốt hơn dựa trên finger search.

3.2.1. Finger search

Finger search là biến thể của tìm kiếm: ta giữ phần tử tìm được ở lần trước, gọi là *finger*, rồi tìm phần tử tiếp theo từ vị trí đó. Với dãy có thứ tự, gọi $d(x, y)$ là chênh lệch thứ hạng giữa x và y ; nếu tìm từ *finger* x tới y với chi phí phụ thuộc vào $d(x, y)$, chẳng hạn $O(f(d(x, y)))$, thì đó là finger search. Finger search tree là cây tìm kiếm nhị phân lưu con trỏ tới nút thao tác lần trước và dùng nó như điểm xuất phát.

3.2.2. Splay

Splay là cây tìm kiếm nhị phân tự điều chỉnh bằng xoay. Sau mỗi thao tác, nút vừa thao tác được xoay lên gốc, nên gốc có thể xem là *finger*. Các kết quả kinh điển về *dynamic finger* cho biết chi phí một thao tác có thể chặn bởi $O(\log d)$, trong đó d là chênh lệch thứ hạng so với thao tác trước.

Khi hợp nhất hai splay, lấy cây nhỏ theo thứ tự trung tố rồi chen lần lượt vào cây lớn. Giả sử cây nhỏ có m phần tử, cây lớn có n phần tử, và khoảng cách thứ hạng giữa hai phần tử liên tiếp được chen là d_i . Lần chen đầu tốn $O(\log n)$; các lần sau tốn xấp xỉ $\sum \log d_i$. Dùng bất đẳng thức trung bình, tổng này không vượt quá $O(m \log(n/m))$ ở một lần hợp nhất. Cộng qua các lần một phần tử chuyển sang tập lớn hơn, tổng chi phí toàn cục là $O(n \log n)$.

3.2.3. Treap có xoay

Treap gắn mỗi khóa với một ưu tiên ngẫu nhiên: khóa thỏa tính chất cây tìm kiếm nhị phân, ưu tiên thỏa tính chất heap. Có thể nhìn treap như dựng cây bằng cách chọn phần tử có ưu tiên cao nhất làm gốc rồi chia hai phía; do chia ngẫu nhiên, chiều cao kỳ vọng là $O(\log n)$. Kết quả về treap cũng cho khoảng cách kỳ vọng giữa hai khóa x, y là $O(\log d(x, y))$.

Để hợp nhất, lấy cây nhỏ theo trung tố và chen vào cây lớn, đồng thời giữ phần tử vừa chen làm *finger*. Khi cần chen y sau $x < y$, từ nút x đi lên để tìm LCA của x và y , rồi đi xuống tới vị trí của y . Nếu đi lên một cách thô, ta có thể duyệt thêm một đoạn không liên quan tới khoảng cách thật giữa x và y . Bài gốc minh họa tình huống này bằng một hình treap. Cách sửa là vừa đi lên để cập nhật ứng viên LCA, vừa đồng thời thử đi xuống tìm y ; mỗi lần đổi ứng viên thì khởi động lại nhánh tìm kiếm. Khi đó số nút duyệt không quá hằng số lần khoảng cách cây giữa x và y . Suy ra một lần gộp hai cây kích thước $m \leq n$ có chi phí kỳ vọng $O(m \log(n/m))$, và tổng chi phí toàn cục vẫn là $O(n \log n)$.

3.2.4. Treap không xoay

Treap có xoay không thuận tiện cho khả năng lưu phiên bản và cài đặt cũng dài. Với treap không xoay, các thao tác cơ bản là:

- $\text{Split}(A, \text{val})$: tách A thành phần khóa nhỏ hơn val , phần khóa lớn hơn val , và nút bằng val nếu có;
- $\text{Merge}(A, B)$: gộp hai cây khi mọi khóa của A nhỏ hơn mọi khóa của B ;
- $\text{Union}(A, B)$, $\text{Intersection}(A, B)$ và $\text{Difference}(A, B)$: lấy hợp, giao và hiệu tập.

Với Union , giả sử gốc của A có ưu tiên cao hơn. Lấy khóa của gốc A làm chốt, dùng Split tách B thành phần nhỏ hơn, bằng và lớn hơn chốt; bỏ phần bằng nếu cần, rồi đệ quy gộp hai phía với hai con của gốc A . Với hai cây kích thước $m \leq n$, các thao tác tập này có độ phức tạp kỳ vọng $O(m \log(n/m))$; chi tiết chứng minh được trình bày trong bài về set operations dùng treap của Blelloch và Reid-Miller.

Khi khóa có thể trùng nhau, cách an toàn là gắn thêm nhãn phụ để biến khóa thành duy nhất. Nếu chỉ sửa Split thành tách theo $< \text{val}$ và $> \text{val}$, rồi để nhiều phần tử cùng khóa cùng tồn tại, Union có thể làm chiều cao suy biến. Trường hợp cực đoan là mọi khóa của hai cây đều bằng nhau: mỗi lần tách chỉ nối phần còn lại vào một nhánh, khiến độ dài chuỗi phải của cây kết quả bằng tổng độ dài hai chuỗi cũ, không còn bảo đảm chiều cao kỳ vọng $O(\log n)$.

3.3. Hợp nhất cây đoạn

Cây đoạn cũng duy trì tập thông tin có thứ tự. So với cây cân bằng, ưu điểm là cấu trúc rõ, hằng số nhỏ, dễ phân tích và dễ mở rộng. Khi hợp nhất hai cây đoạn có cùng miền giá trị, nếu một nút rỗng thì lấy

luôn nút còn lại; nếu cả hai tồn tại thì gộp hai con và cập nhật thông tin. Mỗi lần hai nút thật gộp thành một nút thật, số nút giảm đi một. Nếu tổng số lần chen điểm là $O(n)$, tổng số nút được tạo là $O(n \log n)$, nên tổng chi phí hợp nhất cũng là $O(n \log n)$.

Ví dụ 8. Đếm cách gán cạnh trên cây. Cho cây có gốc và nhiều ràng buộc (u, v) , trong đó u là tổ tiên của v . Cần đếm số cách gán 0/1 cho mỗi cạnh sao cho với mỗi ràng buộc, trên đường từ u tới v có ít nhất một cạnh mang 1.

Dùng bao hàm–loại trừ: khi chọn một tập ràng buộc bị vi phạm, các cạnh trên đường tương ứng bị ép bằng 0, và hệ số đổi dấu theo số ràng buộc. Làm dp trên cây, với $f_{x,j}$ biểu thị sau khi xử lý cây con của x , mọi cạnh trong đoạn độ sâu $[\text{dep}_x, j]$ đang bị ép bằng 0. Khi gộp hai cây con, công thức có dạng nhân chéo và cộng theo độ sâu; cây đoạn cần duy trì tổng và nhân nhân. Gộp theo độ sâu từ lớn xuống nhỏ, không cần tạo nút mới ngoài các nút đã có, nên tổng độ phức tạp là $O(n \log n)$.

3.3.1. Khi đẩy nhãn phải tạo nút

Trong một số bài, cây đoạn hợp nhất đi kèm cập nhật đoạn; khi đẩy nhãn xuống cần tạo nút mới. Khi đó không thể chỉ nói “mỗi lần gộp giảm một nút” mà phải phân tích số nút mới sinh.

Ví dụ 9. Liên thông khối trên cây theo trọng số. Cho cây $n \leq 1500$, mỗi đỉnh có trọng số. Với mọi khối liên thông kích thước ít nhất k , cần xét các tổng trọng số lớn nhất theo thứ tự giảm dần. Sau khi rời rạc hóa trọng số $w_1 < w_2 < \dots < w_n$, ta tính đóng góp của từng ngưỡng: đếm số khối liên thông chứa ít nhất k đỉnh có trọng số không nhỏ hơn ngưỡng.

Đặt $f_{x,i}$ là số khối liên thông trong cây con x , chứa x , và chứa ít nhất i đỉnh vượt ngưỡng. Chuyển trạng thái giữa các cây con là convolution. Viết dưới dạng đa thức rồi tính trên các điểm giá trị $s \in [0, n]$, phép gộp trở thành nhân từng điểm. Khi thay đổi ngưỡng theo thứ hạng trọng số của đỉnh x , trạng thái ban đầu chỉ là các đoạn hằng; ta dùng cây đoạn với cộng đoạn và nhân đoạn, còn gộp hai cây con là gộp cây đoạn theo từng vị trí.

Điểm quan trọng là hình dạng cây đoạn ban đầu có tính chất: mỗi nút hoặc có đủ hai con, hoặc không có con. Vì vậy khi đẩy nhãn xuống trong quá trình này không sinh thêm nút ngoài số có thể chặn trước. Tổng số nút vẫn là $O(n \log n)$, và độ phức tạp cuối cùng là $O(n^2 \log n)$ theo cách quét các điểm giá trị.

Nếu bỏ tính chất trên, có thể tưởng rằng gộp hai chuỗi nút đơn sẽ làm mỗi lần đệ quy sâu $O(\log n)$ nhưng chỉ giảm một nút, dẫn đến $O(n \log^2 n)$. Bài gốc chỉ ra cách nhìn đúng: những nút mới chỉ xuất hiện để “bổ sung” các nút trước đó chỉ có một con. Nếu ngay từ đầu ta coi các vị trí này đã được bổ đủ, số nút tiềm năng vẫn là $O(n \log n)$, nên tổng chi phí không tăng.

3.3.2. Tối ưu không gian cho cây đoạn hợp nhất

Thông thường cây đoạn động với $O(n)$ lần chen điểm trên miền $O(n)$ có cận thô $O(n \log n)$ nút. Nhưng nếu xét một cây đoạn duy nhất trên miền $O(n)$, tổng số nút thật chỉ là $O(n)$. Điều này gợi ý thay đổi thứ tự hợp nhất và tái sử dụng nút bị xóa để đạt cận không gian chặt hơn.

Xét dạng bài trên một cây: mỗi đỉnh có một số thao tác chen điểm hoặc cập nhật đoạn, và với mỗi cây con cần biết kết quả hợp nhất mọi thao tác trong cây con. Cập nhật đoạn $[L, R]$ có thể xem gần như hai thao tác chen điểm trên các đường từ gốc cây đoạn xuống L và R , bổ sung các nhánh còn thiếu.

Ta làm heavy-light theo số thao tác của các cây con, tương tự DSU on tree: xử lý con nặng trước và giữ lại cây đoạn của nó; sau đó xử lý từng con nhẹ, gộp cây đoạn của con nhẹ vào cây đang giữ và thu hồi nút không còn dùng. Ở mọi thời điểm chỉ còn $O(\log n)$ cây đoạn sống. Nếu cây thứ i có nhiều nhất 2^i

thao tác, thì trên mỗi tầng của cây đoạn số nút mới bị chặn bởi $\min(2^i, 2^\ell)$. Cộng theo các tầng và các cây đang sống, tổng số nút đồng thời chỉ là $O(n)$.

4. Tổng kết

Bài viết giới thiệu một số cách xử lý hợp nhất thông tin từ hai phía. Nhóm trực tiếp gồm hợp nhất gợi ý, DSU on tree, liệt kê LCA trong quá trình hợp nhất và phân nhóm nhị phân. Nhóm cấu trúc gồm leftist heap, hợp nhất cây cân bằng thông qua finger search, splay, treap có xoay, treap không xoay, và cây đoạn hợp nhất kể cả trường hợp phải tạo nút mới hoặc cần tối ưu không gian.

Điểm chung của các phương pháp là không chỉ cài đặt được thao tác gộp, mà còn phải tìm đúng đơn vị để phân tích chi phí: phần tử, nút cấu trúc, số lần kích thước tăng gấp đôi, hoặc số nút tiềm năng được bổ sung. Một khi chọn đúng đại lượng thể năng, nhiều thủ pháp tưởng như bạo lực lại có độ phức tạp tốt và rất hữu dụng trong thi thuật toán.

Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn cha mẹ đã nuôi dưỡng.

Cảm ơn các thầy Jiang Zongpei và Huang Xinjun của Trường Trung học Ba Thục đã hướng dẫn và quan tâm.

Cảm ơn bạn Xiao Ziqi của Trường Trung học Nhã Lễ đã giúp đỡ trong quá trình viết bài.

Cảm ơn các bạn Ma Weiran, Xiao Zigong, Wu Chang, Guo Yuhao, Zhou Ziheng, Yan He và Luo Siyuan đã đọc duyệt bài viết.

Cảm ơn mọi độc giả đã dành thời gian đọc bài.

Tài liệu tham khảo

1. Ren Zhizhou. *Bàn về tư tưởng gợi ý trong thi tin học*. Tập luận văn đội tuyển quốc gia Trung Quốc năm 2015.
2. Zhou Zhendong. *Bàn về một lớp bài toán liên quan tới đường đi trên cây*. Tập luận văn đội tuyển quốc gia Trung Quốc năm 2021.
3. OI Wiki. *DSU on tree*. <https://oi-wiki.org/graph/dsu-on-tree/>.
4. xiaoziyao. *Bàn về một lớp thủ pháp tái cấu trúc bạo lực thông tin*. <https://www.cnblogs.com/xiaoziyao/p/17413029.html>.
5. Xu Haoran. *Bàn về vài lời giải phi kinh điển cho bài cấu trúc dữ liệu*. Tập luận văn đội tuyển quốc gia Trung Quốc năm 2013.
6. OI Wiki. *Leftist tree*. <https://oi-wiki.org/ds/leftist-tree/>.
7. Wikipedia. *Finger search*. https://en.wikipedia.org/wiki/Finger_search.
8. Wikipedia. *Finger search tree*. https://en.wikipedia.org/wiki/Finger_search_tree.
9. D. D. Sleator and R. E. Tarjan. *Self-adjusting binary search trees*. Journal of the ACM.
10. R. Cole. *On the dynamic finger conjecture for splay trees. Part II: The proof*. SIAM Journal on Computing, 30(1):44–85, 2000.

11. R. Cole, B. Mishra, J. Schmidt, and A. Siegel. *On the dynamic finger conjecture for splay trees. Part I: Splay sorting log n -block sequences*. SIAM Journal on Computing, 30(1):1–43, 2000.
12. R. Seidel and C. R. Aragon. *Randomized search trees*. Algorithmica, 16(4/5):464–497, 1996.
13. Gerth Stolting Brodal. *Finger Search Trees*. University of Aarhus, 2005.
14. Guy E. Blelloch and Margaret Reid-Miller. *Fast set operations using treaps*. 10th Annual ACM Symposium on Parallel Algorithms and Architectures, Puerto Vallarta, Mexico, 1998.
15. Dong Weishou. *Bàn về tính chất và ứng dụng của Splay và Treap*. Tập luận văn đội tuyển quốc gia Trung Quốc năm 2018.
16. Joseph. *Finger-search và hợp nhất gợi ý*. Luogu Blog, <https://www.luogu.com.cn/blog/ICANTAKIOI/>.
17. dpair. *Ghi chép một trick về hợp nhất cây đoạn*. <https://dpair.gitee.io/articles/sgt-mret/>.

3 Chủ đề chính

Từ mục lục, tập năm 2024 bao phủ các nhóm chủ đề sau:

- hàm nhân tính, số học, liên phân số, thuật toán Euclid và q -analogue;
- matroid tuyến tính, matching có trọng số đỉnh và tô màu danh sách;
- cấu trúc dữ liệu, cập nhật-truy vấn đoạn và các bài toán trên cây;
- quy hoạch động, hợp nhất thông tin, bao hàm–loại trừ và nghịch đảo;
- hàm sinh, quá trình ngẫu nhiên, đếm đường đi trên lưới;
- tối ưu hằng số trên phần cứng hiện đại, lý thuyết tính toán và độ phức tạp;
- lattice, phân tích đa thức hệ số nguyên, chu trình tỉ lệ nhỏ nhất và các báo cáo ra đề.

4 Ghi chú về OCR

Phần mục lục của tập 2024 trích xuất được rõ ràng, nhưng lớp văn bản thân bài không ổn định: nhiều đoạn trở thành mojibake và công cụ PDF báo cảnh báo phông chữ. Bài đầu tiên ở trên được dịch từ OCR tại `/tmp/oipl-ocr/2024-p9-18.txt` và đối chiếu với ảnh render các trang PDF 9–18. Bài thứ hai được dịch từ lớp văn bản PDF và OCR tại `/tmp/oipl-2024-parity/`, đối chiếu với ảnh render PDF trang 19–31, tương ứng trang in 11–23. Một số công thức dài trong phần 4 được giữ ở mức có thể xác nhận từ ảnh trang; các biến phụ như $\omega(m)$ được hiểu theo ký hiệu chuẩn là số lượng thừa số nguyên tố phân biệt. Bài thứ ba được dịch từ OCR và lớp văn bản PDF tại `/tmp/oipl-2024-ds/`, đối chiếu với ảnh render PDF trang 32–44, tương ứng trang in 24–36; hình minh họa Top Tree trong nguồn được diễn giải bằng lời thay vì vẽ lại. Bài thứ tư được dịch từ OCR và lớp văn bản PDF tại `/tmp/oipl-2024-lagrange/`, đối chiếu với ảnh render PDF trang 45–61, tương ứng trang in 37–53; một số tên riêng trong phần cảm ơn được phiên âm theo kết quả OCR và ảnh trang. Bài thứ năm được dịch từ lớp văn bản PDF và ảnh render tại `/tmp/oipl-2024-range/`, đối chiếu với ảnh render PDF trang 62–76, tương ứng trang in 54–68; các hình minh họa phân lớp tương đương, chuyển đổi lớp tương đương và đường quét xoay được diễn giải bằng lời thay vì nhập ảnh. Công thức biến đổi ở ví dụ UNR #4 trong nguồn có dấu hiệu thiếu nhất quán với mô tả “bán kính R_i ”; bản dịch dùng phép biến đổi đại số theo mô tả hình học đó. Bài thứ sáu được dịch từ OCR tại `/tmp/oipl-2024-dp/` và đối chiếu với ảnh render PDF trang 77–92, tương ứng trang in

69–84; lớp văn bản nhúng bị mojibake nên các công thức và tên bài ví dụ dài chỉ được giữ ở mức xác nhận được từ ảnh trang, còn các công thức quá nhiều được diễn giải bằng lời. Bài thứ bảy được dịch từ OCR tại /tmp/oip1-2024-merge/ và đối chiếu với ảnh render PDF trang 93–107, tương ứng trang in 85–99; lớp văn bản nhúng của lát cắt này cũng bị mojibake, nên các công thức dài trong phần hợp nhất cây cân bằng và cây đoạn được giữ ở mức có thể xác nhận, còn ba hình minh họa trong nguồn được diễn giải bằng lời thay vì nhập ảnh hoặc để lại chú thích rời.